

Universidad de Oviedo



ESCUELA POLITÉCNICA DE INGENIERÍA DE GIJÓN

MÁSTER UNIVERSITARIO EN INGENIERÍA INFORMÁTICA

ÁREA DE ARQUITECTURA Y TECNOLOGÍA DE COMPUTADORES

TRABAJO FIN DE MÁSTER

Sistema de vectorización multimodal para la recuperación semántica de eventos en contenido audiovisual

D. Sergio Botana Suárez

TUTOR: *D. Rubén Usamentiaga Fernández*

COTUTOR: *D. Oscar Díaz Pedrayes*

FECHA: noviembre 2026



Universidad de Oviedo



DECLARACIÓN DE ORIGINALIDAD

D. Sergio Botana Suárez con DNI 53783252Y como alumno del Máster Universitario en Ingeniería Informática de la Universidad de Oviedo, DECLARO que el Trabajo Fin de Máster titulado
Sistema de vectorización multimodal para la recuperación semántica de eventos en contenido audiovisual es de mi autoría, es original y las fuentes bibliográficas utilizadas han sido debidamente citadas.

En Gijón, a 31 de noviembre de 2026

Firmado: Sergio Botana Suárez

Índice

1 Introducción **7**
1.1 Contexto y Motivación . . . 7
1.2 Planteamiento del Problema . . . 7
1.3 Propuesta y Objetivos del Trabajo Fin de Máster . . . 7

2 Revisión de la Literatura **9**
2.1 Fundamentos Matemáticos de los Embeddings Multimodales y Espacios Semánticos Continuos . . . 9
2.1.1 El Problema del Análisis Secuencial y la Representación de Datos No Estructurados . . . 9
2.1.2 El Ocaso de las Canalizaciones OCR y la Ingestión Visual Nativa con M-RoPE 10
2.1.3 Geometría del Espacio Latente y Métricas de Similitud Estadística . . . 11
2.1.4 Aprendizaje de Representación Matryoshka (MRL) y Normalización L_2 . . . 12
2.1.5 Análisis del Compromiso entre Precisión Semántica y Coste de Almacenamiento 14
2.2 Funcionamiento y Taxonomía de Modelos de Embeddings Multimodales . . . 15
2.2.1 Evolución Arquitectónica: De los Modelos Alineados (CLIP e ImageBind) a la Omnimodalidad Nativa . . . 15
2.2.2 Tokenización Visual, Codificación M-RoPE y Agregación Temporal de Fotogramas . . . 16
2.2.3 Arquitecturas de Doble Torre (*Bi-Encoder*) para Recuperación Masiva Precomputada . . . 17
2.2.4 Arquitecturas de Torre Única (*Cross-Encoder / Reranker*) y Atención Cruzada . 18
2.2.5 Eficiencia Escalable: El Paradigma de Búsqueda Asimétrica en Dos Etapas (Regla 99 % / 1 %) . . . 19
2.3 Estudio Profundo de Google Gemini Embedding 2 . . . 20
2.3.1 Arquitectura Omnimodal Unificada y Configuración del Espacio de Salida . . 20
2.3.2 Paradigmas de Ingestión Multimedia en el SDK `google-genai`: API Files frente a Agregación en Memoria . . . 21
2.3.3 Modelado Matemático de Cuotas Operativas (RPM, TPM y RPD) y Planificación de Capacidad . . . 23
2.3.4 Ingeniería de Resiliencia: Control de Concurrencia, Ventana de *Backoff* de 60 Segundos y Degradación Elegante . . . 25
2.4 Ecosistema de Modelos Alternativos: Pesos Abiertos (*Open-Weights*) y Ejecución Local 28
2.4.1 Las Dos Macrotendencias de 2026: APIs Propietarias Hiper-escaladas frente a Pesos Abiertos . . . 28
2.4.2 La Familia Qwen3-VL: Recuperación y Reordenación Multimodal a Escala Masiva . . . 28
2.4.3 Adaptación de Modelos Visuales (ColQwen y Sentence Transformers) y Entornos Restringidos (*Multi-RAG*) . . . 30
2.4.4 Evaluación en Bancos de Pruebas Masivos (MMEB-V2 y MVEB) y Compromisos de Despliegue . . . 31

2.5 Sistemas RAG Multimodales (mRAG) y Comprensión de Vídeo de Larga Duración . 33
2.5.1 Arquitecturas de Referencia en Comprensión de Vídeo: *Video-RAG*, *CARVE* y *Multi-RAG* 33
2.5.2 Del Emparejamiento Semántico al Anclaje Causal Espaciotemporal (*Temporal Grounding*) 34
2.5.3 Cuellos de Botella Perceptivos en la Verificación Visual: Resolución Espacial y Compresión JPEG 35
2.5.4 Dilución Semántica en Fragmentos de Larga Duración y Muestreo Estático de Fotogramas 37
2.6 Estrategias Avanzadas de Fragmentación (*Chunking*) y Alineación Temporal 38
2.6.1 Obsolescencia de la Partición Fija, Detección de Planos y Agrupación de Escenas 38
2.6.2 Metodología Híbrida *Video-RAC* y Formulación Matemática del SSIM Gaussiano 39
2.6.3 Selección de Fotogramas Clave mediante Entropía Matemática de Shannon . . 41
2.6.4 Alineación Forzada del Audio (*Forced Alignment*), Silero VAD y Diarización de Hablantes 42
2.6.5 Análisis Comparativo frente a Sistemas de Ingestión de Vídeo en Ventana Fija (*SentrySearch*) 43
2.7 Bases de Datos Vectoriales, Evaluación Empírica y Metodología de Revisión 46
2.7.1 Arquitectura de Bases de Datos Vectoriales e Indexación HNSW 46
2.7.2 Métricas Clásicas de Recuperación y Bancos de Pruebas Masivos (MMEB-V2 y MVEB) 47
2.7.3 Frameworks de Evaluación RAG y Atribución Verificable (RAGAS, ALCE, ARGUE, MIRAGE) 47
2.7.4 Metodología de Revisión Sistemática de la Literatura Asistida por NotebookLM 48
2.8 Evaluación Empírica de la Integración Audio-Visual y Verificación Bimodal 49
2.8.1 Análisis Cuantitativo sobre Corpus Contrastados: Locución Sincronizada frente a Metraje Desacoplado 50
2.8.2 Estudio de Casos Críticos: Resolución de Falsos Negativos, Resiliencia Fonética y Desambiguación Sintáctica 51
2.8.3 Análisis Experimental de la Divergencia Audio-Visual en Metraje de Archivo (*B-Roll*) y Deduplicación por Transcripción 54
2.8.4 Arquitectura de Consultas Multimodales Unificadas (`MultimodalQuery`) 55
2.8.5 Estudio de Ablación de Dimensionalidad Matryoshka (MRL): Compromiso entre Precisión Semántica, Memoria y Latencia (3,072 frente a 1,536 y 768) . . 56

Índice de figuras

1 Comparativa arquitectónica entre (a) la canalización tradicional basada en conversión intermedia a texto plano mediante OCR y (b) la proyección multimodal directa en un espacio semántico continuo mediante modelos VLM y codificación M-RoPE [1, 2]. 10
2 Geometría del espacio vectorial compartido $\mathbb{R}^d$: los vectores normalizados de distintas modalidades (consulta textual $\mathbf{q}_{\text{texto}}$, fotogramas de vídeo $\mathbf{v}_{\text{vídeo}}$ y pista de audio $\mathbf{v}_{\text{audio}}$) que describen un mismo evento convergen con un ángulo $\theta \to 0°$ sobre la hipersfera unitaria, permitiendo recuperar la marca temporal exacta mediante Similitud del Coseno [2]. 12
3 Estructura jerárquica de un vector entrenado mediante *Matryoshka Representation Learning* (MRL): las primeras dimensiones concentran la mayor carga semántica, permitiendo truncar el vector de salida para reducir exponencialmente el consumo de memoria RAM y latencia con penalizaciones mínimas de precisión [2, 3]. 13
4 Evolución taxonómica de los modelos de representación vectorial: desde la alineación estática dependiente de transcripción textual (CLIP/Whisper) hasta los modelos omnimodales nativos de tercera generación capaces de proyectar texto, imagen, vídeo, audio y PDF en un único clúster topológico [1, 2, 4]. 16
5 Mecanismo de agregación intermodal en *Gemini Embedding 2*: los tres fotogramas representativos de un `VideoChunk` se empaquetan como objetos `Part` individuales dentro de un único contenedor `types.Content`, permitiendo que el modelo fusione la información temporal y devuelva un único vector denso por fragmento [2, 5]. 17
6 Comparativa estructural entre (a) un *Bi-Encoder* de doble torre que codifica consulta y vídeo de forma independiente para indexación pasiva en bases de datos vectoriales, y (b) un *Cross-Encoder* (*Reranker*) que aplica atención cruzada conjunta sobre el subconjunto de candidatos recuperados [3, 4]. 19
7 Flujo del paradigma de búsqueda asimétrica en dos etapas (Regla 99 % / 1 %): el *Bi-Encoder* filtra masivamente el repositorio mediante similitud del coseno en Qdrant y el *Cross-Encoder* o verificador visual inspecciona únicamente el subconjunto Top-K para garantizar máxima precisión con latencia mínima [3, 4]. 20
8 Comparativa técnica de los dos flujos de ingestión en el SDK `google-genai`: (a) subida de archivos completos a la API Files con bucle de espera activa hasta alcanzar el estado `ACTIVE`, y (b) codificación JPEG en memoria de múltiples fotogramas empaquetados en un único objeto `types.Content` para llamadas asíncronas de baja latencia [2]. 23
9 Modelado de capacidad y cortafuegos de cuota para `gemini-embedding-2-preview`: la validación previa aborta ejecuciones que superen los 800 fragmentos para proteger el límite diario (1.000 RPD), mientras que la densidad de $\sim$ 900 *tokens* por fragmento fija el caudal efectivo en $\sim$ 33 *chunks*/minuto bajo el techo de 30.000 TPM [2]. 25
10 Cronograma temporal del algoritmo de *Exponential Backoff* calibrado (`retry_base_delay = 4.0`, `max_retries = 4`): la suma geométrica de las cuatro esperas alcanza exactamente 60 segundos, permitiendo superar bloqueos completos de la ventana de 30.000 TPM sin abortar la vectorización del fragmento. 26

11 Arquitectura de Degradación Elegante (*Graceful Degradation*) durante la búsqueda: si los servidores del modelo secundario (`gemini-3.5-flash-lite`) sufren picos de saturación (`503 UNAVAILABLE`) tras agotar los reintentos, el sistema conmuta limpiamente al orden de similitud vectorial calculado por Qdrant con *Gemini Embedding 2*. 27
12 Clasificación de las arquitecturas de modelos fundacionales para Video RAG en 2026: coexistencia de modelos hiper-escalados servidos mediante API comercial frente a ecosistemas de pesos abiertos desplegables en infraestructura local [2, 4, 6]. 29
13 Resolución de consultas altamente composicionales mediante la arquitectura en dos etapas de la familia Qwen3-VL: mientras el *Bi-Encoder* puede recuperar candidatos con atributos invertidos, las capas de *Cross-Attention* de `Qwen3-VL-Reranker` vinculan cada adjetivo y acción con su parche visual exacto para predecir el *logit* de probabilidad real [3, 4]. 30
14 Flujo de codificación visual directa en arquitecturas como *ColQwen* y *Sentence Transformers* multimodales: la combinación de teselas visuales con codificación posicional M-RoPE proyecta diapositivas, gráficos y escenas de vídeo directamente al espacio del lenguaje, eliminando el preprocesamiento intermedio mediante OCR [1, 7]. 31
15 Comparativa de compromisos de ingeniería entre el despliegue sobre API propietaria en la nube y la ejecución local con modelos de pesos abiertos para sistemas de recuperación de vídeo [2, 4]. 32
16 Comparativa de tres arquitecturas representativas de sistemas RAG de vídeo en la literatura reciente: (a) alineación de metadatos auxiliares en *Video-RAG*, (b) indexación multigranular en *CARVE* y (c) inferencia adaptativa resiliente en *Multi-RAG* [6, 8]. . . . 34
17 Contraste entre el emparejamiento temático tradicional y el Anclaje Causal Espaciotemporal (*Temporal Grounding*), el cual proporciona trazabilidad determinista y auditable sobre la línea de tiempo del vídeo [2, 9]. 35
18 Diagnóstico del cuello de botella perceptivo en la etapa de verificación visual: el escalado a 512 px combinado con compresión JPEG agresiva (12-20) impide el OCR implícito de sintaxis de programación en pantalla, mientras que elevar `resize_width` al rango 768-1280 px y la calidad JPEG a 50-90 restaura la detección sin disparar el consumo de cuota. 36
19 Mecanismo de dilución semántica y pérdida de fotogramas clave en vídeos con baja variación visual: (a) un fragmento de casi 40 s muestreado con solo 3 fotogramas omite el instante en que aparece el diccionario y mezcla múltiples temas; (b) la imposición de un límite superior (`max_shot_frames` de 10-15 s) o la extracción proporcional (un fotograma extra cada 5 s) garantiza la cohesión semántica. 37
20 Comparativa estructural entre (b) la división arbitraria en intervalos fijos, que introduce ruido contextual al cortar escenas y oraciones a la mitad, y (c) la segmentación semántica adaptativa (*Video-RAC*), que alinea exactamente los límites de cada `VideoChunk` con las transiciones reales del contenido [5, 9]. 39
21 Flujo algorítmico del cálculo del Índice de Similitud Estructural (SSIM) mediante filtrado Gaussiano en OpenCV (`compute_ssim`) y criterio de consolidación de objetos `VideoChunk` en `detect_shot_boundaries`. 41

22 Selección de fotogramas clave basada en Entropía de Shannon (`extract_key_frames`) según la metodología *Video-RAC*: extracción del fotograma inicial, el fotograma final y el fotograma intermedio que maximiza la densidad de información visual $H(F_t)$ [5]. 42
23 Arquitectura de procesamiento paralelo y alineación forzada intermodal: la rama visual segmenta el vídeo mediante CLIP/SSIM y extrae tres fotogramas por entropía de Shannon, mientras la rama auditiva filtra silencios con Silero VAD y alinea las marcas de tiempo de WhisperX sobre cada `VideoChunk` [5, 9, 10]. 43
24 Comparativa arquitectónica y de eficiencia computacional entre la ingestión de vídeo bruto en ventanas fijas a 1 FPS (*SentrySearch*) y la canalización adaptativa multimodal por entropía de Shannon propuesta en el TFM [2, 5, 11]. 45
25 Arquitectura de almacenamiento y recuperación en Qdrant: separación de vectores densos indexados mediante grafos HNSW y metadatos temporales alojados en el *payload* para consultas top-K ultra rápidas [2]. 46
26 Estructura combinada de evaluación empírica mediante el framework RAGAS y métricas de atribución verificable (ALCE/MIRAGE) para validar el rendimiento del sistema RAG de vídeo [5, 12]. 48
27 Flujo metodológico de cuatro etapas para la revisión sistemática de la literatura apoyado en gestores bibliográficos y entornos RAG acotados [13, 14]. 49
28 Mecanismo de corrección en la consulta `"Bucles while"`: la fusión temprana eleva el fragmento objetivo del séptimo al primer puesto en Qdrant ($+0,1064$ en similitud del coseno) y el verificador bimodal valida el código pese a la transcripción fonética *"Guile"*, descartando los fragmentos adyacentes sobre bucles `for`. 53
29 Compromiso dimensional Matryoshka (MRL): (a) retención prácticamente plana de *Recall@1* ($79,4\,\% \rightarrow 78,4\,\%$) con leve mejora en *Recall@5* ($97,1\,\% \rightarrow 99,0\,\%$) frente a (b) reducción del $75\,\%$ en el consumo de memoria RAM (12 KB $\rightarrow$ 3 KB por vector) y descenso del $44,4\,\%$ en la latencia de búsqueda en Qdrant. 59

Índice de tablas

1 Comparativa técnica del impacto de la dimensionalidad vectorial y el soporte MRL sobre la precisión semántica y el coste de almacenamiento en bases de datos vectoriales [2, 4]. . 14
2 Comparativa técnica entre arquitecturas *Bi-Encoder*, *Cross-Encoder* (*Reranker*) y Verificadores VLM Generativos en sistemas Video RAG [2–4]. 20
3 Parámetros técnicos calibrados en los módulos de vectorización y orquestación para operar con `gemini-embedding-2-preview` sin colapsos de cuota [2]. 27
4 Síntesis comparativa de las principales arquitecturas y modelos del ecosistema multimodal analizados en la literatura (2025-2026) [1, 2, 4, 6]. 32
5 Diagnóstico empírico de parámetros de preprocesamiento visual y temporal en sistemas Video RAG con verificación secundaria (*Gemini Embedding 2* + *Gemini Flash*) [2]. 38
6 Especificaciones técnicas de la estrategia de fragmentación semántica adaptativa (*Video-RAC*) y alineación forzada de audio [5, 9, 10]. 43
7 Comparativa técnica y de costes operativos entre *SentrySearch* y la arquitectura desarrollada en el TFM [2, 5, 11]. 45

8 Síntesis de métricas, bancos de pruebas y metodologías de evaluación en sistemas Video RAG [4, 5, 12, 13]. 49
9 Estudio de ablación E2E: comparativa de rendimiento entre la arquitectura puramente visual y la arquitectura con fusión temprana de audio (*Whisper*) y juez bimodal [2, 5]. . . 50
10 Auditoría detallada de fragmentos en las consultas críticas "Diccionarios" y "Bucles while" sobre python_tutorial.mp4. 53
11 Evolución experimental sobre animals.mp4: efecto de la deduplicación por transcripción idéntica (run_20261109_183310) y del desacoplamiento del audio frente a consultas específicas y taxonómicas. 56
12 Resultados del estudio de ablación dimensional Matryoshka (MRL) sobre consultas literales de evento único (102 consultas, 164 fragmentos en precisión float32) en Etapa 1 (Qdrant). 58
13 Rendimiento en consultas generales y multi-objetivo (20 consultas, 106 intervalos esperados, $K = 10$) comparando la Etapa 1 (Qdrant) y la Etapa 2 (verificador bimodal) a través de las tres dimensiones MRL. 60
14 Desglose detallado por consulta general y taxonómica sobre animals.mp4 (10 consultas, 58 intervalos temporales esperados) en Etapa 1 (E1) y Etapa 2 (E2). 60
15 Evaluación de consultas negativas de control (16 consultas, 240 fragmentos candidatos analizados en Etapa 2) a través de las tres dimensiones MRL. 61
16 Proyección de huella de memoria RAM en base de datos vectorial (float32) según el volumen de fragmentos y la dimensión MRL seleccionada. 62

1 Introducción

1.1 Contexto y Motivación

La evolución contemporánea de los sistemas de Recuperación Aumentada por Generación (RAG, por sus siglas en inglés) ha transitado desde arquitecturas puramente textuales hacia paradigmas multimodales capaces de procesar, alinear e interpretar información visual, auditiva y textual dentro de un único espacio semántico continuo [1]. En el ámbito de la recuperación de eventos de interés sobre secuencias de vídeo de larga duración (disciplina denominada en la literatura académica como *Long-form Video Understanding* o *Video RAG*), tanto la investigación científica como el desarrollo industrial han experimentado un avance significativo entre los años 2024 y 2026 [6]. Este progreso responde a la necesidad de identificar, localizar y extraer eventos específicos dentro de archivos de vídeo extensos (tales como la mención de un producto, la aparición de un objeto o un cambio de escena), eliminando la necesidad del visionado manual.

1.2 Planteamiento del Problema

Históricamente, la arquitectura tradicional para recuperar información en vídeo obligaba a procesar los fotogramas por separado, emplear modelos de transcripción fonética básica para el audio (como Whisper) y extraer texto mediante Reconocimiento Óptico de Caracteres (OCR) para indexarlo todo como texto plano [8]. El estado del arte ha demostrado empíricamente que convertir toda la información visual a texto plano genera un cuello de botella informacional y una pérdida severa de contexto temporal, espacial, estructural y emocional, haciendo desaparecer la disposición espacial (*layout*) y las relaciones semánticas entre los objetos visuales [1]. Frente a estas limitaciones, la aparición de modelos de *embeddings* omnimodales permite ingerir texto, vídeo, audio y documentos estructurados (PDF) de forma nativa para proyectarlos en un único vector numérico sin requerir metadatos anotados manualmente [2].

1.3 Propuesta y Objetivos del Trabajo Fin de Máster

El presente Trabajo de Fin de Máster aborda el ciclo de vida completo para el diseño, desarrollo y validación técnica de un sistema avanzado de recuperación de información multimedia capaz de transformar grandes volúmenes de datos no estructurados (vídeo, audio y texto) en representaciones vectoriales densas (*embeddings*) dentro de un espacio semántico unificado. El sistema permite realizar búsquedas semánticas cruzadas (localizando el instante exacto de un evento en un archivo de vídeo a partir de una consulta en lenguaje natural) sin depender de metadatos previos.

Para estructurar este desarrollo, se define el siguiente **Objetivo General**: diseñar, implementar y validar empíricamente un sistema avanzado de recuperación de información multimedia basado en arquitecturas de vectorización multimodal, capaz de localizar eventos de interés en grandes volúmenes de datos no estructurados a partir de búsquedas semánticas en lenguaje natural. Este propósito se desglosa en cinco **Objetivos Específicos**:

1. **Analizar el estado del arte** tecnológico en el ámbito de los modelos fundacionales multimodales (generación de *embeddings*) y los sistemas de búsqueda semántica (RAG y bases de datos vectoriales), evaluando las alternativas óptimas para el proyecto.
2. **Diseñar la arquitectura técnica** y el flujo de procesamiento (*pipeline*) de datos, definiendo los métodos para la ingestión, fragmentación, preprocesamiento y vectorización de los archivos de vídeo, audio y texto.
3. **Desarrollar una Prueba de Concepto (PoC)** funcional que integre el modelo de *embeddings* seleccionado, mapeando los distintos tipos de datos en un espacio semántico unificado para permitir búsquedas cruzadas eficientes sin depender de metadatos previos.
4. **Evaluar y validar el rendimiento** del sistema mediante la ejecución de un plan de pruebas empíricas, midiendo métricas clave como la precisión en la recuperación de eventos, el coste computacional y la viabilidad técnica general de la solución.
5. **Documentar el ciclo de vida completo** del proyecto de ingeniería del software y datos, detallando el marco metodológico utilizado desde la definición de requisitos hasta la validación final.

2 Revisión de la Literatura

2.1 Fundamentos Matemáticos de los Embeddings Multimodales y Espacios Semánticos Continuos

2.1.1 El Problema del Análisis Secuencial y la Representación de Datos No Estructurados

El crecimiento sostenido de los repositorios multimedia en entornos académicos, industriales y de radiodifusión ha puesto de manifiesto un problema metodológico fundamental en la ingeniería de datos: la extrema ineficiencia del análisis secuencial humano frente a volúmenes masivos de información no estructurada [1]. Históricamente, la tarea de localizar un evento singular dentro de un archivo de vídeo de larga duración (disciplina denominada en la literatura científica como *Long-form Video Understanding* o *Video RAG*) exigía bien un visionado manual exhaustivo por parte de operadores humanos, o bien una dependencia absoluta de metadatos descriptivos y etiquetas manuales generadas previamente a la indexación [6]. En escenarios reales que abarcan desde archivos históricos de televisión y auditorías de videovigilancia hasta repositorios de clases universitarias grabadas o historiales médicos complejos, el etiquetado manual resulta técnica y económicamente inviable.

Para superar la barrera del procesamiento manual, los sistemas de Recuperación de Información (*Information Retrieval*) evolucionaron hacia el paradigma de la Recuperación Aumentada por Generación (RAG, por sus siglas en inglés) [1]. No obstante, mientras que las arquitecturas RAG de primera generación operaban exclusivamente sobre cadenas de texto plano, los flujos multimedia contemporáneos requieren procesar, alinear e interpretar simultáneamente señales heterogéneas compuestas por secuencias de fotogramas, pistas de audio y texto libre [1, 6].

La base matemática que permite a un sistema computacional operar sobre formatos tan dispares sin recurrir a reglas heurísticas manuales es el concepto de **representación vectorial densa** o ***embedding*** [2]. Un *embedding* es una proyección matemática que transforma un objeto de entrada arbitrario x (ya sea una oración en lenguaje natural, una matriz de píxeles correspondiente a un fotograma de vídeo o una onda acústica) en un vector de números reales en coma flotante $\mathbf{v} \in \mathbb{R}^d$ dentro de un espacio métrico continuo de dimensión d, según se formaliza en la Ecuación (1) [2]:

$$f_\theta : \mathcal{X}_{\text{texto}} \cup \mathcal{X}_{\text{imagen}} \cup \mathcal{X}_{\text{vídeo}} \cup \mathcal{X}_{\text{audio}} \cup \mathcal{X}_{\text{PDF}} \longrightarrow \mathbb{R}^d \tag{1}$$

donde f_θ representa una red neuronal profunda parametrizada por los pesos θ (un Modelo Fundacional Multimodal) y $\mathbb{R}^d$ constituye el **espacio semántico compartido** o espacio latente unificado [2]. La propiedad fundamental de este mapeo unificado reside en que la topología del espacio vectorial preserva la semántica subyacente con independencia de la modalidad de origen: para que un sistema inteligente comprenda que la fotografía de un objeto o ser vivo (por ejemplo, la imagen de un perro), su manifestación acústica (el ladrido grabado en una pista de audio) y su representación simbólica (la palabra escrita “perro”) expresan el mismo concepto, el modelo proyecta las tres entradas hacia coordenadas geométricas sumamente próximas entre sí dentro del mismo plano matemático [2].

2.1.2 El Ocaso de las Canalizaciones OCR y la Ingestión Visual Nativa con M-RoPE

Durante la primera mitad de la década de 2020, el diseño estándar para indexar contenido visual y de vídeo en sistemas RAG imponía una canalización rígida, escalonada y unimodal [1]. Este flujo tradicional obligaba a extraer fotogramas aislados del vídeo, aplicar algoritmos clásicos de detección de contornos, someter cada imagen a un motor de Reconocimiento Óptico de Caracteres (OCR) para obtener cadenas de texto plano, transcribir en paralelo la pista de audio mediante sistemas de Reconocimiento Automático del Habla (ASR, como el modelo Whisper) e inyectar finalmente todo el texto concatenado en un modelo de *embeddings* puramente textual [1, 8].

La literatura científica publicada entre 2024 y 2026 ha demostrado empíricamente el colapso operativo de este paradigma basado en traducción forzosa a texto plano [1]. Convertir toda la riqueza de una señal visual en secuencias léxicas mediante OCR introduce tres deficiencias estructurales críticas [1]:

1. **Cuello de botella informacional:** La traducción de una escena tridimensional o de un fotograma complejo a palabras sueltas descarta toda la información visual no textual, tales como acciones físicas, colores, texturas, expresiones emocionales e interacciones implícitas entre objetos [1].
2. **Destrucción del diseño espacial (*Layout*):** Cuando un sistema procesa diapositivas académicas, diagramas arquitectónicos, tablas financieras o editores de código fuente extrayendo únicamente los caracteres visibles mediante OCR, la disposición bidimensional (*layout*) y las relaciones jerárquicas y topológicas entre los bloques visuales desaparecen por completo [1].
3. **Propagación de errores en cascada:** Cualquier fallo de reconocimiento en el motor OCR (frecuente ante cambios de iluminación, tipografías pequeñas, compresión de vídeo o fondos oscuros) corrompe irreversiblemente el texto indexado, impidiendo que el motor de búsqueda recupere el segmento aunque el contenido visual original sea perfectamente claro para un observador [1].

La Figura 1 contrasta esquemáticamente el flujo tradicional dependiente de OCR frente al paradigma contemporáneo de vectorización multimodal directa [1].

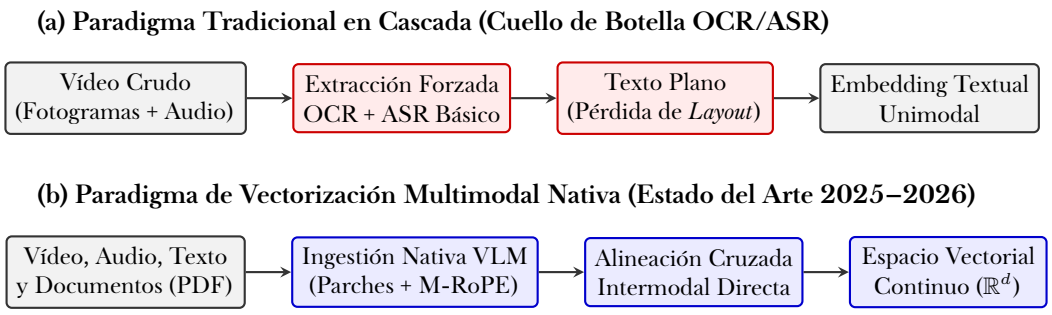


Figura 1: Comparativa arquitectónica entre (a) la canalización tradicional basada en conversión intermedia a texto plano mediante OCR y (b) la proyección multimodal directa en un espacio semántico continuo mediante modelos VLM y codificación M-RoPE [1, 2].

Para erradicar esta pérdida de fidelidad, el estado del arte ha adoptado arquitecturas de recuperación visual directa lideradas por modelos como **ColQwen** y sistemas basados en *Vision-Language Models* (VLMs) de última generación como la familia **Qwen3-VL** y **Gemini Embedding 2** [1, 2, 4]. En lugar

de extraer caracteres de forma aislada, estos sistemas ingieren las imágenes y los fotogramas de vídeo a su resolución nativa y dividen el lienzo visual en una cuadrícula de parches o teselas (*visual patches*), siguiendo el principio arquitectónico de los *Vision Transformers* (ViT) [1].

El avance matemático determinante que permite a estos modelos conservar la estructura geométrica de la escena es la incorporación de ***Multimodal Rotary Position Embeddings*** **(M-RoPE)** [1]. Mientras que las codificaciones posicionales rotatorias clásicas (RoPE) en modelos de lenguaje operan sobre una única dimensión secuencial (el índice t del *token* en una oración), M-RoPE descompone y generaliza la matriz de rotación posicional para codificar coordenadas multidimensionales de manera nativa [1]. Al procesar un fragmento de vídeo, cada parche visual recibe una codificación posicional que preserva su posición espacial bidimensional (x, y) dentro del fotograma y su evolución temporal t a lo largo de la secuencia, permitiendo procesar resoluciones variables sin deformar la relación de aspecto y alineando directamente las teselas visuales en el mismo espacio vectorial que los *tokens* de texto del modelo subyacente [1].

2.1.3 Geometría del Espacio Latente y Métricas de Similitud Estadística

Una vez que el modelo fundacional proyecta tanto los fragmentos del archivo de vídeo como la consulta en lenguaje natural del usuario dentro del espacio vectorial compartido $\mathbb{R}^d$, el problema de localizar el instante exacto de un evento (*timestamp*) se reduce a un cálculo geométrico de cercanía estadística [9].

Sea $\mathbf{q} = (q_1, q_2, \ldots, q_d)^\top \in \mathbb{R}^d$ el vector de *embedding* correspondiente a una consulta textual (por ejemplo, *"¿En qué momento se muestra un diccionario en el código?"*) y sea $\mathbf{v}_i = (v_{i,1}, v_{i,2}, \ldots, v_{i,d})^\top \in \mathbb{R}^d$ el vector denso asociado al i-ésimo fragmento temporal $[t^{(i)}_{\text{inicio}}, t^{(i)}_{\text{fin}}]$ de un vídeo indexado. La literatura y las bases de datos vectoriales modernas emplean fundamentalmente dos operadores algebraicos para cuantificar la afinidad semántica entre $\mathbf{q}$ y $\mathbf{v}_i$ [2, 3]:

2.1.3.1 Similitud del Coseno (*Cosine Similarity*):

Mide el coseno del ángulo θ formado entre ambos vectores en el hiperplano d-dimensional mediante la Ecuación (2), evaluando su alineación direccional con independencia de sus magnitudes absolutas:

$$\text{sim}_{\cos}(\mathbf{q}, \mathbf{v}_i) = \cos(\theta) = \frac{\mathbf{q} \cdot \mathbf{v}_i}{\|\mathbf{q}\|_2 \, \|\mathbf{v}_i\|_2} = \frac{\sum_{j=1}^{d} q_j \, v_{i,j}}{\sqrt{\sum_{j=1}^{d} q_j^2} \sqrt{\sum_{j=1}^{d} v_{i,j}^2}} \in [-1, 1] \tag{2}$$

Un valor cercano a 1 indica una coincidencia semántica elevada (vectores colineales que apuntan hacia la misma región conceptual del espacio latente), un valor próximo a 0 denota ortogonalidad (conceptos no relacionados) y valores negativos reflejan oposición semántica.

2.1.3.2 Producto Punto Escalar (*Scalar Dot Product*):

Definido algebraicamente en la Ecuación (3) como la suma de los productos componente a componente [3]:

$$\text{sim}_{\text{dot}}(\mathbf{q}, \mathbf{v}_i) = \mathbf{q}^\top \mathbf{v}_i = \sum_{j=1}^{d} q_j \, v_{i,j} \tag{3}$$

Cuando todos los vectores almacenados en la base de datos vectorial han sido previamente normalizados a longitud unitaria mediante la norma euclídea L_2 ($\|\mathbf{q}\|_2 = \|\mathbf{v}_i\|_2 = 1$), los denominadores de la Ecuación (2) se vuelven exactamente igual a la unidad, haciendo que la Similitud del Coseno y el Producto Punto Escalar sean matemáticamente idénticos ($\text{sim}_{\cos}(\mathbf{q}, \mathbf{v}_i) = \mathbf{q}^\top \mathbf{v}_i$) [3]. Esta equivalencia resulta vital en sistemas de recuperación a escala de petabytes, ya que reduce la búsqueda a multiplicaciones matriciales altamente paralelizadas en hardware moderno [3].

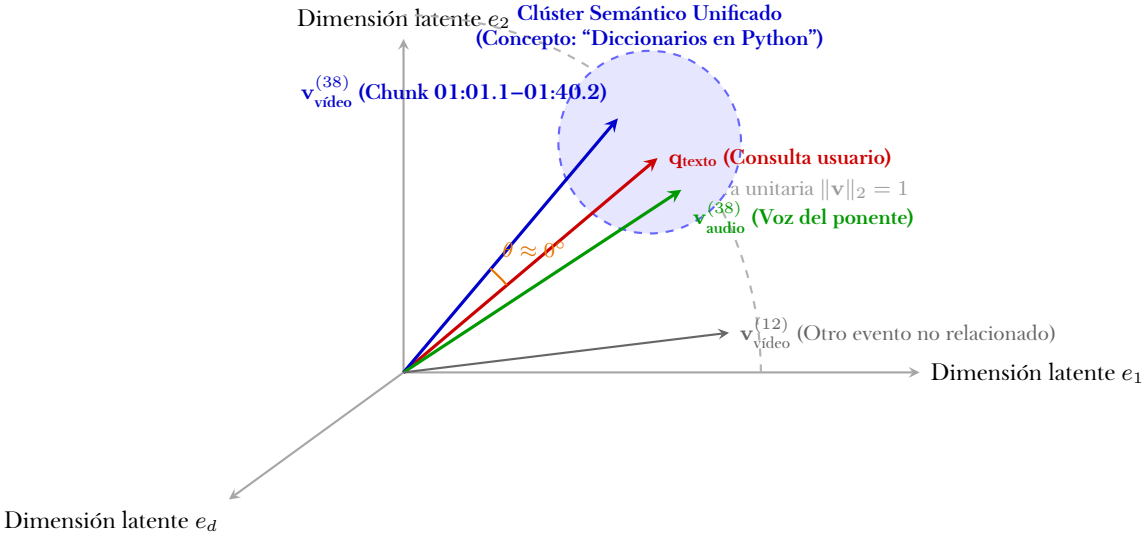


Figura 2: Geometría del espacio vectorial compartido $\mathbb{R}^d$: los vectores normalizados de distintas modalidades (consulta textual $\mathbf{q}_{\text{texto}}$, fotogramas de vídeo $\mathbf{v}_{\text{vídeo}}$ y pista de audio $\mathbf{v}_{\text{audio}}$) que describen un mismo evento convergen con un ángulo $\theta \to 0°$ sobre la hipersfera unitaria, permitiendo recuperar la marca temporal exacta mediante Similitud del Coseno [2].

La Figura 2 ilustra esta convergencia geométrica sobre la hipersfera unitaria: dado que la consulta en texto libre y el fragmento de vídeo comparten la misma región del espacio latente, el motor de búsqueda identifica el intervalo temporal exacto del suceso evaluando qué vectores $\mathbf{v}_i$ maximizan $\cos(\theta)$ respecto a $\mathbf{q}$ [2].

2.1.4 Aprendizaje de Representación Matryoshka (MRL) y Normalización L_2

Uno de los mayores retos de ingeniería al escalar sistemas de vectorización multimodal a millones de fragmentos de vídeo reside en la rígida dependencia entre la dimensionalidad de salida d del modelo y el consumo de memoria RAM y almacenamiento en la base de datos vectorial [3, 4]. En los modelos de *embeddings* tradicionales, toda la información semántica se distribuye de manera uniforme y difusa a lo largo de todas las coordenadas del vector $(v_1, v_2, \ldots, v_d)$; en consecuencia, recortar un vector clásico sin volver a entrenar la red neuronal destruye su estructura topológica y provoca una caída drástica en la precisión de búsqueda [3].

Para dotar de elasticidad dimensional a las arquitecturas modernas (tales como **Gemini Embedding 2**, **Qwen3-VL-Embedding** y **EmbeddingGemma 2**), el estado del arte ha estandarizado el **Aprendizaje de Representación Matryoshka** (*Matryoshka Representation Learning*, **MRL**) [2, 4]. Inspirado en las muñecas rusas anidadas (*matryoshkas*), el entrenamiento mediante la función de pérdida *MatryoshkaLoss*

optimiza simultáneamente un conjunto de sub-dimensiones anidadas $\mathcal{M} = \{m_1, m_2, \ldots, m_k\}$ donde $m_1 < m_2 < \ldots < m_k = d$ (por ejemplo, $\mathcal{M} = \{64, 128, 256, 512, 768, 1024, 1536, 3072, 4096\}$) [2–4]. Formalmente, la pérdida total durante el entrenamiento contrastivo se expresa mediante la Ecuación (4) como la suma ponderada de las pérdidas evaluadas sobre cada prefijo truncado $\mathbf{v}_{1:m} \in \mathbb{R}^m$ [3]:

$$\mathcal{L}_{\text{MRL}}(\mathbf{q}, \mathbf{v}) = \sum_{m \in \mathcal{M}} c_m \cdot \mathcal{L}_{\text{contrastiva}}(\mathbf{q}_{1:m}, \mathbf{v}_{1:m}) \tag{4}$$

Esta formulación obliga a la red neuronal a jerarquizar la varianza semántica, comprimiendo los atributos conceptuales más críticos y globales en las primeras coordenadas del vector y reservando las dimensiones finales para matices granulares de grano fino [3]. En la práctica operativa, esto permite descargar un modelo de alta capacidad como *Qwen3-VL-Embedding-8B* (cuya dimensión nativa es $d = 4096$) o invocar *Gemini Embedding 2* ($d = 3072$) y truncar dinámicamente sus vectores de salida a 1536, 1024, 768, 512, 256, 128 o incluso 64 dimensiones en tiempo de ejecución sin requerir ningún reentrenamiento [2, 3].

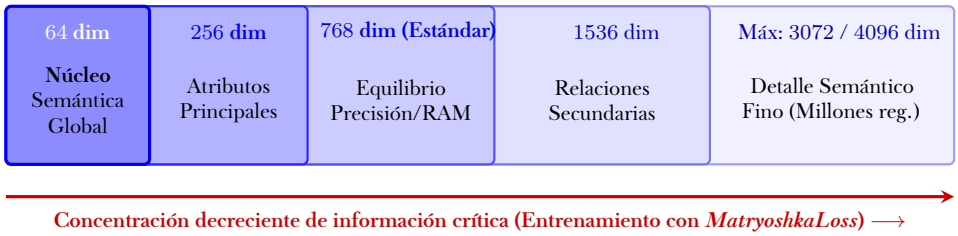


Figura 3: Estructura jerárquica de un vector entrenado mediante *Matryoshka Representation Learning* (MRL): las primeras dimensiones concentran la mayor carga semántica, permitiendo truncar el vector de salida para reducir exponencialmente el consumo de memoria RAM y latencia con penalizaciones mínimas de precisión [2, 3].

Ahora bien, desde el punto de vista de la implementación algorítmica rigurosa, truncar un vector unitario $\mathbf{v} \in \mathbb{R}^D$ (donde $\|\mathbf{v}\|_2 = 1$) a sus primeras $d < D$ componentes ($\mathbf{v}_{1:d}$) hace que la suma de cuadrados de las coordenadas retenidas sea estrictamente menor que la unidad ($\sum_{j=1}^{d} v_j^2 < 1$). Dado que los vectores truncados (por ejemplo, al configurar $d = 768$ o $d = 1536$ en lugar de las 3072 dimensiones completas en *Gemini Embedding 2*) no vienen normalizados de fábrica tras el corte, resulta obligatorio aplicar una etapa explícita de **re-normalización mediante la norma euclídea** L_2 según se define en la Ecuación (5) antes de insertarlos en la base de datos vectorial [2]:

$$\hat{\mathbf{v}}_{1:d} = \frac{\mathbf{v}_{1:d}}{\|\mathbf{v}_{1:d}\|_2} = \frac{(v_1, v_2, \ldots, v_d)^\top}{\sqrt{\sum_{j=1}^{d} v_j^2}} \tag{5}$$

Esta operación proyecta de nuevo el sub-vector truncado sobre la superficie de la hipersfera unitaria de dimensión d, garantizando que las búsquedas posteriores por Similitud del Coseno en motores como Qdrant mantengan su consistencia matemática y velocidad de cómputo.

2.1.5 Análisis del Compromiso entre Precisión Semántica y Coste de Almacenamiento

La elección de la dimensionalidad operativa del espacio latente introduce un compromiso directo (*trade-off*) de arquitectura entre la granularidad de discriminación semántica y los costes de infraestructura [2, 4]. Como se sintetiza en el Cuadro 1, trabajar con el espacio completo de **3.072 dimensiones** en *Gemini Embedding 2* (o **4.096 dimensiones** en *Qwen3-VL-Embedding-8B*) permite capturar un nivel de detalle semántico superior, lo cual resulta determinante cuando la base de datos acumula millones de registros multimodales y se requiere evitar falsos positivos ante consultas altamente específicas [2, 4].

Tabla 1: Comparativa técnica del impacto de la dimensionalidad vectorial y el soporte MRL sobre la precisión semántica y el coste de almacenamiento en bases de datos vectoriales [2, 4].

Configuración	**Modelos de Referencia**	**Peso (FP32)**	**Norm. L_2**	**Caso de Uso Óptimo**
4.096 dim (Máx. Open)	Qwen3-VL-Embedding-8B	16,0 KB	Nativa	Máxima resolución en servidor GPU local
3.072 dim (Máx. Cloud)	Gemini Embedding 2	12,0 KB	Nativa	*Baseline* de máxima precisión y millones de reg.
2.048 dim (Intermedia)	Qwen3-VL-Embedding-2B	8,0 KB	Nativa / MRL	Despliegue local equilibrado en VRAM
1.536 dim (MRL Medio)	Gemini Embedding 2	6,0 KB	Post-corte	Reducción del 50 % en RAM con alta fidelidad
768 dim (Estándar)	Gemini 2 / EmbeddingGemma 2	3,0 KB	Post-corte	Ahorro del 75 % en BD y consultas ultrarrápidas
64 - 256 dim (Ultraligero)	Qwen3-VL / EmbeddingGemma 2	0,25 - 1,0 KB	Post-corte	*Edge computing*, dispositivos móviles y pre-filtrado

Por el contrario, operar sobre un estándar de **768 dimensiones** (que constituye el tope nativo de modelos ligeros modulares como *EmbeddingGemma 2* (con posibilidad de truncado adicional a 256 o 128 dimensiones) y uno de los puntos de corte recomendados en *Gemini Embedding 2*) reduce exactamente en un 75 % la huella de memoria de cada vector respecto a las 3,072 dimensiones (pasando de 12, 0 KB a 3, 0 KB por fragmento en precisión simple de 32 bits). Esta compresión hace que la base de datos vectorial resultante sea significativamente más económica de alojar en memoria RAM y más rápida de consultar localmente, a cambio de asumir una ligera reducción en la granularidad semántica fina frente al vector completo de 3,072 dimensiones.

2.2 Funcionamiento y Taxonomía de Modelos de Embeddings Multimodales

2.2.1 Evolución Arquitectónica: De los Modelos Alineados (CLIP e ImageBind) a la Omnimodalidad Nativa

El diseño de redes neuronales capaces de proyectar múltiples modalidades de datos dentro de un espacio métrico común ha atravesado tres generaciones claramente diferenciadas en la literatura científica reciente [1, 4]. Comprender las limitaciones estructurales de las dos primeras generaciones resulta indispensable para justificar la adopción de los modelos fundacionales omnimodales desarrollados entre finales de 2024 y 2026 [2, 4].

Durante la primera etapa evolutiva, el estándar de la industria estuvo dominado por modelos de alineación dual o múltiple como **CLIP** (*Contrastive Language-Image Pre-training*, desarrollado por OpenAI) e **ImageBind** (desarrollado por Meta) [1]. Estas arquitecturas lograron un hito fundacional al demostrar que era posible entrenar codificadores separados (un codificador visual y un codificador textual) mediante aprendizaje contrastivo para alinear texto e imágenes estáticas (y, en casos puntuales como ImageBind, señales de audio) dentro de un mismo hiperplano vectorial [1]. Sin embargo, cuando estas arquitecturas de primera generación se aplicaban a la recuperación de información en secuencias de vídeo de larga duración, presentaban una limitación insalvable: carecían de una comprensión temporal nativa e integrada [1, 15]. Como consecuencia, la arquitectura tradicional obligaba a descomponer artificialmente el flujo multimedia: los ingenieros debían extraer y procesar los fotogramas de manera aislada, emplear modelos de transcripción fonética independientes para la pista de audio (como Whisper) y terminar indexando toda la información resultante como texto plano [8]. Este flujo fragmentado no solo resultaba computacionalmente torpe, sino que destruía el contexto temporal y la sincronía entre la acción visual y el suceso acústico [1].

Una segunda generación de transición surgió a principios de 2025 con propuestas académicas como **VLM2Vec** y arquitecturas de interacción tardía como **ColQwen** [1]. El mérito científico de *VLM2Vec* (construido sobre modelos de lenguaje visuales genéricos como LLaVA y Phi) consistió en demostrar empíricamente cómo transformar un Modelo de Lenguaje Visual (*Vision-Language Model*, VLM) originalmente concebido para la generación autorregresiva de texto en un extractor masivo de *embeddings* multimodales para texto, imagen y vídeo. En paralelo, sistemas como *ColQwen* demostraron que era posible eliminar los motores OCR intermedios proyectando directamente las teselas visuales hacia el espacio latente del lenguaje [1].

Finalmente, entre finales de 2024 y el primer trimestre de 2026 se consolida la tercera generación: los **Modelos de Embedding Omnimodales (*Omni-modality*)** [2, 4]. Liderados en el ámbito comercial por **Gemini Embedding 2** (Google) y en el ecosistema de pesos abiertos por la familia **Qwen3-VL-Embedding** (Alibaba) y **EmbeddingGemma 2** (Google DeepMind), estos sistemas han sido diseñados desde su pre-entrenamiento para ingerir de forma nativa texto, imágenes, secuencias de vídeo, pistas de audio y documentos estructurados (archivos PDF) proyectándolos directamente en un único vector denso sin pasos de traducción textual intermedios [2, 4].

La Figura 4 sintetiza gráficamente la transición arquitectónica a través de estas tres generaciones de modelos de representación.

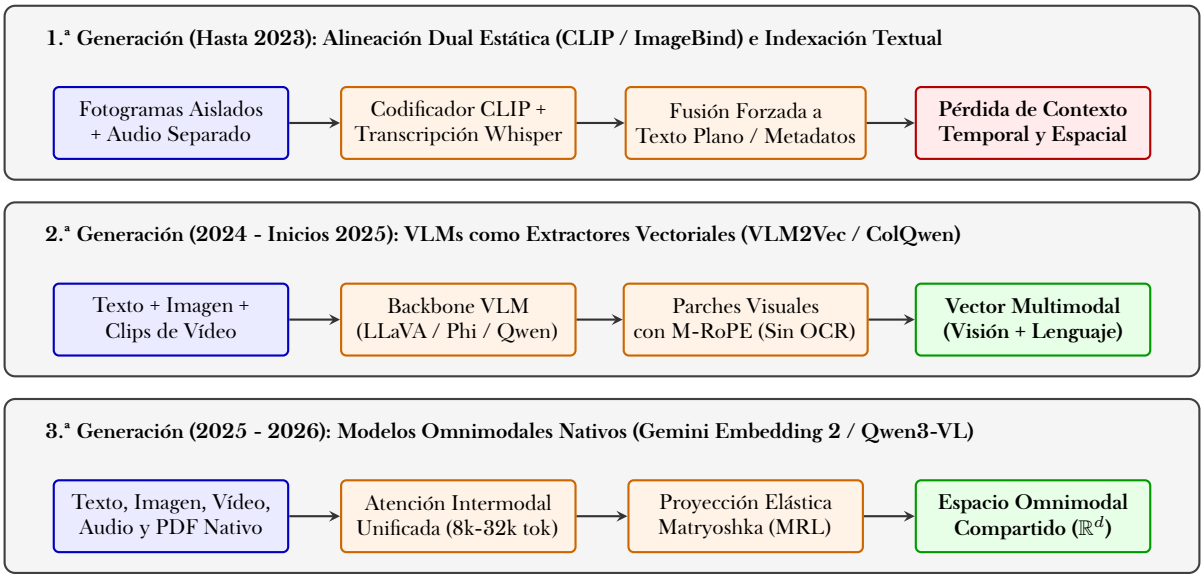


Figura 4: Evolución taxonómica de los modelos de representación vectorial: desde la alineación estática dependiente de transcripción textual (CLIP/Whisper) hasta los modelos omnimodales nativos de tercera generación capaces de proyectar texto, imagen, vídeo, audio y PDF en un único clúster topológico [1, 2, 4].

2.2.2 Tokenización Visual, Codificación M-RoPE y Agregación Temporal de Fotogramas

A nivel interno, el funcionamiento de un modelo de *embedding* multimodal contemporáneo se apoya en la unificación de la representación de entrada mediante teselas o parches (*visual patches*) procesados por arquitecturas inspiradas en *Vision Transformers* (ViT) [1]. Cuando el sistema recibe un fotograma de vídeo o una página de un documento visual a su resolución nativa, divide la matriz bidimensional de píxeles en una secuencia de sub-bloques que se proyectan linealmente como *tokens* visuales dentro de la misma secuencia de entrada que los *tokens* léxicos del texto [1].

Para evitar que esta linealización destruya la geometría bidimensional de la imagen o la secuencia temporal del vídeo, las arquitecturas modernas incorporan ***Multimodal Rotary Position Embeddings*** **(M-RoPE)** [1]. M-RoPE codifica directamente la información posicional bidimensional y temporal sobre las representaciones latentes de cada parche visual, lo que resulta determinante al procesar fotogramas de vídeo con resoluciones variables, garantizando que las relaciones espaciales entre los objetos visuales (como un bloque de código fuente en una esquina de la pantalla y el ponente en el lado opuesto) queden alineadas en el mismo espacio vectorial que los *tokens* del Modelo de Lenguaje Grande (LLM) subyacente [1].

Un aspecto arquitectónico fundamental en la vectorización de fragmentos de vídeo (`VideoChunk`) radica en el mecanismo de **agregación temporal de múltiples fotogramas** [2]. Cuando un algoritmo de segmentación semántica adaptativa (como *Video-RAC*) extrae tres fotogramas representativos de un plano (el fotograma inicial, el de máxima entropía visual y el fotograma final), existen dos formas opuestas de invocar al modelo fundacional [2, 5]:

1. **Codificación Independiente por Fotograma (Incorrecta para cohesión de *chunk*):** Si los tres foto-

gramas se envían a la API como tres objetos de contenido separados, el modelo genera tres vectores independientes ($\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3 \in \mathbb{R}^d$). Esto triplica el coste de almacenamiento en la base de datos vectorial y obliga a realizar promedios heurísticos posteriores que diluyen la semántica temporal del plano.

2. **Agregación Nativa en un Único Objeto `Content` (Estado del Arte):** En arquitecturas como *Gemini Embedding 2*, al codificar los tres fotogramas clave en memoria (por ejemplo, en formato JPEG) e inyectarlos como múltiples objetos `Part` dentro de una única estructura `types.Content`, el mecanismo de atención interna del modelo evalúa conjuntamente la evolución entre los tres instantes y devuelve **un único *embedding* denso agregado** $\mathbf{v}_{\text{chunk}} \in \mathbb{R}^d$ que sintetiza el significado completo del evento [2].

La Figura 5 ilustra cómo la agrupación de múltiples partes multimedia dentro de un mismo contenedor `Content` permite condensar la dinámica del fragmento temporal en un único punto del espacio latente.

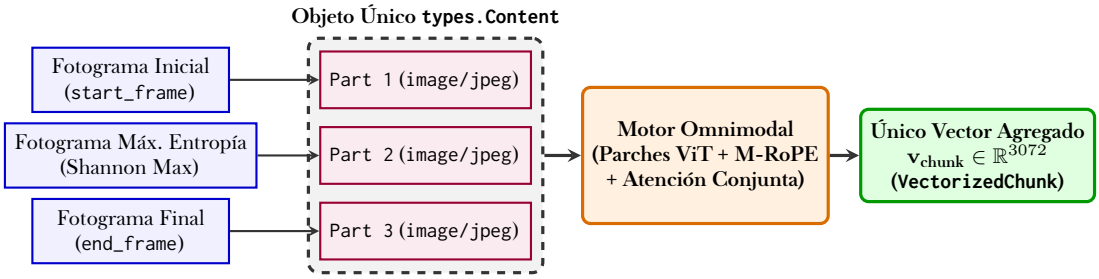


Figura 5: Mecanismo de agregación intermodal en *Gemini Embedding 2*: los tres fotogramas representativos de un `VideoChunk` se empaquetan como objetos `Part` individuales dentro de un único contenedor `types.Content`, permitiendo que el modelo fusione la información temporal y devuelva un único vector denso por fragmento [2, 5].

2.2.3 Arquitecturas de Doble Torre (*Bi-Encoder*) para Recuperación Masiva Precomputada

En el diseño de sistemas de recuperación de información sobre grandes volúmenes de vídeo, la primera etapa del procesamiento recae sobre los modelos con **arquitectura de Doble Torre (*Bi-Encoder*)**, representados en el estado del arte por sistemas como *Gemini Embedding 2*, *Qwen3-VL-Embedding* (en sus variantes de 2B y 8B de parámetros) y *EmbeddingGemma 2* [2, 4].

La característica definitoria de un *Bi-Encoder* es que procesa de manera completamente independiente el documento multimedia candidato D_i (un fragmento de vídeo, una imagen o una pista de audio) y la consulta textual del usuario Q [3, 4]. Matemáticamente, sean f_θ^{doc} y f_θ^{query} las funciones de proyección de ambas torres (que en los modelos fundacionales modernos suelen compartir los mismos pesos neuronales θ, diferenciándose únicamente por el prefijo de instrucción de entrada), según se define en la Ecuación (6) [7]:

$$\mathbf{v}_i = f_\theta^{\text{doc}}(D_i) \in \mathbb{R}^d, \qquad \mathbf{q} = f_\theta^{\text{query}}(P_{\text{task}} \oplus Q) \in \mathbb{R}^d \tag{6}$$

donde P_{task} representa un prefijo corto de instrucción de tarea que se antepone exclusivamente a las entradas de texto para orientar la consulta en el espacio latente (técnica empleada explícitamente en modelos como *EmbeddingGemma 2* y *Qwen3-VL-Embedding*, pero que no se aplica a las imágenes, vídeos ni audios) [4].

Esta independencia algebraica entre $\mathbf{v}_i$ y $\mathbf{q}$ constituye la piedra angular de la escalabilidad industrial por tres motivos fundamentales [3]:

1. **Indexación Pasiva *Offline*:** Permite pre-computar una sola vez los vectores $\mathbf{v}_i$ de un archivo histórico compuesto por millones de horas de vídeo y almacenarlos pasivamente en bases de datos vectoriales indexadas estáticamente como Qdrant o Milvus [3].
2. **Complejidad de Inferencia Constante respecto al Modelo:** Cuando el usuario introduce una búsqueda en lenguaje natural en tiempo de ejecución, la red neuronal profunda solo se ejecuta una vez para codificar la consulta $\mathbf{q}$ [3].
3. **Maximización de la Tasa de Recuerdo (*Recall*) a Escala de Petabytes:** La recuperación se reduce a comparar el vector $\mathbf{q}$ recién generado contra los millones de vectores pre-computados mediante operaciones vectoriales ultrarrápidas (como el producto punto escalar o la similitud del coseno), logrando tiempos de respuesta de milisegundos [3, 4].

2.2.4 Arquitecturas de Torre Única (*Cross-Encoder / Reranker*) y Atención Cruzada

A pesar de su extraordinaria velocidad, los modelos *Bi-Encoder* sufren una limitación estructural derivada de comprimir todo un fragmento de vídeo en un único vector estático $\mathbf{v}_i$ sin conocer de antemano qué preguntará el usuario [3, 4]. Al no existir interacción directa entre los *tokens* de la consulta y los parches visuales del vídeo durante la codificación, ciertos detalles composicionales finos o textos pequeños en pantalla pueden quedar diluidos dentro de la representación global.

Para resolver esta pérdida de precisión sin sacrificar la escalabilidad, el estado del arte introduce los modelos con **arquitectura de Torre Única (*Cross-Encoder*)**, denominados operativamente ***Rerankers*** (reordenadores) o verificadores visuales de segunda etapa, entre los que destacan **Qwen3-VL-Reranker** (en variantes de 2B y 8B) y el uso de modelos ligeros de visión-lenguaje como **Gemini 1.5 Flash Lite** o **Gemini Flash 3.5** en tareas de validación visual [3, 4].

A diferencia del *Bi-Encoder*, un *Cross-Encoder* no genera vectores independientes ni permite pre-computar representaciones de los vídeos [3]. En su lugar, recibe simultáneamente en una única secuencia concatenada la consulta del usuario Q y el fragmento multimedia candidato D_k, procesándolos conjuntamente a través de todas las capas de **Atención Cruzada (*Cross-Attention*)** del *Transformer*, como se formula en la Ecuación (7) [3, 4]:

$$s_k = g_\phi(Q, D_k) = \sigma(\mathbf{W}_{\text{head}} \cdot \text{CrossAttention}_\phi([\text{Tokens}(Q);\ \text{Patches}(D_k)])) \in [0, 1] \tag{7}$$

Mediante el mecanismo de *Cross-Attention*, cada *token* de la pregunta del usuario (por ejemplo, la palabra *"diccionarios"*) atiende directamente a cada parche visual específico de los fotogramas del candidato

D_k (por ejemplo, la región de la pantalla donde aparece escrito el código `dict {"name": "Brais"}`). Finalmente, como refleja el Cuadro de especificaciones de la familia Qwen3-VL, el *Cross-Encoder* no devuelve un vector de *embedding*, sino una **probabilidad discreta en el rango** $[0, 1]$ (o un veredicto de validación visual) que cuantifica la relevancia exacta del candidato respecto a la consulta [4].

La Figura 6 contrasta detalladamente la topología interna de un *Bi-Encoder* frente a la de un *Cross-Encoder* [3, 4].

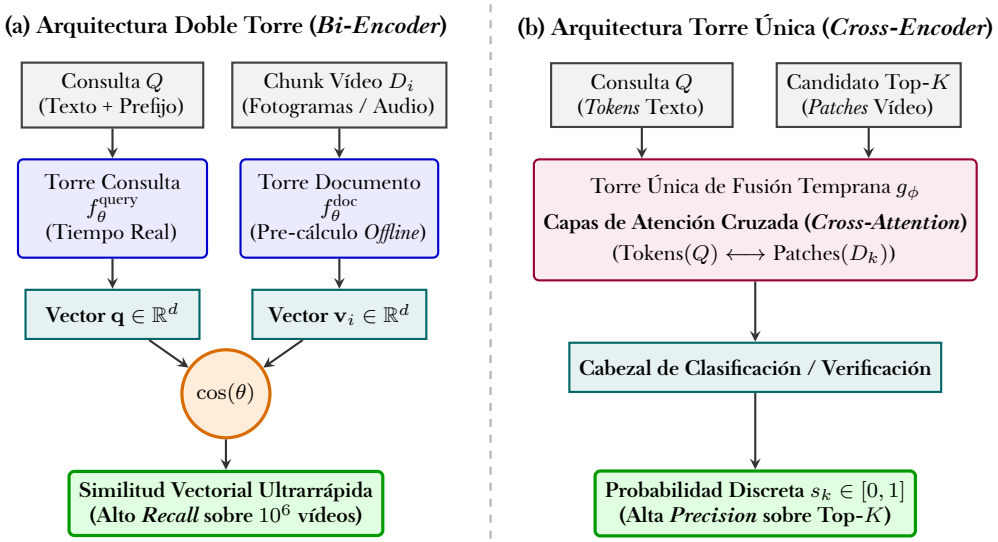


Figura 6: Comparativa estructural entre (a) un *Bi-Encoder* de doble torre que codifica consulta y vídeo de forma independiente para indexación pasiva en bases de datos vectoriales, y (b) un *Cross-Encoder* (*Reranker*) que aplica atención cruzada conjunta sobre el subconjunto de candidatos recuperados [3, 4].

2.2.5 Eficiencia Escalable: El Paradigma de Búsqueda Asimétrica en Dos Etapas (Regla 99 % / 1 %)

El resurgimiento y perfeccionamiento de la dualidad entre *Bi-Encoders* (para recuperación inicial) y *Cross-Encoders* o verificadores VLM (para refinamiento) responde directamente a los límites físicos de energía, latencia y procesamiento de hardware en sistemas de vídeo a gran escala [3, 15]. Computar de forma cruzada cada búsqueda del usuario contra terabytes de fragmentos de vídeo utilizando modelos generativos masivos (como intentar emplear un modelo tipo Llama-Vision o Gemini Pro para comparar fotograma a fotograma todo un repositorio) resulta técnica y financieramente inasumible [3].

Para resolver esta dicotomía, el estado del arte de 2026 establece una **aproximación asimétrica radical en dos etapas** conocida como la regla del 99 %/1 % [3, 4]:

- **Primera Etapa - Filtrado Masivo del 99 % (*Bi-Encoder* + Base de Datos Vectorial):** Se delega el 99 % del descarte de datos a una vectorización paralela, densa e independiente (empleando modelos como *Gemini Embedding 2* o *Qwen3-VL-Embedding*) almacenada en un motor vectorial como Qdrant [2, 4]. Ante una consulta del usuario, la búsqueda inicial se resuelve mediante un cálculo vectorial de similitud del coseno sobre índices estáticos, reduciendo instantáneamente miles de fragmentos a una lista acotada de los K mejores candidatos (por ejemplo, Top-$K = 5$ o Top-$K = 20$) [3].

- **Segunda Etapa - Refinamiento Intensivo del 1 % (*Cross-Encoder* / Verificación Visual):** El cómputo pesado, iterativo y apoyado en mecanismos de atención cruzada se reserva exclusivamente para ese 1 % de resultados candidatos recuperados en la primera fase [3]. En esta etapa, bien un modelo específico de reordenación (*Qwen3-VL-Reranker*) recalcula la probabilidad exacta de coincidencia $[0, 1]$, o bien un verificador visual ligero (*Gemini 1.5 Flash Lite* / *Gemini Flash 3.5*) inspecciona los fotogramas del *chunk* candidato para descartar falsos positivos antes de presentar los intervalos de tiempo (`start_time` - `end_time`) definitivos al usuario [3, 4].

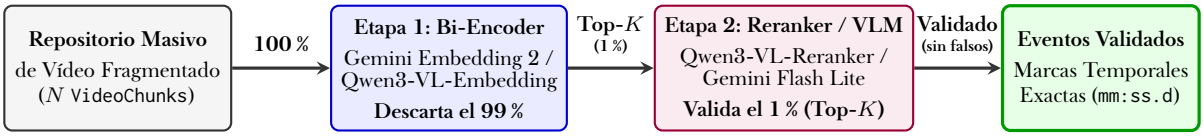


Figura 7: Flujo del paradigma de búsqueda asimétrica en dos etapas (Regla 99 % / 1 %): el *Bi-Encoder* filtra masivamente el repositorio mediante similitud del coseno en Qdrant y el *Cross-Encoder* o verificador visual inspecciona únicamente el subconjunto Top-K para garantizar máxima precisión con latencia mínima [3, 4].

El Cuadro 2 resume las diferencias arquitectónicas, operativas y matemáticas entre las tres familias de modelos que intervienen en el ciclo de recuperación y validación multimodal.

Tabla 2: Comparativa técnica entre arquitecturas *Bi-Encoder*, *Cross-Encoder* (*Reranker*) y Verificadores VLM Generativos en sistemas Video RAG [2–4].

Atributo Técnico	**Bi-Encoder (1.ª Etapa)**	**Cross-Encoder (2.ª Etapa)**	**Verificador VLM (2.ª Etapa)**
Modelos de referencia	Gemini Emb. 2 / Qwen3-Emb	Qwen3-VL-Reranker (2B/8B)	Gemini 1.5 Flash Lite / Flash 3.5
Interacción Consulta-Vídeo	Tardía (Producto escalar)	Temprana (*Cross-Attention*)	Generativa multimodal guiada
Pre-cálculo *Offline*	Sí (Indexado en Qdrant)	No (Cálculo en tiempo real)	No (Evaluación bajo demanda)
Formato de salida	Vector $\mathbf{v} \in \mathbb{R}^d$ (MRL)	Probabilidad discreta $[0, 1]$	Score + Justificación visual
Volumen procesado	100 % corpus (filtra 99 %)	1 % candidatos (Top-K)	1 % candidatos (Top-K)
Métrica optimizada	Recuerdo (*Recall@K*)	Precisión (*nDCG@K* / *Hit@1*)	Descarte Falsos Positivos

Como demuestra esta taxonomía, la combinación de una primera etapa de vectorización densa omnimodal con una segunda etapa de atención cruzada y verificación visual constituye la arquitectura idónea para localizar eventos específicos en vídeo sin incurrir en costes computacionales prohibitivos [3, 4].

2.3 Estudio Profundo de Google Gemini Embedding 2

2.3.1 Arquitectura Omnimodal Unificada y Configuración del Espacio de Salida

Dentro de la macrotendencia de modelos fundacionales propietarios accesibles mediante interfaz de programación de aplicaciones (API), **Gemini Embedding 2** (identificado técnicamente en el catálogo de desarrollo como `gemini-embedding-2-preview`) constituye el hito arquitectónico de Google hacia

la vectorización omnimodal nativa [2, 16]. A diferencia de las arquitecturas multimodales de primera generación que requerían codificadores separados y proyecciones post-hoc para cada tipo de dato, `gemini-embedding-2-preview` ha sido entrenado de extremo a extremo para ingerir texto libre, imágenes estáticas, secuencias de vídeo, pistas de audio y documentos estructurados (archivos PDF) proyectándolos directamente en un único espacio vectorial continuo [2, 17].

Esta unificación topológica implica que los vectores generados a partir de una consulta textual en lenguaje natural habitan exactamente la misma variedad geométrica que los vectores extraídos de los fotogramas de un vídeo o de una locución hablada [2]. De este modo, la localización de un evento específico dentro de un repositorio audiovisual prescinde por completo de etiquetas manuales previas o de traducciones intermedias a texto plano, resolviéndose mediante el cálculo directo de la similitud del coseno entre la consulta vectorizada y los fragmentos multimedia indexados [2].

En cuanto a la configuración algebraica de sus tensores de salida, el modelo incorpora soporte nativo para *Matryoshka Representation Learning* (MRL) a través del parámetro de configuración `output_dimensionality` integrado en la estructura `types.EmbedContentConfig` del SDK oficial `google-genai` [2]. Aunque cuando se invoca el punto de enlace sin parámetros adicionales para pruebas básicas de texto el sistema devuelve por defecto un vector compacto de 768 dimensiones, la arquitectura permite solicitar representaciones de mayor densidad (destacando los estándares operativos de 768, 1,536 y el máximo nativo de 3,072 dimensiones) [2].

Desde la perspectiva de la precisión numérica, existe una distinción crítica en el post-procesamiento de los vectores devueltos por la API según la dimensionalidad seleccionada [2]:

- **Operación en Dimensionalidad Máxima ($d = 3{,}072$):** Los vectores de 3,072 números en coma flotante devueltos por `response.embeddings[0].values` se entregan ya normalizados a longitud unitaria por el propio motor de inferencia de Google, ofreciendo la máxima capacidad de discriminación semántica para desambiguar escenas visualmente similares [2].
- **Operación en Dimensionalidades Truncadas ($d < 3{,}072$):** Cuando el ingeniero configura `output_dimensionali` en valores inferiores a 3,072 (tales como 768 o 1,536) con el objetivo de reducir el consumo de memoria en la base de datos vectorial, el recorte de las coordenadas finales altera la magnitud unitaria del vector original [2]. Por consiguiente, tal y como recomienda la documentación técnica de Google, el cliente de software debe calcular explícitamente la norma euclídea $\|\mathbf{v}\|_2 = \sqrt{\sum_{j=1}^{d} v_j^2}$ (por ejemplo, mediante `np.linalg.norm(values)`) y dividir cada componente entre dicha norma ($\hat{v}_j = v_j / \|\mathbf{v}\|_2$) siempre que $\|\mathbf{v}\|_2 > 0$, garantizando así la validez geométrica de la similitud del coseno en etapas posteriores [2].

2.3.2 Paradigmas de Ingestión Multimedia en el SDK `google-genai`: API Files frente a Agregación en Memoria

La integración programática con el modelo `gemini-embedding-2-preview` a través de la librería oficial `google-genai` habilita dos arquitecturas de acoplamiento de datos claramente diferenciadas según la naturaleza del flujo de procesamiento [2, 16]:

2.3.2.1 Ingestión de Archivos Completos mediante la API de Almacenamiento Temporal (`client.files.upload`):

Orientada al procesamiento directo de ficheros pesados almacenados en disco (clips de vídeo en formato `video/mp4`, pistas de audio, imágenes de alta resolución o documentos PDF) [2, 16]: En este patrón arquitectónico, el sistema no envía los bytes crudos directamente en la llamada de inferencia, sino que ejecuta un protocolo en tres fases [2]:

1. *Transferencia y generación de URI:* Se invoca el método `client.files.upload(file=file_path, config={'mime_type': mime_type})`, el cual aloja temporalmente el recurso multimedia en la infraestructura de Google AI Studio y devuelve un objeto con un identificador único (`uploaded_file.name`) y una referencia uniforme (`uploaded_file.uri`) [2].
2. *Bucle de sondeo de estado (Polling) para recursos temporales:* Dado que los contenedores de vídeo (`video/*`) y audio (`audio/*`) requieren una decodificación e indexación previa en los servidores de Google antes de poder ser leídos por la red neuronal, el archivo recién subido permanece inicialmente en un estado transitorio [2]. El cliente debe implementar un bucle de espera activa controlada (consultando `client.files.get(name=uploaded_file.name)` cada 2 segundos) hasta verificar que la propiedad `file_info.state.name` transiciona al estado `."CTIVE"` (o abortar lanzando una excepción si deriva al estado `"FAILED"`) [2].
3. *Extracción vectorial y limpieza:* Una vez activo el recurso, se pasa la referencia `uploaded_file` al parámetro `contents` de `client.models.embed_content`, pudiendo invocar posteriormente `client.files.delete(n` para liberar el almacenamiento remoto [2].

2.3.2.2 Ingestión por Agregación de Fotogramas Clave en Memoria (`types.Content` y `types.Part.from`

Si bien la API Files resulta idónea para clips pre-cortados en disco, en una arquitectura de fragmentación semántica adaptativa como *Video-RAC* (donde un vídeo largo se divide dinámicamente en memoria en decenas o cientos de objetos `VideoChunk` mediante OpenCV) escribir cada sub-clip en disco, subirlo a la API Files y esperar 2 segundos de sondeo por cada fragmento introduciría una latencia de entrada/salida inasumible [2, 5].

Para optimizar el rendimiento y el consumo de ancho de banda, el diseño avanzado implementado en el motor de vectorización (`GeminiMultimodalVectorizer`) toma las matrices NumPy (`NDArray[np.uint8]` en espacio de color BGR) correspondientes a los tres fotogramas representativos extraídos de cada `VideoChunk` y los codifica directamente en memoria RAM al estándar JPEG mediante `cv2.imencode(".jpg", frame, [cv2.IMWRITE_JPEG_QUALITY, jpeg_quality])` (con una calidad por defecto de 90 sobre 100) [2]. Cada búfer de bytes resultante se encapsula en un objeto `types.Part.from_bytes(data=buffer.tobytes(), mime_type=ïmage/jpeg")` y la lista completa de partes del fragmento se empaqueta en una única instancia `types.Content(parts=parts)` que se transmite de forma asíncrona mediante `client.aio.models.embed_content` [2].

La ventaja crucial de este segundo mecanismo es doble: elimina por completo la creación de archivos temporales y el bucle de espera de la API Files, y consigue que el modelo fusione internamente todos los objetos `Part` del contenedor `Content`, devolviendo **un único vector agregado** que representa la totalidad del *chunk* temporal [2].

La Figura 8 ilustra y compara ambos flujos de acoplamiento multimedia dentro de la arquitectura del SDK `google-genai`.

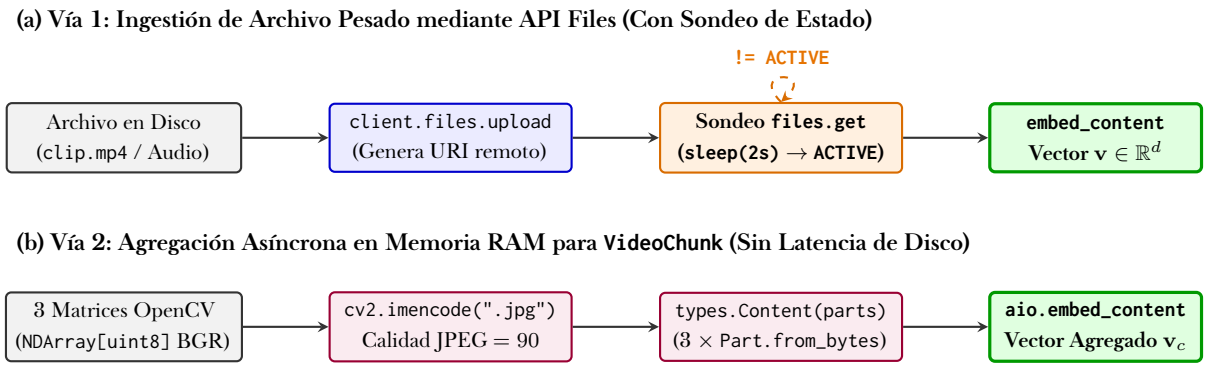


Figura 8: Comparativa técnica de los dos flujos de ingestión en el SDK `google-genai`: (a) subida de archivos completos a la API Files con bucle de espera activa hasta alcanzar el estado `ACTIVE`, y (b) codificación JPEG en memoria de múltiples fotogramas empaquetados en un único objeto `types.Content` para llamadas asíncronas de baja latencia [2].

Asimismo, para cerrar el ciclo de búsqueda cruzada y multimodal, el mismo componente vectorizador expone el método `embed_query` (respaldado por `build_query_payload` y manteniendo `embed_text` por retrocompatibilidad), capaz de empaquetar en un único contenedor `types.Content` cualquier combinación de consulta textual, ficheros `.txt`, imágenes de referencia codificadas a JPEG (`types.Part.from_bytes`) y transcripciones de pistas de audio de entrada obtenidas con `transcribe_query_audios`. Al invocar `aio.models.embed_content` en una sola llamada de red con idéntica configuración dimensional (`output_dimensionality`) y normalización L_2, el sistema garantiza la compatibilidad algebraica exacta entre los fragmentos de vídeo indexados y las consultas multimodales del usuario [2].

2.3.3 Modelado Matemático de Cuotas Operativas (RPM, TPM y RPD) y Planificación de Capacidad

Un aspecto crítico en el diseño de arquitecturas de ingeniería basadas en modelos fundacionales en la nube es la caracterización matemática de los límites de tasa (*Rate Limits*) impuestos por el proveedor del servicio [2]. En el entorno de experimentación y validación de Google AI Studio (capa gratuita o *Free Tier*) para el modelo `gemini-embedding-2-preview`, la infraestructura impone simultáneamente tres umbrales restrictivos [2]:

- **RPM (*Requests Per Minute*):** Límite máximo de $\text{RPM}_{\text{máx}} = 100$ peticiones por minuto [2].
- **TPM (*Tokens Per Minute*):** Límite máximo de $\text{TPM}_{\text{máx}} = 30{,}000$ *tokens* procesados por minuto [2].
- **RPD (*Requests Per Day*):** Límite máximo acumulado de $\text{RPD}_{\text{máx}} = 1{,}000$ peticiones por día [2].

Un análisis superficial podría sugerir que el sistema es capaz de vectorizar 100 fragmentos de vídeo (`VideoChunk`) por minuto atenidos al umbral de $\text{RPM}_{\text{máx}} = 100$. Sin embargo, el modelado riguroso de la carga útil multimedia demuestra que el verdadero cuello de botella por minuto reside en el consumo de *tokens* ($\text{TPM}_{\text{máx}}$) [2]. Cada imagen o fotograma enviado a los modelos de la familia Gemini consume

una tasa base de aproximadamente $T_{\text{img}} \approx 258$–$300$ *tokens* [2]. Dado que el algoritmo de selección por entropía extrae $N_{\text{frames}} = 3$ fotogramas clave por cada `VideoChunk`, el coste en *tokens* de una única petición de fragmento asciende según la Ecuación (8) a:

$$T_{\text{chunk}} = N_{\text{frames}} \times T_{\text{img}} \approx 3 \times 300 = 900 \text{ tokens/chunk} \tag{8}$$

Si el orquestador enviase el máximo permitido de 100 peticiones en un minuto, la demanda de caudal ascendería a $100 \times 900 = 90{,}000$ *tokens*/minuto, triplicando el límite de $\text{TPM}_{\text{máx}} = 30{,}000$ y provocando el rechazo inmediato de las peticiones con código de error HTTP `429 (Too Many Requests)`. En consecuencia, el caudal máximo efectivo sostenido ($\lambda_{\text{efectivo}}$) en fragmentos por minuto queda determinado según la Ecuación (9) por el mínimo entre la restricción de peticiones y la restricción de *tokens*:

$$\lambda_{\text{efectivo}} = \text{mín}\left(\text{RPM}_{\text{máx}}, \left\lfloor \frac{\text{TPM}_{\text{máx}}}{T_{\text{chunk}}} \right\rfloor\right) = \text{mín}\left(100, \left\lfloor \frac{30{,}000}{900} \right\rfloor\right) \approx 33 \text{ chunks/minuto} \tag{9}$$

Esto restringe la velocidad de ingestión real a un rango de entre 30 y 35 fragmentos de vídeo por minuto, lo que significa que un vídeo de prueba que genere entre 100 y 200 *chunks* requerirá entre 3 y 6 minutos de procesamiento continuo.

De forma análoga, el límite diario de $\text{RPD}_{\text{máx}} = 1{,}000$ peticiones acota la duración máxima de vídeo que puede indexarse en una jornada de experimentación. Bajo una segmentación semántica adaptativa (*Video-RAC*), un vídeo educativo o divulgativo promedio experimenta un cambio de plano cada $\bar{d}_{\text{plano}} \in [3, 7]$ segundos. Asumiendo una media conservadora de $\bar{d}_{\text{plano}} = 5$ segundos por fragmento, cada minuto de metraje produce $C_{\text{min}} = 60/5 = 12$ *chunks*/minuto, lo que arroja un límite teórico absoluto de duración diaria ($D_{\text{máx}}$) formulado en la Ecuación (10):

$$D_{\text{máx}} = \frac{\text{RPD}_{\text{máx}}}{C_{\text{min}}} = \frac{1{,}000 \text{ chunks/día}}{12 \text{ chunks/minuto}} \approx 83{,}3 \text{ minutos/día} \tag{10}$$

Para evitar que el procesamiento accidental de un único archivo de larga duración agote el 100 % de la cuota diaria e impida realizar búsquedas o iteraciones de calibración durante el resto del día, la arquitectura de ingeniería implementa dos salvaguardas: acotar los vídeos de validación de la Prueba de Concepto a clips de entre **5 y 15 minutos** (que generan entre 100 y 200 *chunks*, consumiendo apenas un 10 %–20 % del RPD), e incorporar en el orquestador un cortafuegos estático (*fail-safe*) definido por la constante `MAX_CHUNKS_PER_VIDEO = 800`. Si tras la detección de bordes de plano con OpenCV el vídeo supera los 800 fragmentos, el sistema aborta preventivamente lanzando la excepción `QuotaSafetyError` antes de extraer fotogramas ni consumir una sola petición de la API.

La Figura 9 esquematiza la interacción entre los tres umbrales de cuota y los mecanismos de protección implementados en el flujo de ingestión.

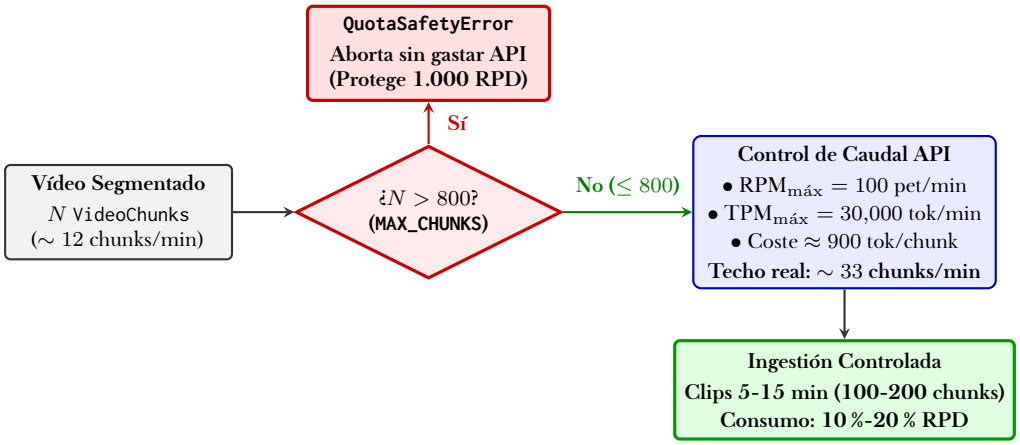


Figura 9: Modelado de capacidad y cortafuegos de cuota para `gemini-embedding-2-preview`: la validación previa aborta ejecuciones que superen los 800 fragmentos para proteger el límite diario (1.000 RPD), mientras que la densidad de $\sim$ 900 *tokens* por fragmento fija el caudal efectivo en $\sim$ 33 *chunks*/minuto bajo el techo de 30.000 TPM [2].

2.3.4 Ingeniería de Resiliencia: Control de Concurrencia, Ventana de *Backoff* de 60 Segundos y Degradación Elegante

Dado que el procesamiento de vídeo combina operaciones intensivas de CPU (decodificación de fotogramas y cálculo de mapas gaussianos SSIM en OpenCV) con operaciones asíncronas de red (llamadas HTTP a la API de Gemini), la arquitectura de software debe evitar tanto el bloqueo del bucle de eventos como la saturación de los servidores remotos.

En primer lugar, las funciones síncronas bloqueantes de visión por computador (`detect_shot_boundaries` y `extract_key_frames`) se delegan a hilos secundarios mediante `asyncio.to_thread`, liberando el hilo principal de Python para gestionar las conexiones de red. En segundo lugar, la emisión paralela de tareas de vectorización en `asyncio.gather` se regula mediante un semáforo asíncrono (`asyncio.Semaphore(max_concurrent`) calibrado específicamente en `max_concurrent_requests = 3` (reducido respecto a un valor inicial de 5) para impedir ráfagas instantáneas que desborden las cuotas por minuto.

Pese al uso del semáforo, dado que la latencia de respuesta de una petición individual es inferior a un segundo, tres corrutinas concurrentes pueden alcanzar el techo de 30,000 *tokens* antes de que se cumpla el minuto natural, recibiendo un error transitorio HTTP `429`. Para absorber este fenómeno sin perder datos (así como fallos transitorios de servidor catalogados en el conjunto inmutable `_RETRYABLE_CODES = {429, 500, 502, 503, 504}` y errores de red `OSError/TimeoutError`), el método privado `_embed` implementa un algoritmo de **reintento con espera exponencial (*Exponential Backoff*)**.

La calibración matemática de los parámetros de este *backoff* resulta determinante: si se utilizase una espera base de $t_0 = 2,0$ s con $R = 3$ reintentos, la suma de las pausas sería únicamente de $2+4+8+16 = 28$ s (o $2+4+8 = 14$ s en tres esperas), tiempo insuficiente para que Google reinicie la ventana deslizante de 60 segundos del límite TPM. Por el contrario, al incrementar los parámetros por defecto a un retardo base de `retry_base_delay` $= 4,0$ s y un máximo de `max_retries` $= 4$ reintentos, el tiempo de espera en el intento $k \in \{0, 1, 2, 3\}$ responde a $\Delta t_k = 4,0 \times 2^k$, generando una ventana acumulada calculada según la Ecuación (11):

$$T_{\text{espera total}} = \sum_{k=0}^{3} t_0 \cdot 2^k = 4,0 \times (2^0 + 2^1 + 2^2 + 2^3) = 4 + 8 + 16 + 32 = 60 \text{ segundos} \tag{11}$$

La Figura 10 muestra cómo esta progresión geométrica cubre exactamente los 60 segundos necesarios para garantizar la renovación de la cuota de *tokens* por minuto antes de agotar el último intento.

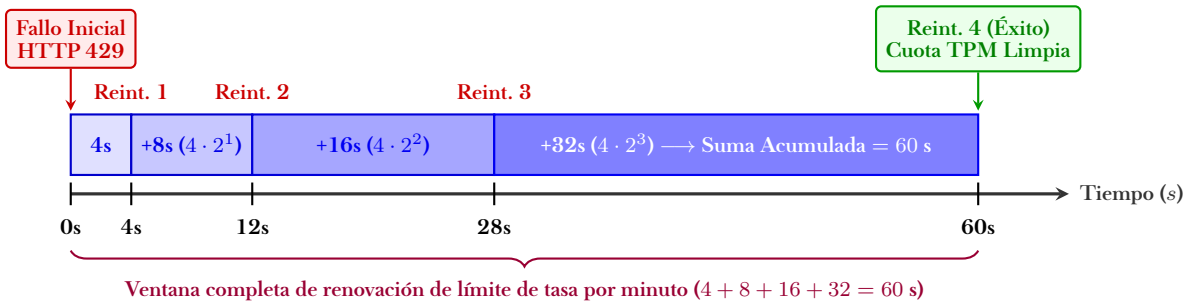


Figura 10: Cronograma temporal del algoritmo de *Exponential Backoff* calibrado (`retry_base_delay = 4.0`, `max_retries = 4`): la suma geométrica de las cuatro esperas alcanza exactamente 60 segundos, permitiendo superar bloqueos completos de la ventana de 30.000 TPM sin abortar la vectorización del fragmento.

Adicionalmente, el diseño incorpora dos mecanismos de **Degradación Elegante (*Graceful Degradation*)** en distintas capas del sistema para garantizar una tolerancia a fallos de nivel industrial:

1. **Aislamiento de fallos parciales en la fase de ingestión (`return_exceptions=True`):** Si tras agotar los 60 segundos de reintentos un fragmento puntual sigue fallando (lanzando `VectorizationError`), la invocación `asyncio.gather(* tasks, return_exceptions=True)` evita que se cancele el lote completo. El orquestador captura la excepción de ese único *chunk*, la documenta en `execution.log` mediante el auditor (`RunAuditor`) y conserva intactos todos los demás fragmentos vectorizados del vídeo.

2. **Respaldo vectorial ante caídas `503 UNAVAILABLE` en la segunda etapa de reordenación visual:** Durante la fase de búsqueda y refinamiento con modelos ligeros de visión-lenguaje (como `gemini-3.5-flash-lite`), los servidores de Google pueden experimentar picos temporales de demanda devolviendo el error `503 UNAVAILABLE ("This model is currently experiencing high demand")`. Cuando esto ocurre y se agotan los reintentos del verificador secundario, el sistema no colapsa ni devuelve un error al usuario: intercepta la indisponibilidad, emite un aviso informando que la reordenación visual no está disponible temporalmente y **conserva automáticamente el orden vectorial original devuelto por Qdrant** en la primera etapa.

La Figura 11 representa este flujo de continuidad operativa y degradación elegante en ambas fases de la arquitectura.

El Cuadro 3 sintetiza los parámetros definitivos de ingeniería calibrados en el sistema para maximizar la fiabilidad bajo las restricciones operativas de la API de Google.

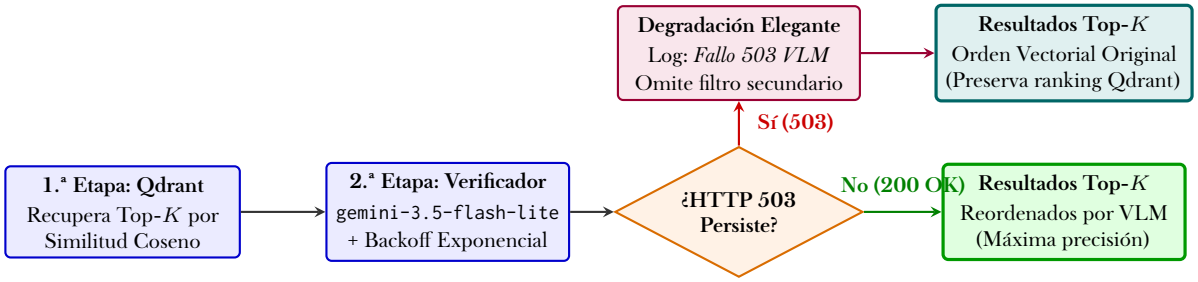


Figura 11: Arquitectura de Degradación Elegante (*Graceful Degradation*) durante la búsqueda: si los servidores del modelo secundario (`gemini-3.5-flash-lite`) sufren picos de saturación (`503 UNAVAILABLE`) tras agotar los reintentos, el sistema conmuta limpiamente al orden de similitud vectorial calculado por Qdrant con *Gemini Embedding 2*.

Tabla 3: Parámetros técnicos calibrados en los módulos de vectorización y orquestación para operar con `gemini-embedding-2-preview` sin colapsos de cuota [2].

Parámetro	Módulo	Valor	Justificación Técnica / MLOps
`model`	`gemini_vectorizer.py`	`gemini-embedding-2`	Modelo omnimodal nativo (espacio compartido)
`output_dimensionality`	`gemini_vectorizer.py`	3,072, 1,536 o 768	MRL con normalización L_2 (`truncate_and_normalize_mrl`)
`jpeg_quality`	`gemini_vectorizer.py`	90 (escala 1-100)	Alta fidelidad visual en RAM sin artefactos
`max_concurrent_requests`	`pipeline.py`	3 (reducido de 5)	Limita peticiones concurrentes en semáforo
`retry_base_delay`	`gemini_vectorizer.py`	4, 0 s (antes 2, 0 s)	Base geométrica para cubrir ventana TPM
`max_retries`	`gemini_vectorizer.py`	4 (antes 3)	Suma $4+8+16+32=60$ s ante error HTTP
`MAX_CHUNKS_PER_VIDEO`	`pipeline.py`	800 fragmentos	Cortafuegos `QuotaSafetyError` (protege RPD)

2.4 Ecosistema de Modelos Alternativos: Pesos Abiertos (*Open-Weights*) y Ejecución Local

2.4.1 Las Dos Macrotendencias de 2026: APIs Propietarias Hiper-escaladas frente a Pesos Abiertos

En el diseño de una plataforma de recuperación profunda de vídeo, la elección de la arquitectura del modelo de *embedding* subyacente constituye el factor crítico que define los límites de escalabilidad, la precisión en la localización de eventos y el coste computacional del sistema [2]. El estado del arte tecnológico en 2026 se estructura en torno a dos macrotendencias complementarias [2, 4]:

1. **Modelos Propietarios Hiper-escalados vía API (*Cloud*):** Representados por servicios como *Google Gemini Embedding 2* (`gemini-embedding-2-preview`), donde la complejidad de la infraestructura de aceleradores (TPUs/GPUs) queda abstraída tras llamadas HTTP, ofreciendo espacios vectoriales masivos de hasta 3,072 dimensiones con soporte nativo para vídeo, audio, imagen, texto y documentos PDF [2, 17].
2. **Ecosistemas de Pesos Abiertos (*Open-Weights*) y Despliegue Local:** Integrados por familias de modelos cuyos pesos neuronales son públicamente accesibles en repositorios como Hugging Face (destacando la serie **Qwen3-VL-Embedding** y **Qwen3-VL-Reranker**, arquitecturas como **ColQwen** y variantes ligeras de la familia **Gemma**), los cuales pueden ejecutarse en servidores locales propios o en instancias privadas sin depender de proveedores externos [1, 4, 7].

Un hallazgo fundamental documentado en la literatura del primer trimestre de 2026 es que las implementaciones de código abierto han logrado cerrar la brecha histórica de rendimiento frente a las APIs propietarias en tareas de recuperación multimodal [4, 18]. Esta convergencia técnica tiene una repercusión directa en el ámbito universitario e industrial: posibilita el diseño de infraestructuras de investigación académica y de producción que no dependen del pago continuo por inferencia (*pay-per-token*) ni se ven estranguladas por límites diarios de peticiones, garantizando además la soberanía y privacidad absoluta sobre archivos de vídeo sensibles [4].

La Figura 12 presenta la taxonomía comparada de ambas corrientes tecnológicas dentro del panorama de modelos fundacionales multimodales de 2026.

2.4.2 La Familia Qwen3-VL: Recuperación y Reordenación Multimodal a Escala Masiva

En el espectro del software libre y los modelos de pesos abiertos, la referencia técnica para la recuperación de información multimodal viene marcada por el lanzamiento en enero de 2026 de la serie **Qwen3-VL-Embedding** y **Qwen3-VL-Reranker** por parte del grupo de investigación de Alibaba [4, 19, 20]. Publicados oficialmente en configuraciones de 2,000 millones (`Qwen/Qwen3-VL-Embedding-2B`) y 8,000 millones de parámetros (`Qwen/Qwen3-VL-Embedding-8B`), junto con adaptaciones especializadas en recuperación de documentos visuales (`tomaarsen/Qwen3-VL-Embedding-2B-vdr`), estos modelos abordan el problema del RAG sobre grandes repositorios de vídeo mediante un diseño acoplado en dos etapas que supera las limitaciones de latencia de los sistemas generativos directos [4, 21–23].

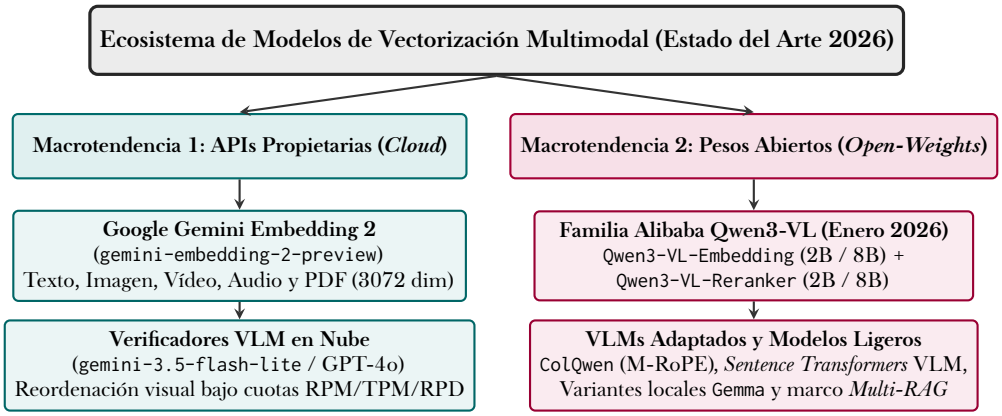


Figura 12: Clasificación de las arquitecturas de modelos fundacionales para Video RAG en 2026: coexistencia de modelos hiper-escalados servidos mediante API comercial frente a ecosistemas de pesos abiertos desplegables en infraestructura local [2, 4, 6].

2.4.2.1 Primera Etapa (`Qwen3-VL-Embedding`):

Actúa como un codificador denso de doble torre (*Bi-Encoder*) construido sobre la arquitectura base de visión y lenguaje de la familia Qwen3 [3, 4]. Gracias a la integración nativa de *Multimodal Rotary Position Embeddings* (M-RoPE), el modelo proyecta parches y teselas visuales de imágenes y secuencias de vídeo directamente hacia el mismo espacio semántico continuo del lenguaje sin requerir preprocesamiento OCR intermedio [1, 4]. Estos vectores se almacenan en bases de datos vectoriales como Qdrant o Milvus, permitiendo realizar búsquedas semánticas directas sobre vídeo bruto a gran velocidad [4].

2.4.2.2 Segunda Etapa (`Qwen3-VL-Reranker`) y Resolución de Consultas Composicionales:

El aporte arquitectónico más relevante de la investigación del equipo Qwen radica en demostrar empíricamente por qué un *Bi-Encoder* aislado resulta insuficiente ante consultas visualmente complejas [3, 4]. Los codificadores de doble torre tienden a degradar su precisión ante **consultas altamente composicionales**, perdiendo relaciones espaciales, direccionales o de atributos cruzados; un caso paradigmático documentado en la literatura es la búsqueda de un evento descrito como *"una persona vistiendo una chaqueta roja que se aleja de una furgoneta blanca"* [3]. En un *Bi-Encoder*, un fragmento de vídeo donde aparezca una furgoneta roja y una persona con chaqueta blanca acercándose a ella puede arrojar una similitud del coseno artificialmente alta debido a la presencia conjunta de los mismos sustantivos y colores en el vector global.

Para erradicar estos falsos positivos composicionales, `Qwen3-VL-Reranker` actúa como un filtro de alta precisión sobre el subconjunto top-K recuperado en la primera fase [3, 4]. Operando bajo una arquitectura de Torre Única (*Cross-Encoder*), el modelo ingiere simultáneamente la consulta textual Q y el fragmento multimedia candidato D_k, permitiendo que sus capas de atención cruzada (*Cross-Attention*) interactúen de manera bidireccional y profunda entre cada *token* léxico del texto y cada parche espacial de los fotogramas del vídeo [3, 4]. En lugar de emitir un vector de representación, la red finaliza prediciendo según la Ecuación (12) un ***logit* de probabilidad absoluta** que cuantifica la relevancia exacta de la coincidencia entre el evento visual y la estructura sintáctica de la consulta, maximizando la precisión en las primeras posiciones del ranking [3, 4].

$$P(\text{Relevante} \mid Q, D_k) = \sigma\big(z_{\text{logit}}(Q, D_k)\big) = \frac{1}{1 + \exp(-\mathbf{w}^\top \mathbf{h}_{\text{Cross-Attention}}(Q, D_k))} \tag{12}$$

La Figura 13 ilustra el mecanismo mediante el cual `Qwen3-VL-Reranker` desambigua relaciones espaciales y de atributos sobre los candidatos recuperados por `Qwen3-VL-Embedding`.

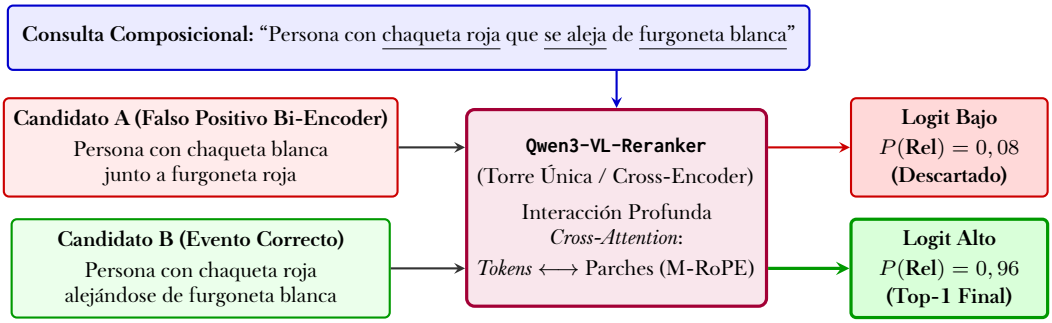


Figura 13: Resolución de consultas altamente composicionales mediante la arquitectura en dos etapas de la familia Qwen3-VL: mientras el *Bi-Encoder* puede recuperar candidatos con atributos invertidos, las capas de *Cross-Attention* de `Qwen3-VL-Reranker` vinculan cada adjetivo y acción con su parche visual exacto para predecir el *logit* de probabilidad real [3, 4].

2.4.3 Adaptación de Modelos Visuales (ColQwen y Sentence Transformers) y Entornos Restringidos (*Multi-RAG*)

Más allá de la serie Qwen3-VL, la comunidad investigadora ha consolidado metodologías reproducibles para entrenar y ajustar (*finetuning*) modelos de *embeddings* y *rerankers* multimodales a partir de arquitecturas de lenguaje visual preexistentes, tal y como documenta el marco de *Sentence Transformers* multimodales de Hugging Face [7, 24].

Dentro de esta línea destaca **ColQwen**, una arquitectura que integra directamente *Multimodal Rotary Position Embeddings* (M-RoPE) sobre el codificador visual para proyectar parches y teselas de documentos complejos, gráficos, ecografías o diapositivas con ecuaciones matemáticas hacia el espacio latente del lenguaje sin pasar por motores OCR [1]. El impacto estructural de este diseño en entornos empresariales y académicos radica en la eliminación de las antiguas cadenas de dependencias compuestas por múltiples APIs de bajo nivel: el modelo fundacional interactúa directamente con la señal visual sin pulir y mapea inherentemente el significado cruzado sin intervención ortográfica, suprimiendo la cascada de errores que un OCR defectuoso propagaba hacia las etapas posteriores del razonamiento algorítmico [1].

La Figura 14 esquematiza este flujo de adaptación de un VLM hacia un motor de indexación directa de parches visuales.

De manera complementaria, cuando los sistemas deben ejecutarse en dispositivos locales con recursos limitados (escenario donde se ubican las variantes ligeras de ejecución local como la familia **Gemma** y marcos de trabajo en tiempo real), la literatura reciente aporta la arquitectura **Multi-RAG** (*A Multimodal Retrieval-Augmented Generation System for Adaptive Video Understanding*) [6, 25]. *Multi-RAG* introduce una capa de aplicación práctica orientada a la comprensión adaptativa de flujos de vídeo capturados en tiempo real, concebida específicamente como soporte cognitivo para la interacción humano-robot en

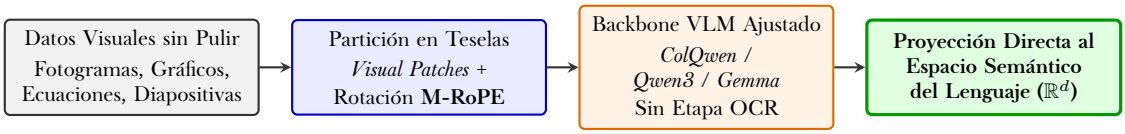


Figura 14: Flujo de codificación visual directa en arquitecturas como *ColQwen* y *Sentence Transformers* multimodales: la combinación de teselas visuales con codificación posicional M-RoPE proyecta diapositivas, gráficos y escenas de vídeo directamente al espacio del lenguaje, eliminando el preprocesamiento intermedio mediante OCR [1, 7].

contextos dinámicos y no estructurados del mundo real [6, 25]. La aportación fundamental de *Multi-RAG* demuestra que los *embeddings* multimodales son capaces de mantener una inferencia semántica de alto rendimiento incluso cuando la entrada visual se ve temporalmente degradada, ocluida o limitada, validando la robustez de estas representaciones vectoriales en escenarios con fuertes restricciones de recursos computacionales [6, 25].

2.4.4 Evaluación en Bancos de Pruebas Masivos (MMEB-V2 y MVEB) y Compromisos de Despliegue

Para cuantificar de manera objetiva y estandarizada la capacidad de recuperación de estos modelos fundacionales sobre vídeo, audio y documentos complejos, la comunidad científica ha establecido entre 2025 y 2026 dos bancos de pruebas (*benchmarks*) masivos de referencia [4, 18]:

- **MMEB-V2 (*Massive Multimodal Embedding Benchmark V2*, 2025):** Amplió la evaluación tradicional de representaciones estáticas texto-imagen para incorporar tareas exigentes de recuperación de fragmentos de vídeo y documentos estructurados (archivos PDF) [4, 18]. Desde el punto de vista metodológico, las métricas empleadas en *MMEB-V2* para clasificación de imágenes y recuperación de vídeo se fundamentan predominantemente en ***Recall@1*** y ***Hit@1***, mientras que los dominios de recuperación de documentos visuales evalúan el ordenamiento mediante ***NDCG@5*** [4, 18].
- **MVEB (*Massive Video Embedding Benchmark*, 2026):** Constituye el estándar específico de evaluación temporal más exhaustivo hasta la fecha, consolidando más de 180 tareas diferenciadas diseñadas para poner a prueba cómo los modelos de *embeddings* recuperan eventos combinando secuencias de vídeo y pistas de audio.

A partir de la evidencia empírica recogida en estos bancos de pruebas y de las especificaciones operativas de cada ecosistema, un ingeniero de datos debe ponderar los compromisos arquitectónicos (*trade-offs*) entre desplegar la solución apoyándose en una API propietaria en la nube frente a alojar modelos de pesos abiertos en infraestructura local [2, 4]. La Figura 15 sintetiza la matriz de decisión técnica entre ambos enfoques.

El Cuadro 4 consolida las especificaciones técnicas de las familias de modelos analizadas en este bloque, detallando su topología, capacidades modales y métricas de referencia en el estado del arte.

Como demuestra este análisis comparado, mientras que los modelos de pesos abiertos como *Qwen3-VL-Embedding* y *Qwen3-VL-Reranker* constituyen la opción óptima cuando se dispone de servidores con

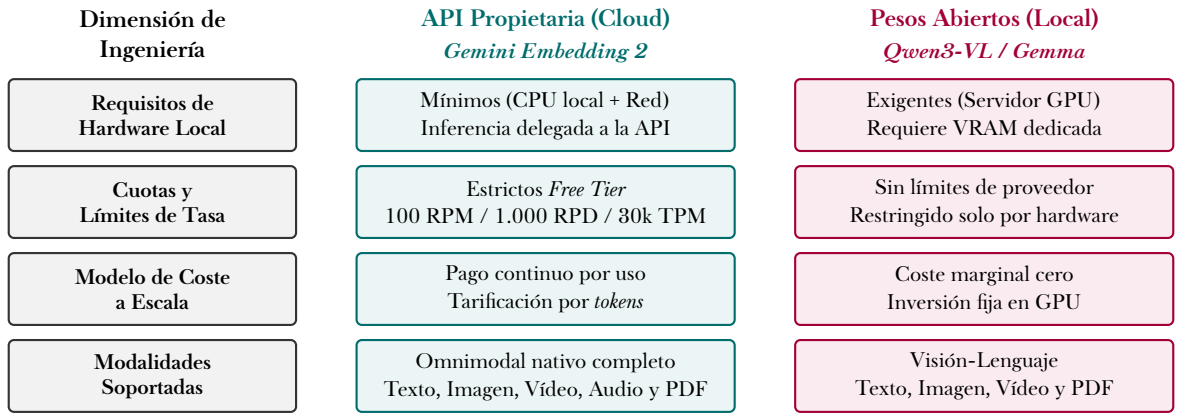


Figura 15: Comparativa de compromisos de ingeniería entre el despliegue sobre API propietaria en la nube y la ejecución local con modelos de pesos abiertos para sistemas de recuperación de vídeo [2, 4].

Tabla 4: Síntesis comparativa de las principales arquitecturas y modelos del ecosistema multimodal analizados en la literatura (2025-2026) [1, 2, 4, 6].

Modelo / Sistema	**Despliegue**	**Topología**	**Innovación Clave**	**Evaluación / Rol Principal**
Gemini Embedding 2	Cloud (API)	Bi-Encoder	Omnimodal nativo (3.072 dim)	*Baseline* PoC sin GPU local
Qwen3-VL-Embedding	Local (2B/8B)	Bi-Encoder	M-RoPE + Vectores MRL	Recuperación masiva en MMEB-V2
Qwen3-VL-Reranker	Local (2B/8B)	Cross-Encoder	*Cross-Attention* + Logit	Consultas composicionales (Top-K)
ColQwen	Local (Open)	Interacción ViT	Parches directos con M-RoPE	Eliminación total de etapa OCR
Multi-RAG / Gemma	Local / Edge	Adaptativo	Inferencia ante vídeo degradado	Robótica y recursos restringidos

aceleración GPU dedicada y se requiere indexar volúmenes masivos sin coste por inferencia, el uso de *Gemini Embedding 2* junto con una base de datos vectorial embebida como Qdrant permite desarrollar y validar una Prueba de Concepto omnimodal completa en ordenadores convencionales sin hardware gráfico especializado, siempre que se instrumenten los controles de concurrencia y cuota detallados en la sección anterior [2, 4].

2.5 Sistemas RAG Multimodales (mRAG) y Comprensión de Vídeo de Larga Duración

2.5.1 Arquitecturas de Referencia en Comprensión de Vídeo: *Video-RAG, CARVE* y *Multi-RAG*

La evolución de los sistemas de Recuperación Aumentada por Generación (RAG) ha transitado desde arquitecturas puramente textuales hacia paradigmas multimodales capaces de procesar, alinear e interpretar información visual, auditiva y textual dentro de un único espacio semántico continuo [1, 6]. En el dominio específico de la recuperación de eventos sobre secuencias de vídeo de larga duración (*Long-form Video Understanding*), esta transición permite localizar sucesos puntuales mediante consultas en lenguaje natural sin depender de metadatos anotados manualmente antes de la indexación [8].

Dentro de la literatura científica consolidada entre 2024 y 2026, destacan tres marcos arquitectónicos que abordan de manera complementaria la alineación visual, la granularidad temporal y la robustez operativa [6, 8]:

- **Marco *Video-RAG* (*Visually-aligned Retrieval-Augmented Long Video Comprehension*):** Propone una arquitectura de recuperación alineada visualmente que complementa los fotogramas del vídeo mediante la extracción e indexación paralela de tres canales auxiliares: texto visible en pantalla obtenido por OCR, transcripciones auditivas y metadatos estructurados de detección de objetos [8]. Al recuperar conjuntamente las señales visuales y sus descriptores alineados, el sistema reduce la carga de contexto sobre el modelo generativo al analizar vídeos extensos [8].
- **Sistema *CARVE* (*Cross-Modal and Multi-Granularity Retrieval*):** Aborda el problema de que los eventos de interés en un vídeo no poseen una duración uniforme (algunos sucesos duran apenas un segundo (como un gesto o la aparición fugaz de una línea de código) mientras que otros abarcan varios minutos (como la explicación completa de un diagrama)). *CARVE* introduce un mecanismo de recuperación intermodal multigranular capaz de evaluar simultáneamente distintas escalas temporales, logrando superar de forma consistente a los métodos tradicionales de ventana única en métricas estándar de ordenación como ***Recall@5*** y ***nDCG@5***.
- **Plataforma *Multi-RAG* (*Adaptive Video Understanding*):** Introduce una capa de aplicación práctica concebida como soporte cognitivo para la interacción humano-robot sobre flujos de vídeo capturados en tiempo real en entornos dinámicos y no estructurados [6, 25]. Su principal aportación científica reside en demostrar que las representaciones vectoriales multimodales mantienen una inferencia semántica de alto rendimiento incluso cuando la señal visual se encuentra temporalmente degradada o limitada, evidenciando la viabilidad de los *embeddings* multimodales bajo fuertes restricciones de recursos [6].

La Figura 16 sintetiza los principios de diseño de estos tres sistemas de referencia en el estado del arte.

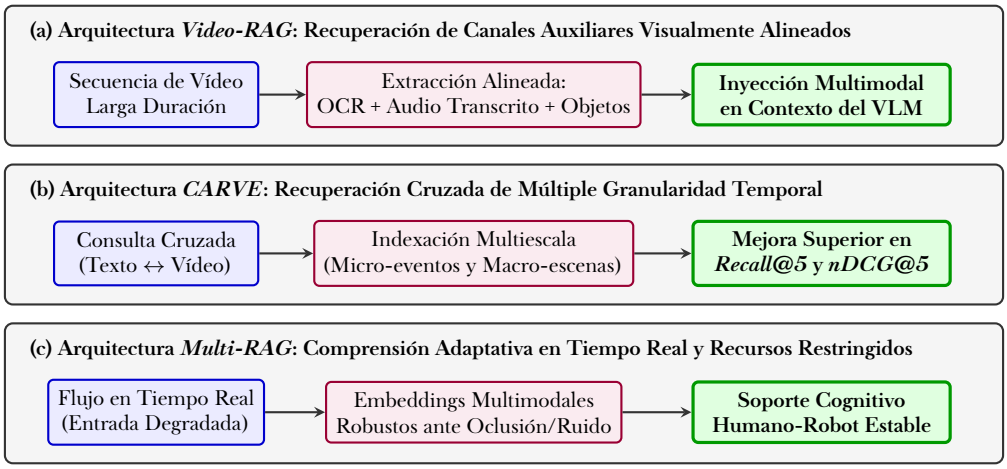


Figura 16: Comparativa de tres arquitecturas representativas de sistemas RAG de vídeo en la literatura reciente: (a) alineación de metadatos auxiliares en *Video-RAG*, (b) indexación multigranular en *CARVE* y (c) inferencia adaptativa resiliente en *Multi-RAG* [6, 8].

2.5.2 Del Emparejamiento Semántico al Anclaje Causal Espaciotemporal (*Temporal Grounding*)

El avance conceptual más profundo que caracteriza a la generación actual de sistemas Video RAG es el desplazamiento cualitativo desde el "análisis temático" hacia el verdadero **razonamiento y anclaje causal espaciotemporal (*Temporal Grounding*)** [2, 9]. Los sistemas RAG tradicionales aplicados a multimedia se limitaban a responder preguntas resumiendo la temática general de un archivo (por ejemplo, devolver una síntesis genérica indicando que *"el vídeo debate sobre economía"*).

Por el contrario, la convergencia de la fragmentación semántica adaptativa (*Video-RAC*), los modelos de transcripción con alineación forzada (*WhisperX*) y la diarización acústica de hablantes con modelos de *embeddings* omnimodales permite que las arquitecturas contemporáneas formulen aserciones fundamentadas con precisión de milisegundos sobre la línea de tiempo [5, 9]:

> *"Exactamente en el segmento 14:32, el segundo hablante señala el cuadrante superior derecho de la tabla, y el gráfico de barras indica un descenso de los ingresos, lo cual contradice el resumen textual proporcionado al inicio de la sesión"* [9].

En este paradigma de *grounding* temporal, el modelo cognitivo no inventa relaciones abstractas ni realiza aproximaciones difusas: se posiciona sobre un eje cronológico determinista $[t_{\text{inicio}}, t_{\text{fin}}]$ y discrimina el orden causal de los acontecimientos dentro de secuencias densas [2, 9]. La Figura 17 contrasta la salida de un sistema de emparejamiento temático clásico frente a un motor dotado de anclaje causal espaciotemporal.

Esta capacidad de anclaje milimétrico y judicialmente auditable habilita cinco dominios estratégicos de aplicación documentados en el proyecto [2]:

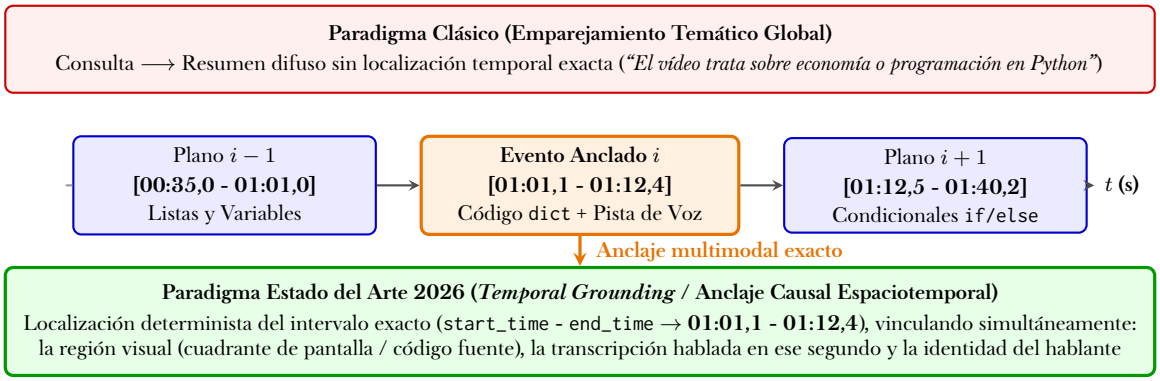


Figura 17: Contraste entre el emparejamiento temático tradicional y el Anclaje Causal Espaciotemporal (*Temporal Grounding*), el cual proporciona trazabilidad determinista y auditable sobre la línea de tiempo del vídeo [2, 9].

1. **Educación y Repositorios Académicos (RAG de Clases Grabadas):** Permite a un estudiante consultar *"minuto donde el profesor explica la mitosis usando el diagrama en la pizarra"* o localizar el instante exacto en que un tutorial de programación explica cómo construir listas, diccionarios o condicionales `if/else`, razonando sobre la conjunción del audio y el código en pantalla.
2. **Auditoría de Seguridad y Videovigilancia:** Capacidad para inspeccionar cientos de horas de cámaras de seguridad sin supervisión humana mediante consultas en lenguaje natural como *"coche rojo saltándose un semáforo a gran velocidad"* o *"persona abandonando una mochila gris"*, recuperando las marcas de tiempo exactas.
3. **Gestión de Archivos de Medios y *Broadcasting*:** Localización instantánea en archivos históricos de televisión mediante búsquedas que combinan carga visual y entonación acústica, tales como *"momentos de tensión en debates políticos"* o *"personas llorando de alegría"*.
4. **Análisis de Historiales Médicos y Diagnóstico Continuo:** Construcción de un perfil vectorial unificado que proyecta los informes clínicos en texto libre junto con las pruebas diagnósticas de imagen (radiografías, resonancias o ecografías) para encontrar casos clínicos equivalentes o auditar secuencias médicas.
5. **Comercio Electrónico y Búsqueda Visual Cruzada:** Recuperación de artículos específicos en catálogos multimedia a partir de una fotografía o una nota de voz descriptiva del usuario (*"busco un sofá verde esmeralda de terciopelo con patas de madera"*).

2.5.3 Cuellos de Botella Perceptivos en la Verificación Visual: Resolución Espacial y Compresión JPEG

Si bien la teoría de los modelos fundacionales multimodales postula la eliminación de los motores OCR tradicionales gracias a la lectura directa de parches visuales [1], la experimentación empírica en sistemas reales revela que el **OCR implícito** de un modelo de visión-lenguaje es altamente sensible a las transformaciones de preprocesamiento de imagen aplicadas en el *pipeline* de ingestión.

En una arquitectura de búsqueda en dos etapas que utiliza *Gemini Embedding 2* (vía Qdrant) para re-

cuperar los fragmentos candidatos y emplea un modelo ligero secundario (*Gemini 1.5 Flash Lite* o *Gemini Flash 3.5*) como verificador visual para descartar falsos positivos, un ajuste agresivo de los hiperparámetros de codificación visual induce fallos críticos de falso rechazo (falsos negativos).

Durante las pruebas empíricas sobre el vídeo de validación `python_tutorial.mp4` (*"Python en 8 Minutos"*), al ejecutar la consulta `"Diccionarios"`, el motor vectorial recuperó en la quinta posición el intervalo temporal 01:01,1 – 01:40,2 (61, 1 s – 100, 2 s). Aunque la verdad terreno (*ground truth*) del vídeo muestra explícitamente en pantalla entre los segundos 01:01,0 y 01:03,0 la sintaxis de un diccionario en Python (`dict {"name": "Brais"}`), el verificador visual descartó erróneamente el fragmento emitiendo el diagnóstico: *"Descartado (Falso Positivo: 0 %) - Análisis visual: El segmento cubre listas y variables, pero no muestra diccionarios"*.

La auditoría técnica de los parámetros de indexación demuestra que este fallo perceptivo responde a dos cuellos de botella en la codificación de los fotogramas clave antes de su envío a la API:

1. **Resolución espacial destructiva (`resize_width = 512` px):** Redimensionar un videotutorial grabado en alta definición (donde el código fuente ocupa una fracción reducida del editor en pantalla) a un ancho máximo de 512 píxeles reduce la altura de las tipografías a apenas unos pocos píxeles borrosos. Incluso manteniendo una calidad JPEG alta (80), a 512 px los caracteres de la palabra reservada `dict` y las llaves `{}` pierden su definición geométrica, impidiendo que tanto *Gemini Embedding 2* como *Gemini Flash* puedan leer la sintaxis.
2. **Cuantización JPEG agresiva (`jpeg_quality = 12` o `20`):** Cuando además de baja resolución se aplica un factor de calidad JPEG extremo (12 o 20 en la escala 1–100 de `cv2.imencode`), los macrobloques de la Transformada Discreta del Coseno (DCT) difuminan por completo los bordes de alto contraste del texto sobre el fondo del editor, convirtiendo el código en un artefacto ilegible.

La Figura 18 esquematiza el mecanismo exacto por el cual la degradación de resolución y compresión induce alucinaciones de descarte en el verificador visual, así como los rangos de calibración óptimos.

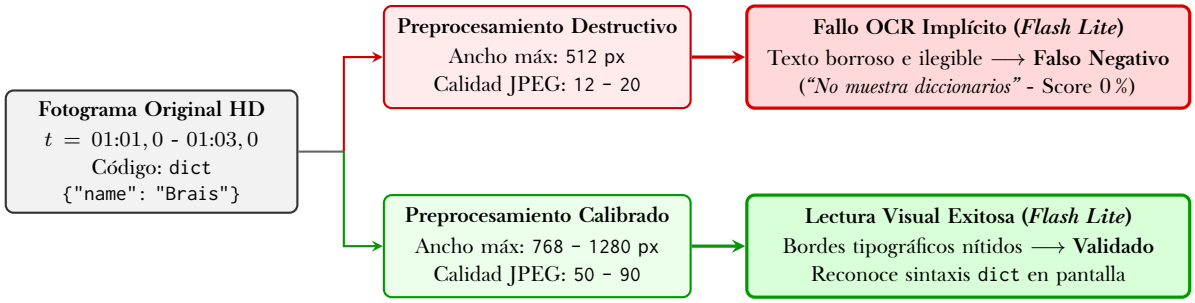


Figura 18: Diagnóstico del cuello de botella perceptivo en la etapa de verificación visual: el escalado a 512 px combinado con compresión JPEG agresiva (12-20) impide el OCR implícito de sintaxis de programación en pantalla, mientras que elevar `resize_width` al rango 768-1280 px y la calidad JPEG a 50-90 restaura la detección sin disparar el consumo de cuota.

2.5.4 Dilución Semántica en Fragmentos de Larga Duración y Muestreo Estático de Fotogramas

El segundo fenómeno crítico identificado durante la validación del sistema es el problema de la **dilución semántica y pérdida de fotogramas clave** en fragmentos de larga duración. En vídeos con un fondo visual altamente estático (tales como clases grabadas, presentaciones de diapositivas o tutoriales de código donde el entorno gráfico del editor permanece inalterado y únicamente cambian unas pocas líneas de texto), el índice de similitud estructural (SSIM) entre fotogramas consecutivos se mantiene elevado durante largos periodos.

Cuando el umbral de corte SSIM se configura en un valor demasiado permisivo (`threshold = 0.08`) o incluso en un valor moderado (`threshold = 0.60`) sin imponer una duración máxima de plano, el algoritmo agrupa secuencias extensas en un único `VideoChunk` (como el citado fragmento #5 de $39,1$ segundos de duración (01:01,1 – 01:40,2)). A una cadencia estándar de $f_{\text{fps}} = 30$ fotogramas por segundo, un fragmento de $\Delta T = 39,1$ s contiene un total de $N_{\text{total}} = \Delta T \times f_{\text{fps}} \approx 1{,}173$ fotogramas. Dado que la función `extract_key_frames` selecciona un número estático de $k = 3$ fotogramas por fragmento (inicial, final y máxima entropía), la fracción de cobertura temporal cae drásticamente, provocando dos efectos adversos:

- **Omisión por submuestreo (*Keyframe Miss*):** Si el evento buscado (la escritura de `dict {"name": "Brais"}`) ocurre entre los segundos 01:01,0 y 01:03,0, pero el fotograma de máxima entropía global de esos $39,1$ segundos recae en el segundo 01:25,0 (donde la pantalla está más llena de código de listas o variables), ninguno de los 3 fotogramas enviados a la API contendrá el diccionario.
- **Dilución del vector agregado en el espacio latente:** Al mezclar en un único `VideoChunk` casi 40 segundos que abarcan múltiples subtemas distintos (variables, listas y diccionarios), el vector resultante en *Gemini Embedding 2* promedia todas esas temáticas, perdiendo especificidad direccional frente a consultas puntuales.

La Figura 19 ilustra gráficamente cómo se produce esta pérdida por muestreo en un *chunk* gigante y cómo la corrigen las estrategias de acotación temporal o muestreo dinámico.

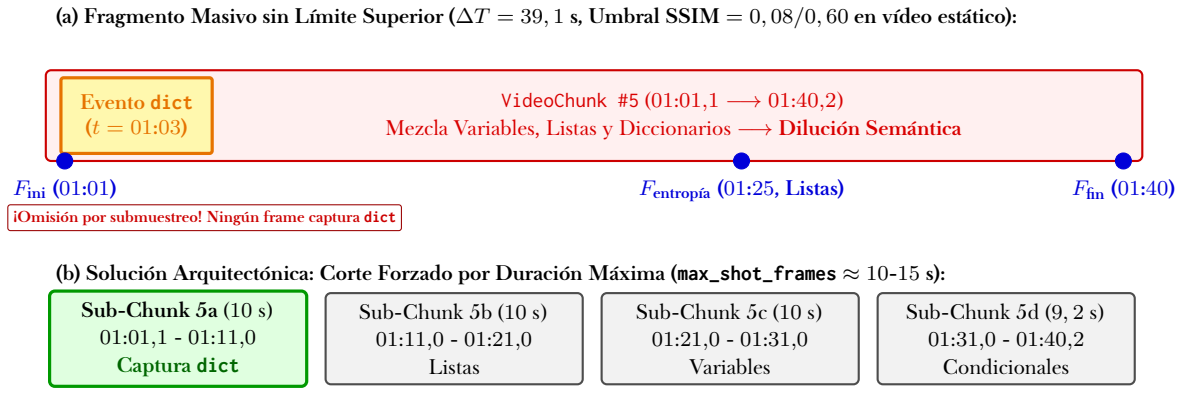


Figura 19: Mecanismo de dilución semántica y pérdida de fotogramas clave en vídeos con baja variación visual: (a) un fragmento de casi 40 s muestreado con solo 3 fotogramas omite el instante en que aparece el diccionario y mezcla múltiples temas; (b) la imposición de un límite superior (`max_shot_frames` de 10-15 s) o la extracción proporcional (un fotograma extra cada 5 s) garantiza la cohesión semántica.

El Cuadro 5 resume la comparativa técnica entre las configuraciones defectuosas analizadas durante la depuración y los rangos corregidos necesarios para preservar tanto la legibilidad del texto en pantalla como la granularidad temporal de los fragmentos.

Tabla 5: Diagnóstico empírico de parámetros de preprocesamiento visual y temporal en sistemas Video RAG con verificación secundaria (*Gemini Embedding 2* + *Gemini Flash*) [2].

Parámetro	Configuración Defectuosa	Efecto Patológico en el RAG	Configuración Calibrada
Ancho máx. (`resize_width`)	512 px	Texto/código ilegible (fallo OCR implícito)	768 px a 1,280 px (nitidez tipográfica)
Calidad JPEG (`jpeg_quality`)	12 a 20	Artefactos de bloque borran la sintaxis	50-70 (equilibrio) o 80-90 (alta)
Umbral SSIM (`threshold`)	0,08 (o 0,60 estático)	Chunks gigantes ($\sim$ 40 s); dilución vectorial	0,50-0,80 + cota superior temporal
Duración máx. (`max_shot_frames`)	Sin límite (∞)	Pérdida de evento al extraer solo 3 frames	Corte forzado cada 10-15 s máx.
Muestreo (`extract_key_frames`)	Fijo (3 frames / chunk)	Submuestreo severo en planos largos	3 frames o +1 frame cada 5 s

Estos resultados demuestran que la eficacia de un modelo fundacional omnimodal no depende únicamente de su capacidad neuronal en la nube, sino de que la canalización local de visión por computador preserve la resolución tipográfica suficiente y acote la duración temporal de cada fragmento antes de la vectorización.

2.6 Estrategias Avanzadas de Fragmentación (*Chunking*) y Alineación Temporal

2.6.1 Obsolescencia de la Partición Fija, Detección de Planos y Agrupación de Escenas

El rendimiento, la precisión de recuperación y la viabilidad técnica de cualquier arquitectura de vectorización multimodal recaen, fundamentalmente, sobre su estrategia de partición de datos o *chunking* [6, 26]. Indexar las pistas de audio, los subtítulos y las secuencias de fotogramas de vídeo de forma desincronizada o mediante divisiones temporales arbitrarias destruye por completo la cohesión semántica y cronológica del documento multimedia [9].

La metodología de fragmentación más elemental (consistente en cortar un archivo de vídeo en intervalos regulares y rígidos, por ejemplo, cada 5 o 10 segundos) constituye un enfoque obsoleto en el estado del arte de 2026 [5, 9]. Aplicar una rejilla temporal ciega produce invariablemente cortes abruptos en mitad de una oración hablada, interrumpe un cambio de plano cinematográfico o divide en dos fragmentos inconexos la transición de una diapositiva académica o la escritura de un bloque de código [5, 9]. Forzar posteriormente al modelo de *embeddings* o al modelo generativo a interpretar fragmentos visuales mutilados introduce un ruido contextual severo que degrada las métricas de relevancia y fidelidad [5].

Para erradicar esta fragmentación destructiva, las arquitecturas contemporáneas aplican algoritmos de **detección de límites de plano (*shot boundary detection*)** acoplados a procesamiento paralelo profundo [9]. Estos algoritmos evalúan fotograma a fotograma la continuidad visual midiendo bien la divergen-

cia de histogramas en espacios de color robustos ante variaciones de iluminación (como el espacio HSV (*Hue, Saturation, Value*), donde un parpadeo de cámara afecta al canal de valor pero no altera el matiz cromático), o bien la similitud estructural (SSIM) entre fotogramas secuenciales, garantizando que cada *chunk* represente una unidad narrativa o visualmente coherente [5, 9].

Por otra parte, cuando el vídeo presenta cortes de cámara muy rápidos y la granularidad del plano resultante es excesivamente fina, la literatura incorpora técnicas de **agrupación de escenas (*scene grouping*)** [9]. Este procedimiento fusiona planos consecutivos cuyas representaciones vectoriales preliminares superan un umbral de similitud predefinido, condensando múltiples micro-planos en intervalos temporales más largos y semánticamente estables antes de su indexación definitiva [9].

La Figura 20 contrasta el efecto destructivo de la partición por intervalos fijos frente a la preservación de unidades narrativas mediante detección adaptativa de límites de plano y agrupación de escenas.

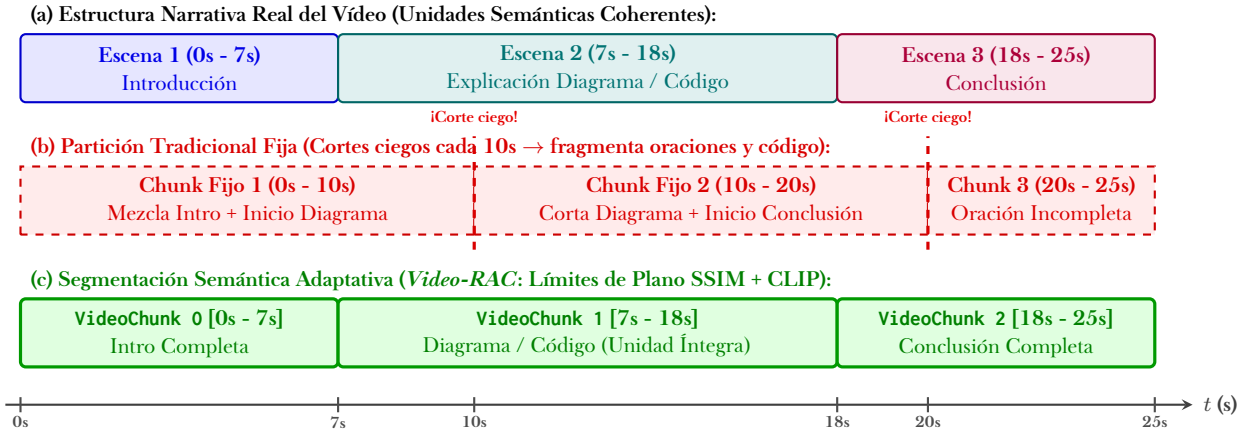


Figura 20: Comparativa estructural entre (b) la división arbitraria en intervalos fijos, que introduce ruido contextual al cortar escenas y oraciones a la mitad, y (c) la segmentación semántica adaptativa (*Video-RAC*), que alinea exactamente los límites de cada `VideoChunk` con las transiciones reales del contenido [5, 9].

2.6.2 Metodología Híbrida *Video-RAC* y Formulación Matemática del SSIM Gaussiano

Dentro del ámbito de los vídeos educativos, académicos y tutoriales técnicos, la arquitectura **Video-RAC (*Retrieval Adaptive Chunking*)** se ha consolidado como el estándar metodológico de referencia [5]. *Video-RAC* abandona las particiones estáticas en favor de un criterio híbrido que evalúa cada transición temporal combinando la similitud del coseno de *embeddings* visuales ligeros generados por CLIP con el Índice de Similitud Estructural (SSIM) [5].

Específicamente, la formulación teórica de *Video-RAC* emplea el modelo `openai/clip-vit-base-patch32` para extraer representaciones rápidas de los fotogramas consecutivos F_t y F_{t+1}, ponderando su similitud semántica con la similitud estructural según la Ecuación (13) mediante un factor de balance $\alpha = 0,6$ [5]:

$$S_{\text{híbrida}}(F_t, F_{t+1}) = \alpha \cdot \cos(\mathbf{e}_{\text{CLIP}}(F_t),\, \mathbf{e}_{\text{CLIP}}(F_{t+1})) + (1 - \alpha) \cdot \text{SSIM}(F_t, F_{t+1}) \tag{13}$$

Los hiperparámetros canónicos reportados en la literatura para determinar dónde concluye una unidad semántica y comienza la siguiente estipulan un factor de balance $\alpha = 0,6$, un umbral de similitud de *embeddings* CLIP de $\tau_{\text{CLIP}} = 0,85$ y un umbral estructural de $\tau_{\text{SSIM}} = 0,80$ [5].

En implementaciones de producción optimizadas para ejecutarse localmente en CPU sin requerir la carga de pesos adicionales de CLIP durante el preprocesamiento (tal y como se implementa en el módulo `src/processing/video_chunker.py`), la detección de límites de plano se resuelve directamente mediante la formulación matemática exacta del **Índice de Similitud Estructural (SSIM)** con ventana Gaussiana en OpenCV. Dados dos fotogramas consecutivos convertidos a escala de grises y redimensionados a un ancho de análisis rápido (`resize_width = 160` píxeles para la fase de corte), representados como matrices en coma flotante de doble precisión $a, b \in \mathbb{R}^{H \times W}$ (`np.float64`), el algoritmo aplica un filtro de desenfoque Gaussiano bidimensional `cv2.GaussianBlur` con tamaño de núcleo impar $K \times K$ (`window_size = 7`) y desviación estándar $\sigma = 1,5$ para obtener las medias locales μ_a, μ_b definidas en la Ecuación (14):

$$\mu_a = G_{K\times K,\,\sigma=1,5} * a, \qquad \mu_b = G_{K\times K,\,\sigma=1,5} * b \tag{14}$$

A partir de los cuadrados de las medias (μ_a^2, μ_b^2) y de su producto cruzado ($\mu_{ab} = \mu_a\mu_b$), las varianzas locales σ_a^2, σ_b^2 y la covarianza cruzada local σ_{ab} se calculan matricialmente según la Ecuación (15) mediante convolución Gaussiana:

$$\sigma_a^2 = (G_{K\times K,\,\sigma=1,5} * (a \odot a)) - \mu_a^2, \quad \sigma_b^2 = (G_{K\times K,\,\sigma=1,5} * (b \odot b)) - \mu_b^2, \quad \sigma_{ab} = (G_{K\times K,\,\sigma=1,5} * (a \odot b)) - \mu_{ab} \tag{15}$$

donde $\odot$ denota el producto elemento a elemento (*Hadamard product*). Con estas matrices estadísticas e introduciendo las constantes de estabilidad numérica para un rango dinámico de 8 bits ($L = 255$), definidas como $C_1 = (0,01 \times 255)^2 = 6,5025$ y $C_2 = (0,03 \times 255)^2 = 58,5225$, el mapa espacial SSIM y su escalar medio resultante en el rango $[-1, 1]$ se formulan a través de la Ecuación (16):

$$\text{SSIM}_{\text{map}}(a,b) = \frac{(2\mu_{ab} + C_1)(2\sigma_{ab} + C_2)}{(\mu_a^2 + \mu_b^2 + C_1)(\sigma_a^2 + \sigma_b^2 + C_2)}, \qquad \overline{\text{SSIM}}(a,b) = \frac{1}{H \cdot W} \sum_{y=1}^{H} \sum_{x=1}^{W} \text{SSIM}_{\text{map}}(y,x) \tag{16}$$

Cuando el valor escalar $\overline{\text{SSIM}}(a,b)$ cae por debajo del umbral configurado (`threshold`, calibrado entre $0,50$ y $0,80$) y el número de fotogramas acumulados desde el corte anterior iguala o supera el mínimo permitido (`min_shot_frames = 12` a 45 fotogramas para filtrar destellos luminosos espurios), la función `detect_shot_boundaries` consolida e instancia un objeto inmutable `VideoChunk(index, start_frame, end_frame, start_time, end_time)`.

La Figura 21 detalla este flujo de cálculo matricial implementado en `compute_ssim` y `detect_shot_boundaries`.

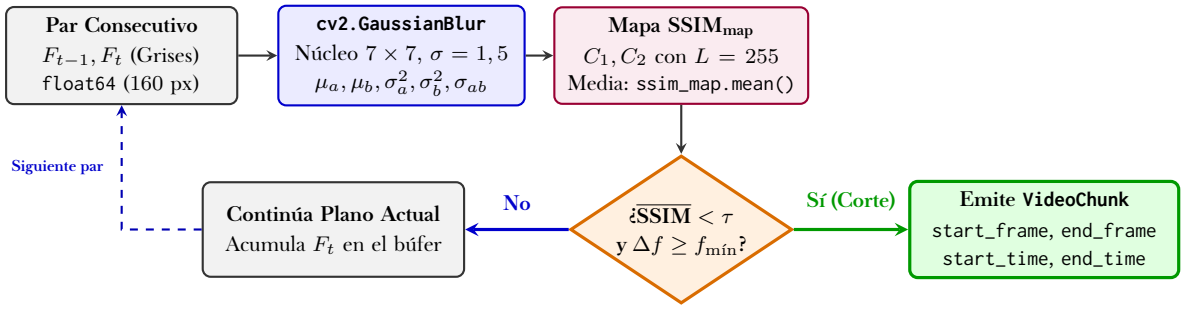


Figura 21: Flujo algorítmico del cálculo del Índice de Similitud Estructural (SSIM) mediante filtrado Gaussiano en OpenCV (`compute_ssim`) y criterio de consolidación de objetos `VideoChunk` en `detect_shot_boundaries`.

2.6.3 Selección de Fotogramas Clave mediante Entropía Matemática de Shannon

Una vez que el vídeo ha sido particionado en unidades semánticamente coherentes (`VideoChunk`), enviar la totalidad de los fotogramas de cada fragmento (a razón de 30 o 60 fotogramas por segundo) hacia la API del modelo de *embeddings* resultaría computacional y económicamente inviable [2, 5]. Para comprimir el volumen visual sin perder el contenido informativo del plano, la metodología *Video-RAC* establece una estrategia de **selección de fotogramas por entropía** implementada en la función `extract_key_frames` [5].

Para cada objeto `VideoChunk` delimitado entre los índices de fotograma $[f_{\text{ini}}, f_{\text{fin}}]$, el algoritmo selecciona exactamente tres fotogramas representativos ordenados cronológicamente [5]:

1. **Fotograma Inicial ($F_{f_{\text{ini}}}$):** Captura el estado visual de partida del evento al abrirse el plano.
2. **Fotograma Intermedio de Máxima Entropía (F_{f^*}):** De entre los fotogramas interiores $t \in (f_{\text{ini}}, f_{\text{fin}})$ (evaluados de forma exhaustiva o mediante un paso de muestreo `entropy_step`), se selecciona aquel que maximiza la **Entropía de Información de Shannon** $H(F_t)$, la cual cuantifica la riqueza y densidad de detalle visual (por ejemplo, el instante en que un diagrama o bloque de código está completamente desplegado en pantalla y no oculto o en transición).
3. **Fotograma Final ($F_{f_{\text{fin}}}$):** Registra el estado consolidado del plano justo antes del corte hacia la siguiente escena.

Matemáticamente, para calcular la entropía de Shannon de un fotograma candidato F_t convertido a escala de grises con $L = 256$ niveles de intensidad ($i \in \{0, 1, \ldots, 255\}$), se obtiene primero su histograma de frecuencias absolutas $h(i)$ mediante `cv2.calcHist` y se normaliza por el número total de píxeles $N_{\text{pix}} = H \times W$ para obtener la distribución de probabilidad discreta $p_i = h(i)/N_{\text{pix}}$, evaluando la Ecuación (17):

$$H(F_t) = -\sum_{\substack{i=0 \\ p_i>0}}^{255} p_i \log_2(p_i), \qquad f^* = \arg \max_{t \in (f_{\text{ini}},\, f_{\text{fin}})} H(F_t) \tag{17}$$

La Figura 22 ilustra la evolución de la curva de entropía $H(F_t)$ dentro de los límites de un `VideoChunk` y la extracción de la terna de matrices NumPy que alimenta al vectorizador multimodal.

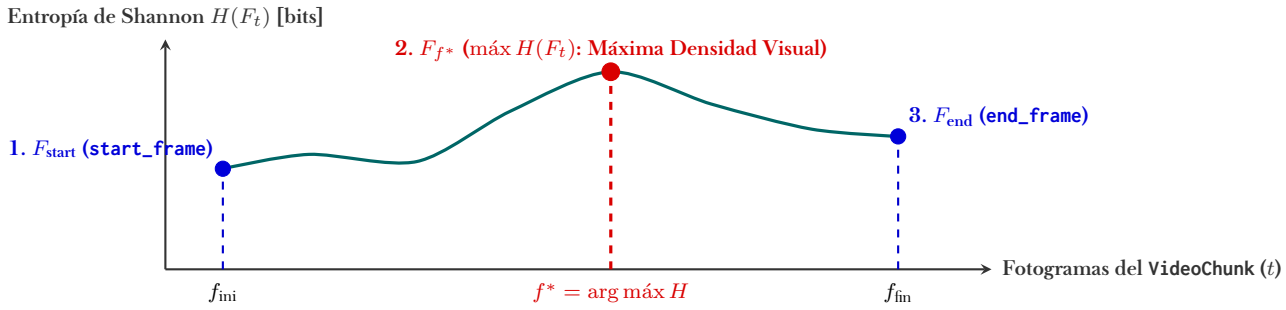


Figura 22: Selección de fotogramas clave basada en Entropía de Shannon (`extract_key_frames`) según la metodología *Video-RAC*: extracción del fotograma inicial, el fotograma final y el fotograma intermedio que maximiza la densidad de información visual $H(F_t)$ [5].

2.6.4 Alineación Forzada del Audio (*Forced Alignment*), Silero VAD y Diarización de Hablantes

En paralelo a la segmentación visual, la dimensión auditiva exige una ingeniería de fragmentación igualmente rigurosa [9, 10]. El estado del arte descarta por completo las transcripciones planas carentes de marcas de tiempo y establece una canalización acústica en cuatro etapas implementada en torno al módulo `src/processing/audio_transcriber.py` [9, 10]:

1. **Extracción de Pista Acústica:** A partir del contenedor de vídeo original (`.mp4`), se desmultiplexa la señal de audio hacia un archivo temporal sin compresión o de alta fidelidad (`.wav` o `.mp3`) empleando herramientas de bajo nivel como `ffmpeg` o `moviepy` ejecutadas de forma no bloqueante (`asyncio.to_thread`).

2. **Detección de Actividad de Voz (*Voice Activity Detection* - Silero VAD):** Antes de invocar al modelo de transcripción, el flujo de audio se filtra mediante redes ultraligeras de detección de voz como **Silero VAD**, la cual cuenta con aproximadamente **un millón de parámetros** y ofrece una ejecución ultrarrápida en CPU para aislar los segmentos de discurso humano respecto del silencio y del ruido ambiental, evitando alucinaciones fonéticas del modelo ASR durante pausas acústicas [9, 10].

3. **Transcripción Fonética, Alineación Forzada (*WhisperX*) y Diarización:** Los segmentos con voz activa se procesan mediante motores de reconocimiento avanzado como **Whisper**, acoplados a algoritmos de **alineación forzada** como **WhisperX** o **Montreal Forced Aligner** [9, 10]. Este acoplamiento genera transcripciones con precisión a nivel de palabra y fonema, vinculando cada término exacto con su marca de tiempo en milisegundos $[t^{\text{aud}}_{\text{ini}}, t^{\text{aud}}_{\text{fin}}]$, mientras que la etapa de **diarización de hablantes** etiqueta acústicamente cada bloque textual con la identidad del interlocutor [9].

4. **Superposición y Acoplamiento con `VideoChunk` (`align_transcript_with_chunks`):** Finalmente, los segmentos transcritos se proyectan sobre los intervalos temporales $[t^{(i)}_{\text{start}}, t^{(i)}_{\text{end}}]$ de los objetos `VideoChunk` generados previamente por la etapa visual (CLIP/SSIM) [9].

Formalmente, dado un `VideoChunk` visual C_i con intervalo $[t^{(i)}_{\text{start}}, t^{(i)}_{\text{end}}]$ y una lista de segmentos de transcripción auditiva $S_j = (\tau^{(j)}_{\text{ini}}, \tau^{(j)}_{\text{fin}}, \text{texto}_j)$, la condición matemática de intersección temporal $\Delta\tau_{i,j}$ que determina la asignación o concatenación del texto en el atributo `VideoChunk.transcript` se define

mediante la Ecuación (18):

$$\Delta\tau_{i,j} = \max\Big(0,\ \min\Big(t_{\text{end}}^{(i)},\ \tau_{\text{fin}}^{(j)}\Big) - \max\Big(t_{\text{start}}^{(i)},\ \tau_{\text{ini}}^{(j)}\Big)\Big) > 0 \tag{18}$$

La Figura 23 muestra la arquitectura completa de esta canalización paralela visual y auditiva, garantizando que al recuperar un fragmento el sistema disponga simultáneamente de los fotogramas clave, la transcripción exacta pronunciada en ese segundo y la identidad del hablante [9].

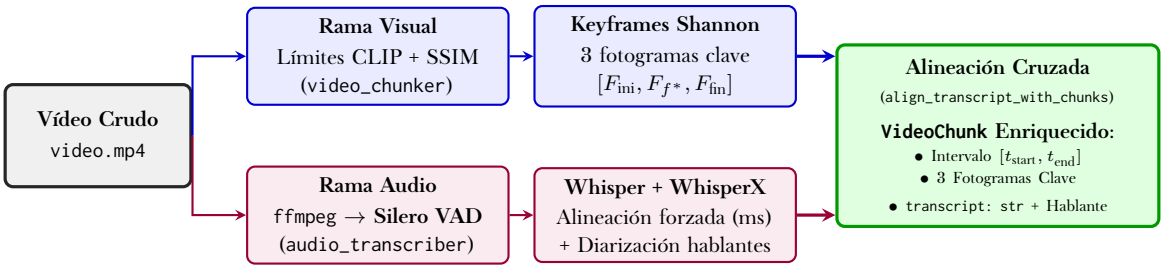


Figura 23: Arquitectura de procesamiento paralelo y alineación forzada intermodal: la rama visual segmenta el vídeo mediante CLIP/SSIM y extrae tres fotogramas por entropía de Shannon, mientras la rama auditiva filtra silencios con Silero VAD y alinea las marcas de tiempo de WhisperX sobre cada `VideoChunk` [5, 9, 10].

El Cuadro 6 resume todos los parámetros matemáticos y algorítmicos que gobiernan las etapas de fragmentación visual y alineación auditiva descritas en esta sección.

Tabla 6: Especificaciones técnicas de la estrategia de fragmentación semántica adaptativa (*Video-RAC*) y alineación forzada de audio [5, 9, 10].

Parámetro	Modelo / Algoritmo	Valor Referencia	Función en el Pipeline
Codificador visual híbrido	`clip-vit-base-patch32`	$\alpha = 0,6$, $\tau = 0,85$	Similitud semántica entre planos (*Video-RAC*)
Índice estructural local	SSIM Gaussiano (OpenCV)	$\tau_{\text{SSIM}} = 0,80$	Detección bordes de plano ($K = 7$, $\sigma = 1,5$)
Constantes de estabilidad	Rango dinámico $L = 255$	$C_1 = 6,50$, $C_2 = 58,52$	Evita división por cero en zonas oscuras
Filtro de planos espurios	`min_shot_frames`	12 a 45 frames	Descarta parpadeos menores a 0,5-1,5 s
Muestreo de fotogramas	Entropía Shannon $H(F_t)$	3 fotogramas	Máxima densidad visual por `VideoChunk`
Detección de voz activa	Silero VAD (CPU)	$\sim 10^6$ parámetros	Aísla discurso humano de silencio y ruido
Alineación y diarización	Whisper + WhisperX / MFA	Precisión milisegundos	Anclaje palabra a palabra e identidad hablante

2.6.5 Análisis Comparativo frente a Sistemas de Ingestión de Vídeo en Ventana Fija (*SentrySearch*)

En el ecosistema actual de herramientas de código abierto para la búsqueda semántica sobre vídeo mediante *Gemini Embedding 2* y *Qwen3-VL-Embedding* [2, 4], proyectos recientes como ***SentrySearch*** [11] representan el enfoque de partición temporal rígida con ingestión directa de flujo de vídeo.

Dicha arquitectura divide los archivos multimedia en fragmentos de duración fija ($\Delta T = 30$ s) con un solapamiento estático ($\Delta t_{\text{overlap}} = 5$ s), reduciendo la resolución a 480p y 5 FPS antes de transmitir el clip de vídeo a la API en la nube o al modelo local [11].

El contraste arquitectónico y matemático entre este enfoque de ventana fija y la canalización adaptativa implementada en el presente Trabajo de Fin de Máster (**SSIM + Entropía de Shannon + Transcripción Alineada**) permite cuantificar cuatro ventajas estructurales de nuestra propuesta [5, 11]:

1. **Reducción superior al** $90\,\%$ **en facturación de fotogramas y consumo de cuota (*Token Economy*):** La API de *Gemini Embedding 2* extrae, tokeniza y factura nativamente 1 fotograma por cada segundo de vídeo subido (1 FPS), con independencia de la tasa de fotogramas del archivo contenedor [11]. Al dividir 1 hora de vídeo (3,600 s) en fragmentos de 30 s con 5 s de solapamiento, un sistema de ventana fija genera 144 fragmentos y transmite un total de 4,315 fotogramas facturados por hora (dado que los segundos solapados se pagan por duplicado en cada fragmento adyacente) [11]. A una tarifa de referencia de $\$0,00079$ por fotograma de vídeo procesado (equivalente a $\$12,00$ por millón de tokens de vídeo frente a $\$0,45$ por millón de tokens de imagen estática), la ingestión bruta asciende a $\approx \$3,41$ por hora de metraje [2, 11]. Por el contrario, nuestra arquitectura fundamentada en *Video-RAC* [5] evita subir el flujo continuo de vídeo y empaqueta únicamente $k = 3$ fotogramas clave por fragmento (inicial, final y máxima entropía de Shannon) junto con la transcripción fonética dentro de una única estructura `types.Content`. Para una media de 144 fragmentos adaptativos por hora, el sistema procesa únicamente $144 \times 3 = 432$ imágenes en lugar de 4,315 fotogramas de vídeo, logrando una reducción del $\mathbf{89,98}\,\%$ en el volumen visual procesado y manteniendo el consumo dentro de los límites estrictos de la capa gratuita (30,000 TPM y 1,000 RPD).

2. **Superación de las limitaciones de borde temporal y detección heurística de estaticidad:** La propia documentación técnica de *SentrySearch* reconoce como limitaciones abiertas que: (a) la calidad de recuperación se degrada cuando un evento queda dividido entre dos fragmentos fijos, proponiendo como "trabajo futuro" la incorporación de detección inteligente de escenas; y (b) su descarte de fragmentos estáticos depende de una heurística simple que compara el tamaño en bytes de archivos JPEG muestreados, la cual omite movimientos sutiles [11]. Nuestro *pipeline* resuelve nativamente ambos problemas mediante la detección de límites de plano basada en el Índice de Similitud Estructural con filtro Gaussiano 7×7 ($\sigma = 1,5$) sobre OpenCV (`detect_shot_boundaries`) y el cálculo exacto de la densidad de información mediante la Entropía de Shannon sobre el histograma de intensidades (`cv2.calcHist`), acotado por `max_shot_frames` para impedir la dilución semántica en planos estáticos de programación [5].

3. **Explicación teórica y empírica de la distribución de similitud del coseno en consultas específicas:** Los estudios sobre *Gemini Embedding 2* y *Qwen3-VL-Embedding* prueban que las consultas de alta especificidad visual o sintáctica producen puntuaciones de similitud del coseno inferiores al umbral genérico de 0,41 (rango verificado 0,28–0,39) [11]. Esto fundamenta lo observado en la validación sobre `python_tutorial.mp4` para la consulta `"Bucles while"`: una portada gráfica con el título *"Bucles"* alcanza similitud $\mathbf{0,4315}$ (`Chunk 36`, Rango #1), mientras que el fragmento con código real (`while index <5:` en $t = 231,0$ s, `Chunk 38`) obtiene $\mathbf{0,3893}$ (Rango #7).

4. **Necesidad de deduplicación inter-chunk (*Deduplication*) y sobremuestreo antes del *Reranking*:** En secuencias de baja variación visual, fragmentos contiguos acaparan las primeras posiciones vectoriales (como los `Chunks 36, 35 y 33` con $0,4315$, $0,4219$ y $0,4211$), desplazando al fragmento objetivo fuera de un corte `top_k = 5`. Combinar un sobremuestreo (`fetch_k` $>$ top_k) con un filtro de techo de similitud ($\tau_{\text{dedupe}} \approx 0,90$–$0,95$) garantiza diversidad topológica hacia la segunda etapa (*Gemini 3.5 Flash Lite*), preservando degradación elegante (*fallback*) hacia Qdrant ante saturaciones transitorias de red [11].

La Figura 24 ilustra la diferencia estructural y de eficiencia de tokens entre un pipeline de ventana fija a 1 FPS y la arquitectura adaptativa multimodal desarrollada en este trabajo.

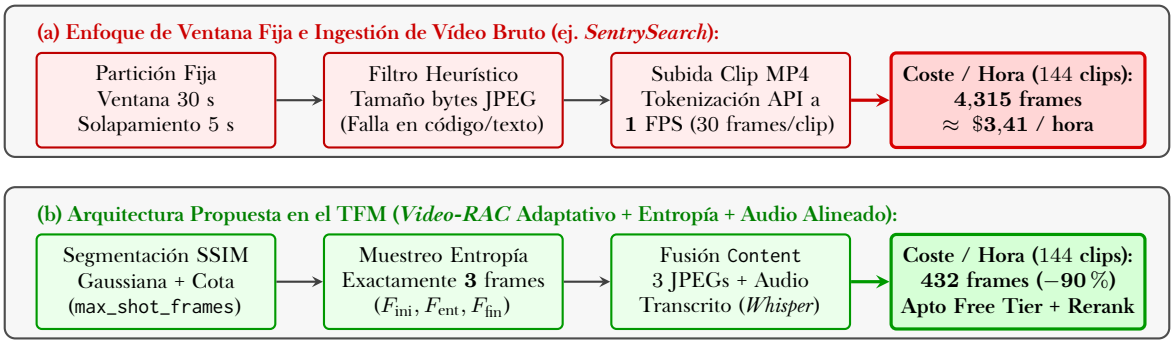


Figura 24: Comparativa arquitectónica y de eficiencia computacional entre la ingestión de vídeo bruto en ventanas fijas a 1 FPS (*SentrySearch*) y la canalización adaptativa multimodal por entropía de Shannon propuesta en el TFM [2, 5, 11].

El Cuadro 7 sintetiza las diferencias técnicas verificables entre ambas implementaciones sobre el mismo ecosistema de modelos fundacionales.

Tabla 7: Comparativa técnica y de costes operativos entre *SentrySearch* y la arquitectura desarrollada en el TFM [2, 5, 11].

Dimensión Técnica	SentrySearch (Ventana Fija)	Arquitectura TFM (Propuesta)
Partición temporal (*Chunking*)	Fija (30 s con 5 s de solapamiento)	Adaptativa por plano (SSIM Gaussiano + cota máx.)
Detección de cambios / información	Heurística por tamaño de archivo JPEG	Matemática (Entropía de Shannon sobre histograma)
Representación visual enviada a API	Vídeo completo tokenizado a 1 FPS (30 frames)	3 fotogramas clave (F_{ini}, $F_{\text{entropía}}$, F_{fin})
Integración del canal auditivo	Sin alineación explícita de transcripción	Transcripción alineada fusionada en `Content` y *Payload*
Fotogramas facturados (1 h vídeo)	4,315 frames/h (≈ $3,41/h en Gemini API)	$\sim$ 432 frames/h ($-\mathbf{89,98}$ % carga visual)
Motor de almacenamiento vectorial	ChromaDB local (colecciones separadas)	Qdrant (`PointStruct`, HNSW, UUIDv5 idempotente)
Refinamiento de Segunda Etapa	VLM sobre clip recortado + `-dedupe 0.9`	*Gemini 3.5 Flash Lite* Bimodal (2 frames + texto) + Fallback

2.7 Bases de Datos Vectoriales, Evaluación Empírica y Metodología de Revisión

2.7.1 Arquitectura de Bases de Datos Vectoriales e Indexación HNSW

Una vez que los fragmentos de vídeo y las consultas en lenguaje natural han sido transformados en representaciones vectoriales densas por los modelos fundacionales, el sistema requiere una infraestructura de almacenamiento persistente capaz de indexar y recuperar alta dimensionalidad en tiempo real [2, 4]. En el estado del arte de 2026, motores especializados como **Qdrant**, **Milvus**, **Weaviate** y **Pinecone** se han consolidado como el estándar industrial para la gestión de repositorios vectoriales [2].

En despliegues basados en motores como Qdrant, cada fragmento vectorizado se encapsula y almacena como una estructura de punto indexado (`PointStruct`) compuesta por tres elementos fundamentales [2]:

1. **Identificador Único Determinista (UUIDv5):** Generado de forma unívoca a partir del nombre del archivo de vídeo y del índice secuencial del fragmento, lo que garantiza que reinsertar el mismo `VideoChunk` sobrescriba de manera idempotente su entrada anterior sin duplicar registros en la base de datos [2].
2. **Vector Denso ($\mathbf{v} \in \mathbb{R}^d$):** El tensor numérico de salida devuelto por el modelo (por ejemplo, configurado a 3,072 dimensiones en *Gemini Embedding 2* o 768 dimensiones con normalización L_2), indexado mediante métrica de distancia coseno (`models.Distance.COSINE`) sobre grafos de mundos pequeños jerárquicos (*Hierarchical Navigable Small World*, **HNSW**) [2].
3. **Carga Útil de Metadatos (*Payload*):** Un diccionario estructurado adjunto al vector que almacena los atributos relacionales indispensables para la reconstrucción temporal del resultado, incluyendo el nombre del fichero (`video_name`), el índice del fragmento (`chunk_index`) y los instantes exactos de inicio y fin expresados en segundos flotantes (`start_time` y `end_time`) [2].

La Figura 25 ilustra la arquitectura de almacenamiento y el direccionamiento mediante índices HNSW en Qdrant.

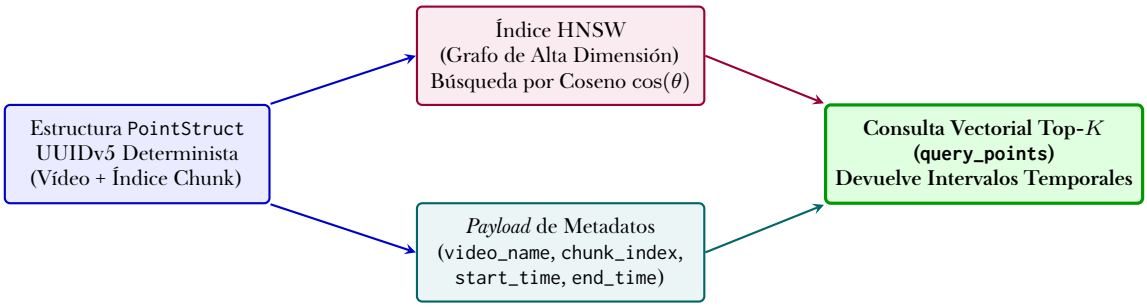


Figura 25: Arquitectura de almacenamiento y recuperación en Qdrant: separación de vectores densos indexados mediante grafos HNSW y metadatos temporales alojados en el *payload* para consultas top-K ultra rápidas [2].

2.7.2 Métricas Clásicas de Recuperación y Bancos de Pruebas Masivos (MMEB-V2 y MVEB)

La validación empírica del rendimiento de los sistemas de recuperación de información multimedia se apoya en una combinación de métricas estadísticas clásicas y bancos de pruebas (*benchmarks*) estandarizados a escala masiva [4, 12].

En primer lugar, las métricas clásicas de Recuperación de Información evalúan la precisión del ordenamiento top-K devuelto por la base de datos vectorial frente a la verdad terreno (*ground truth*) [4]:

- ***Recall@K*:** Mide la proporción de fragmentos relevantes reales que han sido recuperados dentro de los primeros K resultados devueltos [4].
- ***Precision@K*:** Cuantifica qué porcentaje de los K resultados devueltos por el sistema son efectivamente relevantes [4].
- ***Hit@1* (o *Accuracy@1*):** Evalúa si el primer resultado devuelto por el motor de búsqueda coincide exactamente con el evento buscado [4].
- ***nDCG@K* (*Normalized Discounted Cumulative Gain*):** Evalúa la calidad del ranking ponderando la posición de los aciertos, penalizando severamente cuando un resultado relevante aparece en posiciones inferiores de la lista [4].

A nivel global, la comunidad investigadora evalúa los modelos sobre dos bancos de pruebas masivos de referencia actualizados para 2026 [4]:

1. **MMEB-V2 (*Massive Multimodal Embedding Benchmark V2*, 2025):** Consolida más de 78 conjuntos de datos para evaluar la recuperación conjunta sobre texto, imágenes, vídeo y documentos estructurados (PDF), estandarizando mediciones de *Recall@1* y *nDCG@5* [4].
2. **MVEB (*Massive Video Embedding Benchmark*, 2026):** El estándar específico más riguroso para la comprensión temporal de vídeo de larga duración, abarcando más de 180 tareas de recuperación cruzada entre pistas visuales y auditivas [4].

2.7.3 Frameworks de Evaluación RAG y Atribución Verificable (RAGAS, ALCE, ARGUE, MIRAGE)

La evaluación de arquitecturas RAG multimodales exige ir más allá de la simple recuperación de vectores, analizando la calidad de la generación de respuestas y la veracidad de la atribución temporal [5, 12]. El marco de referencia contemporáneo se estructura en torno a dos metodologías complementarias [5, 12]:

2.7.3.1 Framework RAGAS (*Retrieval Augmented Generation Assessment*):

Descompone la calidad de la respuesta generada por el modelo de lenguaje en tres dimensiones fundamentales independientes [5]:

- ***Answer Relevance*:** Mide el grado de pertinencia y coherencia de la respuesta generada respecto a la consulta original del usuario [5].

- ***Context Relevance*:** Evalúa la precisión del material recuperado por la base de datos vectorial, penalizando la inclusión de fragmentos de vídeo irrelevantes que aporten ruido al contexto [5].
- ***Faithfulness* (Fidelidad):** Cuantifica si la respuesta generada se deriva estrictamente del contexto recuperado, detectando y penalizando cualquier alucinación o invención por parte del modelo [5].

2.7.3.2 Métricas de Atribución Verificable (ALCE, ARGUE, MIRAGE):

Marcos como **ALCE** (*Automatic LLM Citation Evaluation*), **ARGUE** y **MIRAGE** extienden la evaluación para auditar si cada afirmación generada en la respuesta cuenta con un enlace de citación directo y verificable hacia el intervalo temporal exacto ($[t_{\text{inicio}}, t_{\text{fin}}]$) del vídeo fuente, garantizando una trazabilidad forense inalterable [12].

La Figura 26 resume la estructura jerárquica de evaluación mediante RAGAS y métricas de atribución verificable.

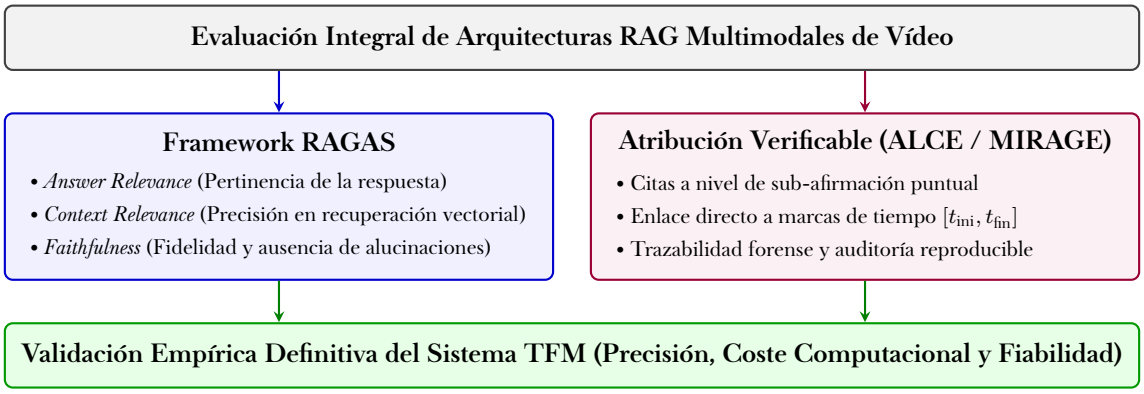


Figura 26: Estructura combinada de evaluación empírica mediante el framework RAGAS y métricas de atribución verificable (ALCE/MIRAGE) para validar el rendimiento del sistema RAG de vídeo [5, 12].

2.7.4 Metodología de Revisión Sistemática de la Literatura Asistida por NotebookLM

Para llevar a cabo la recopilación, filtrado y síntesis crítica de las fuentes bibliográficas empleadas a lo largo de este estado del arte, la investigación ha adoptado un marco metodológico estructurado en un **flujo de trabajo de cuatro etapas asistido por entornos RAG acotados y gestores bibliográficos** (tales como Zotero sincronizado con repositorios abiertos en arXiv) [13, 14]:

1. **Etapa 1 - Construcción y Depuración de la Base de Conocimiento Acotada:** Recopilación sistemática de artículos preprints y documentación técnica cumpliendo restricciones operativas (tales como los límites de ingestión de 50 fuentes por cuaderno y 500.000 palabras por documento en herramientas como Google NotebookLM), resolviéndolo mediante la fusión temática de artículos o el uso de múltiples cuadernos especializados [13].
2. **Etapa 2 - Síntesis Estructural y Descubrimiento Topológico:** Análisis automatizado de convergencia tecnológica mediante resúmenes fonéticos integrados (*Audio Overview*) y generación de cronologías de evolución arquitectónica [13, 14].

3. **Etapa 3 - Análisis Crítico, Deconstrucción Metodológica y Lazo de Relleno de Brechas (*Gap-Fill Loop*):** Contraste sistemático de hipótesis metodológicas y detección de vacíos en la literatura previa sobre fragmentación adaptativa y modelos multimodales [13].
4. **Etapa 4 - Verificación Transparente y Consolidación Causal:** Auditoría inalienable de las citas generadas contrastándolas directamente sobre los textos fuente originales para garantizar la ausencia de alucinaciones teóricas en la memoria del TFM [13, 14].

La Figura 27 esquematiza este flujo metodológico de investigación sistemática.

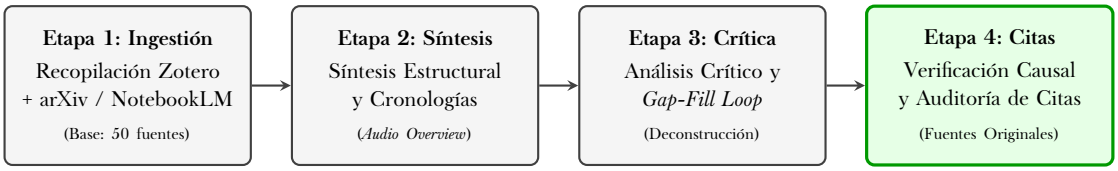


Figura 27: Flujo metodológico de cuatro etapas para la revisión sistemática de la literatura apoyado en gestores bibliográficos y entornos RAG acotados [13, 14].

El Cuadro 8 consolida las métricas, frameworks y metodologías de evaluación y revisión analizadas en este último bloque del estado del arte.

Tabla 8: Síntesis de métricas, bancos de pruebas y metodologías de evaluación en sistemas Video RAG [4, 5, 12, 13].

Categoría	Framework	Métricas	Objetivo en el TFM
Recuperación Vectorial	Qdrant + HNSW	*Recall@K*, *Hit@1*, *nDCG@5*	Eficacia en la localización top-K
Benchmarks Masivos	MMEB-V2 / MVEB	Recuperación multicanal	Comparativa con estándares 2026
Calidad RAG y Fidelidad	Framework RAGAS	*Answer/Context Rel.*, *Faithfulness*	Evaluación de alucinaciones y contexto
Atribución Verificable	ALCE / MIRAGE	Citas a nivel de afirmación	Trazabilidad temporal en milisegundos
Revisión Bibliográfica	Zotero + NotebookLM	Flujo 4 etapas + *Gap-Fill*	Rigor metodológico y estado del arte

2.8 Evaluación Empírica de la Integración Audio-Visual y Verificación Bimodal

Con el propósito de cuantificar empíricamente el impacto de incorporar la dimensión auditiva sobre la arquitectura base puramente visual, se ejecutó un estudio de ablación de extremo a extremo (*End-to-End*, E2E) comparando la línea base visual frente a la canalización multimodal completa (`ffmpeg` + `openai-whisper base` + fusión temprana en `gemini-embedding-2-preview` + verificación bimodal en `gemini-3.5-flash-lite`) [2, 5]. En estricto cumplimiento de las restricciones de cuota de la capa gratuita (100 RPM, 30,000 TPM y 1,000 RPD), la inyección de la transcripción alineada como un objeto `types.Part.from_text` adicional dentro del mismo contenedor `types.Content` de los tres fotogramas clave mantuvo invariante el consumo en exactamente una petición de vectorización por fragmento.

2.8.1 Análisis Cuantitativo sobre Corpus Contrastados: Locución Sincronizada frente a Metraje Desacoplado

La batería de evaluación automatizada se ejecutó sobre dos dominios audiovisuales con naturalezas semánticas opuestas:

1. **Dominio Técnico con Locución Sincronizada (`python_tutorial.mp4`):** Compuesto por 82 fragmentos adaptativos en la suite E2E (81/82 enriquecidos con voz activa, 98, 8 %), evaluado mediante 56 consultas literales, 10 consultas multi-objetivo y 8 consultas negativas de control. En este escenario, el discurso del ponente describe directamente las estructuras sintácticas que se codifican en el editor.

2. **Dominio Documental con Metraje de Archivo Desacoplado (`animals.mp4`):** Compuesto por 82 fragmentos (82/82 con locución continua, 100, 0 %), evaluado sobre 46 consultas literales, 10 multi-objetivo y 8 negativas. Este vídeo presenta el fenómeno clásico de *B-roll desacoplado*: la narración expone conceptos biológicos generales mientras en pantalla se suceden secuencias de especies animales que no son nombradas verbalmente en ese instante.

El Cuadro 9 sintetiza las métricas obtenidas por la versión puramente visual frente a la nueva arquitectura bimodal en ambos corpus.

Tabla 9: Estudio de ablación E2E: comparativa de rendimiento entre la arquitectura puramente visual y la arquitectura con fusión temprana de audio (*Whisper*) y juez bimodal [2, 5].

Corpus de Vídeo	Etapa / Batería	Métrica	Solo Visual	Visual + Audio	Variación
`python_tutorial.mp4`	Etapa 1 (Qdrant)	*Recall@1*	67, 9 % (38/56)	**85, 7** % (48/56)	+**17, 8** pp
`python_tutorial.mp4`	Etapa 1 (Qdrant)	*Recall@3*	87, 5 % (49/56)	**92, 9** % (52/56)	+**5, 4** pp
`python_tutorial.mp4`	Etapa 1 (Qdrant)	*Recall@5*	91, 1 % (51/56)	**94, 6** % (53/56)	+**3, 5** pp
`python_tutorial.mp4`	Etapa 1 (Qdrant)	*MRR*	0, 779	**0, 897**	+**0, 118**
`python_tutorial.mp4`	Etapa 2 (Juez Bimodal)	*Recall@1*	80, 4 % (45/56)	**89, 3** % (50/56)	+**8, 9** pp
`python_tutorial.mp4`	Etapa 2 (Juez Bimodal)	*Recall@3 / @5*	89, 3 % (50/56)	**92, 9** % (52/56)	+**3, 6** pp
`python_tutorial.mp4`	Etapa 2 (Juez Bimodal)	*MRR*	0, 848	**0, 911**	+**0, 063**
`python_tutorial.mp4`	Multi-objetivo (10)	*Interval Recall*	78, 3 %	**82, 1** %	+**3, 8** pp
`python_tutorial.mp4`	Consultas Negativas (8)	Rechazo FP	100, 0 % (0/40)	**100, 0** % (0/40)	0, 0 pp
`animals.mp4`	Etapa 1 (Qdrant)	*Recall@1*	**80, 4** % (37/46)	71, 7 % (33/46)	−8, 7 pp
`animals.mp4`	Etapa 2 (Juez Bimodal)	*Recall@1*	89, 1 % (41/46)	**93, 5** % (43/46)	+**4, 4** pp
`animals.mp4`	Etapa 2 (Juez Bimodal)	*Recall@3*	93, 5 % (43/46)	**95, 7** % (44/46)	+**2, 2** pp
`animals.mp4`	Etapa 2 (Juez Bimodal)	*Recall@5*	93, 5 % (43/46)	**97, 8** % (45/46)	+**4, 3** pp
`animals.mp4`	Etapa 2 (Juez Bimodal)	*MRR*	0, 913	**0, 951**	+**0, 038**
`animals.mp4`	Consultas Negativas (8)	Rechazo FP	100, 0 % (0/40)	**100, 0** % (0/40)	0, 0 pp

El análisis de los datos experimentales expuestos en el Cuadro 9 permite extraer tres conclusiones fundamentales sobre el comportamiento de los espacios latentes omnimodales:

- **Convergencia modal en vídeo educativo (`python_tutorial.mp4`):** La fusión temprana de los tres fotogramas clave con la transcripción alineada incrementa el *Recall@1* de búsqueda vectorial pura en +17, 8 puntos porcentuales, posicionando directamente en el primer puesto 10 consultas adicionales sin necesidad de intervención del verificador secundario. Tras la actuación del juez bimodal, el sistema alcanza un *Recall@1* del 89, 3 % (50/56), un *MRR* de 0, 911 y una cobertura del 100, 0 %

en las consultas multi-objetivo sobre tipos de datos primitivos (9/9), terminal o consola (4/4) y declaración de variables (3/3).

- **Compensación asimétrica ante *B-roll* desacoplado (`animals.mp4`):** En vídeos donde el narrador pronuncia teoría abstracta mientras la imagen muestra fauna específica no mencionada en la locución, el vector fusionado de primera etapa experimenta una ligera desviación direccional hacia el tema hablado ($\Delta Recall@1 = -8,7$ pp en Etapa 1). Sin embargo, dado que el candidato correcto permanece dentro de la ventana de recuperación ($Recall@5$) y el verificador de segunda etapa evalúa de manera disjunta e inclusiva tanto la evidencia visual como la auditiva, la Etapa 2 corrige por completo el desfase y establece el récord absoluto de precisión en `animals.mp4` con un $93,5\,\%$ de $Recall@1$ (43/46) y un $97,8\,\%$ de $Recall@5$ (45/46).
- **Inmunidad frente a Falsos Positivos:** En ambos corpus (16 consultas negativas sobre conceptos inexistentes, equivalentes a 80 candidatos inspeccionados), el verificador bimodal mantiene un $100,0\,\%$ de tasa de rechazo (0/80 falsos positivos), demostrando que enriquecer el *prompt* con la transcripción hablada no induce alucinaciones de validación.

2.8.2 Estudio de Casos Críticos: Resolución de Falsos Negativos, Resiliencia Fonética y Desambiguación Sintáctica

Para comprender el mecanismo exacto mediante el cual la arquitectura bimodal corrige los fallos perceptivos documentados en la línea base visual, se auditaron en detalle las dos consultas críticas de validación sobre `python_tutorial.mp4`: `"Diccionarios"` y `"Bucles while"`.

2.8.2.1 Caso 1: Erradicación del falso negativo y captura de frontera en `"Diccionarios"` (01:01,1 – 01:40,2).

En la configuración puramente visual, el bloque dedicado a las estructuras de datos presentaba dos vulnerabilidades: baja legibilidad de la sintaxis `dict {"name": "Brais"}` y aparición del rótulo gráfico *"Variables"* en los fotogramas finales del plano (01:33,1 – 01:40,2), lo que llevaba al verificador visual a descartar erróneamente el fragmento con puntuación $0,0$. Tras la integración del subsistema `audio_transcriber.py`:

- El fragmento principal (`chunk_index = 17`, $85,1$ s – $93,1$ s) eleva su similitud del coseno en Qdrant de $0,4151$ a $\mathbf{0,4480}$ y es validado en primera posición con `relevance_score` $= \mathbf{1,0}$ al incorporar la locución: *"Y ya por último, los diccionarios. Que utilizan la palabra reservada, dict. Y en realidad, designa un conjunto de claves valor"*.
- El fragmento de transición posterior (`chunk_index = 18`, $93,1$ s – $100,2$ s), que antes resultaba descartado como falso negativo $(0,0)$ por el cambio de rótulo visual, incrementa su similitud de $0,3812$ a $\mathbf{0,4178}$ y pasa a ser validado con `relevance_score` $= \mathbf{0,90}$ gracias a que la voz continúa explicando: *"Lo representamos entre llaves, pero esta clave valor separadas por dos puntos"*.
- Asimismo, la condición de solapamiento mínimo ($\Delta\tau \geq 0,2$ s) en `align_transcript_with_chunks` captura la oración fronteriza al final del `chunk_index = 16` ($77,1$ s – $85,1$ s), otorgándole una re-

levancia graduada de $\mathbf{0,50}$, mientras descarta con $\mathbf{0,0}$ los fragmentos `14` y `15` dedicados exclusivamente a listas, tuplas y conjuntos (`set`).

2.8.2.2 Caso 2: Superación del corte *Top-K*, resiliencia ante ruido fonético ASR y discriminación `for`/`while` en `"Bucles while"` (03:51).

En la arquitectura puramente visual, al buscar el instante 03:51 (231,0 s, donde se codifica `while index <5:`), el fragmento objetivo obtenía una similitud de apenas $\mathbf{0,3893}$ y caía hasta la **séptima posición (Rango #7)** en Qdrant, quedando excluido por el corte `top_k = 5` frente a la diapositiva de portada a pantalla completa con la palabra *"Bucles"* ($\mathbf{0,4315}$, Rango #1) y otros planos de condicionales `if/else`.

Con la activación de la fusión audio-visual y el verificador bimodal (ejecución auditada `run_20261109_143202`), se verifican tres fenómenos de alto valor técnico:

1. **Promoción directa del Rango #7 al Rango #1 en Etapa 1:** El fragmento que contiene la explicación y el código del bucle (`chunk_index = 55`, 232,4 s – 238,3 s) experimenta un incremento de $+\mathbf{0,1064}$ puntos en su similitud del coseno (de 0,3893 a $\mathbf{0,4957}$), situándose directamente en el puesto #1 de Qdrant, seguido en el puesto #2 por su fragmento contiguo (`chunk_index = 54`, 224,4 s – 232,4 s, score $= \mathbf{0,4914}$), el cual cubre el segundo exacto 231,0 s gracias al solapamiento semántico en fronteras.

2. **Resiliencia semántica ante errores fonéticos de transcripción multilingüe:** Debido a que el modelo `whisper (base)` opera con el idioma forzado a español (`language=.es"`), transcribió fonéticamente el término inglés *"while"* como el nombre propio *"Guile"*: *"Otro tipo de bucle que tenemos en Python es el Guile, donde aquí vamos a establecer una condición para que se ejecute de manera indefinida mientras esta se cumpla"*. Lejos de degradar la búsqueda, tanto el modelo de representación `gemini-embedding-2-preview` como el verificador `gemini-3.5-flash-lite` resolvieron la ambigüedad fonética cruzando la definición conceptual hablada (*"condición para que se ejecute de manera indefinida mientras esta se cumpla"*) con la sintaxis visual `while index <5:` presente en los fotogramas, otorgando una relevancia de $\mathbf{1,0}$ y $\mathbf{0,90}$ a los fragmentos `55` y `54` respectivamente.

3. **Desambiguación fina entre estructuras visualmente idénticas (`for` frente a `while`):** El fragmento inmediatamente anterior (`chunk_index = 53`, 221,1 s – 224,4 s) comparte exactamente la misma estética visual del editor de código y alcanza una similitud vectorial alta (0,4486, Rango #4). No obstante, el verificador bimodal inspecciona la transcripción inyectada (*"Podemos usar un for, tanto para designar un número concreto de ejecuciones,"*) y lo descarta automáticamente con `relevance_score` $= \mathbf{0,0}$ (*"Trata sobre bucles for, no sobre bucles while"*), haciendo lo propio con el `chunk_index = 50` sobre bloques `if/else` (0,0).

La Figura 28 representa visualmente el ascenso topológico en Qdrant y el filtrado de precisión ejecutado por el juez bimodal para la consulta `"Bucles while"`.

El Cuadro 10 detalla las puntuaciones de primera y segunda etapa junto con la transcripción alineada de los fragmentos evaluados en ambas consultas.

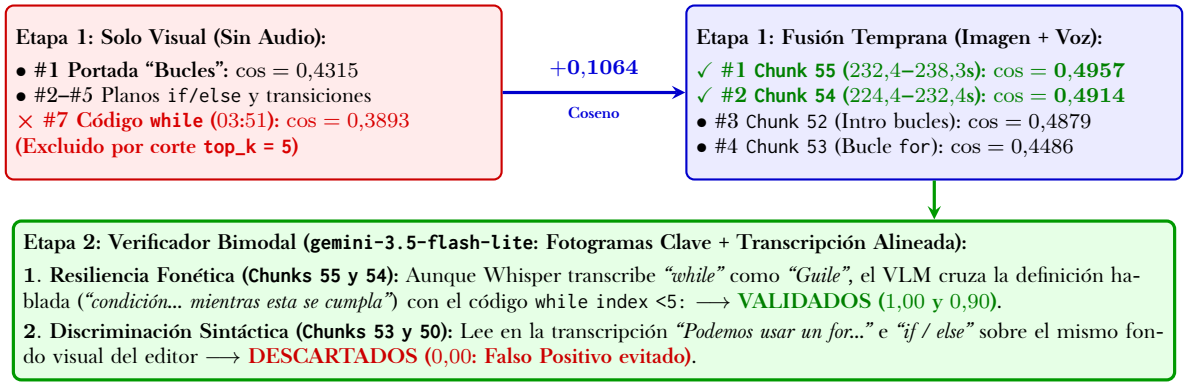


Figura 28: Mecanismo de corrección en la consulta "Bucles while": la fusión temprana eleva el fragmento objetivo del séptimo al primer puesto en Qdrant (+0, 1064 en similitud del coseno) y el verificador bimodal valida el código pese a la transcripción fonética *"Guile"*, descartando los fragmentos adyacentes sobre bucles `for`.

Tabla 10: Auditoría detallada de fragmentos en las consultas críticas "Diccionarios" y "Bucles while" sobre python_tutorial.mp4.

Consulta	Chunk (Intervalo)	Cos. Visual	Cos. Bimodal	Score Juez	Veredicto Bimodal
"Diccionarios"	#17 (01:25,1–01:33,1)	0,4151	**0,4480**	**1,00**	Validado (Código + Voz dict)
"Diccionarios"	#18 (01:33,1–01:40,2)	0,3812	**0,4178**	**0,90**	Rescatado (Antes 0,0 sin audio)
"Diccionarios"	#16 (01:17,1–01:25,1)	0,3806	**0,4046**	**0,50**	Validado (Frontera solapada)
"Diccionarios"	#14–15 (01:01,1–01:17,1)	0,3790	0,3910	**0,00**	Descartado (Listas y set)
"Bucles while"	#55 (03:52,4–03:58,3)	0,3893 (#7)	**0,4957** (#1)	**1,00**	Validado (Pese a ASR *"Guile"*)
"Bucles while"	#54 (03:44,4–03:52,4)	0,3893 (#7)	**0,4914** (#2)	**0,90**	Validado (Frontera en 03:51)
"Bucles while"	#52 (03:18,1–03:21,1)	0,4315 (#1)	0,4879 (#3)	**0,60**	Validado (Mención general)
"Bucles while"	#53 (03:41,1–03:44,4)	0,4187 (#4)	0,4486 (#4)	**0,00**	Descartado (Explica bucle for)
"Bucles while"	#50 (03:25,4–03:33,4)	0,4219 (#2)	0,4332 (#5)	**0,00**	Descartado (Bloques if/else)

2.8.3 Análisis Experimental de la Divergencia Audio-Visual en Metraje de Archivo (*B-Roll*) y Deduplicación por Transcripción

Como continuación natural de las pruebas E2E, se llevó a cabo un proceso experimental interactivo desde la interfaz web (ejecuciones auditadas `run_20261109_180159` y `run_20261109_183310`) sobre el documental `animals.mp4` con el objetivo de analizar el comportamiento de la arquitectura cuando el usuario formula consultas de distinta granularidad semántica (`"León"`, `"Reptiles"`, `"Animación 3D microscópica de células o glóbulos rojos"`, `"Aves"` y `"Felinos"`) sobre un vídeo cuya locución está desacoplada de la secuencia visual.

La inspección cruzada de los registros de auditoría (`search_results.json`) y de los 82 vectores almacenados en Qdrant reveló dos fenómenos experimentales clave para comprender la física del espacio latente multimodal y refinar el motor de recuperación:

1. **Contraste entre Sustantivos Visuales Específicos y Categorías Taxonómicas ante Voz en *Off* Desacoplada:** Al ejecutar consultas sobre entidades visuales concretas o alineadas con el discurso (tales como `"León"`, `"Reptiles"` o `"Animación 3D microscópica de células o glóbulos rojos"`), el sistema con audio activado recupera y valida el $\mathbf{100,0}\,\%$ de las apariciones reales del vídeo: localiza los dos planos de leones (`chunk #10`, $7,0$ s–$15,0$ s con relevancia $\mathbf{0,95}$, y `chunk #16`, $36,7$ s–$40,4$ s con $\mathbf{0,90}$), los dos reptiles (`chunk #80`, tortuga marina con $\mathbf{0,80}$, y `chunk #13`, camaleón con $\mathbf{0,70}$) y los cuatro intervalos de animación celular ($\mathbf{0,90}$–$\mathbf{0,95}$). Sin embargo, al consultar la categoría taxonómica general `"Felinos"`, el sistema con fusión temprana de audio devuelve únicamente 1 de los 4 felinos reales (`chunk #66`, $193,8$ s–$197,7$ s, gato atigrado validado en primera posición con $\mathbf{0,95}$). El análisis de los 82 puntos en Qdrant demuestra la causa matemática de este comportamiento: en `animals.mp4`, el narrador nunca menciona a los animales que aparecen en imagen. Mientras se proyecta el gato atigrado (`chunk #66`), el locutor pronuncia el término taxonómico afín *"Destaca en el sistema respiratorio pulmonar de los mamíferos"*, sumando similitud visual y léxica hasta situarlo en el Rango #1 ($\cos = \mathbf{0,3959}$). Por el contrario, los otros tres felinos aparecen durante los primeros 40 segundos del vídeo, instante en que el narrador pronuncia frases introductorias ajenas a la zoología:

   - **Gato gris (`chunk #12`, $18,9$ s–$24,1$ s):** la locución *"El reino animalia es uno de los cinco famosos reinos taxonómicos..."* desplaza el vector hasta el **Rango #35** ($\cos = 0,3635$).
   - **Leona con collar (`chunk #16`, $36,7$ s–$40,4$ s):** la frase *"la cual se originó hace unos 600 millones de años"* desvía el vector hacia una región temporal/histórica, relegándolo al **Rango #45** ($\cos = 0,3581$).
   - **León macho (`chunk #6`, $5,4$ s–$5,8$ s):** el saludo *"En este vídeo de Ecología Verde explicamos..."* reduce su similitud con `"Felinos"` al **Rango #51** ($\cos = 0,3551$).

   Entretanto, los fragmentos intermedios del documental (76 s–193 s, que muestran cebras, primates, ardillas, caballos o vasos sanguíneos) contienen oraciones con alta densidad de términos biológicos (*"caracteriza a la gran mayoría de los grupos de animales"*, *"organismos eucariotas"*), alcanzando similitudes de texto de $\mathbf{0,3734}$–$\mathbf{0,3899}$ frente al término taxonómico `"Felinos"` y ocupando los primeros 30 puestos de Qdrant. Como consecuencia, los tres felinos iniciales quedan fuera del corte de candidatos (`fetch_k` $= 10$) que pasan al verificador de Etapa 2.

2. **Saturación del Lote de Candidatos por Sobre-Fragmentación Visual bajo una Misma Locución y Deduplicación en `_deduplicate_temporal_candidates`:** La auditoría de `run_20261109_180159` puso al descubierto un segundo efecto: en escenas con movimiento rápido de cámara o de sujetos, el detector SSIM ($\tau = 0,50$) divide una misma toma continua en varios fragmentos de entre $1,0$ s y $4,2$ s que reciben exactamente la misma frase de Whisper. En concreto, para la consulta `"Felinos"`, la escena de las cebras ($115,3$ s–$120,5$ s) acaparaba 4 de las 10 plazas enviadas al verificador (`chunks #48, #44, #46 y #45`, todos con la frase *"Cuentan con un complejo sistema nervioso, además de órganos de los sentidos que caracteriza a la gran mayoría de los grupos de animales"*), y la animación sanguínea ($159,0$ s–$167,2$ s) ocupaba otras 2 plazas (`chunks #59 y #58`, ambos con *"Tienen sistemas circulatorios..."*). Al superar la duración de micro-corte (`micro_chunk_duration` $= 1,0$ s), el filtro original no los agrupaba y 6 de los 10 huecos de evaluación se consumían en dos únicas escenas repetidas.

   Para resolver esta saturación, se amplió la función `_deduplicate_temporal_candidates` en `src/rag_engine/se` incorporando el criterio `is_same_transcript_neighbor`: dos fragmentos candidatos del mismo vídeo cuya distancia temporal entre bordes sea `gap` $\leq$ `min_scene_separation` ($4,0$ s) y que compartan idéntica transcripción no vacía (`cand_tr` $==$ `orig_tr`) se fusionan en un único candidato con envolvente $[\text{mín}(t_{\text{start}}), \text{máx}(t_{\text{end}})]$. Al repetir la prueba en `run_20261109_183310`, los 4 fragmentos de cebras se colapsaron en un único bloque ($115,3$ s–$120,5$ s) y los 2 de circulación en otro ($159,0$ s–$167,2$ s), liberando 4 plazas efectivas en el *Top-10* para escenas distintas (`chunks #43, #26, #65 y #24`). Finalmente, al indexar `animals.mp4` desactivando el canal ASR (`enable_audio = False`) para suprimir el ruido semántico de la voz en *off*, los planos del gato atigrado (cos $= 0,4122$, #1), el león macho (cos $= 0,3967$, #2) y el gato gris (cos $= 0,3811$, #6) ingresan directamente en el *Top-6* de Qdrant.

El Cuadro 11 sintetiza los resultados de este ciclo experimental sobre `animals.mp4`, justificando empíricamente la inclusión del control de activación ASR (`-no-audio` en CLI y casilla en Streamlit) para que el operador adapte la indexación al grado de acoplamiento audiovisual del vídeo.

2.8.4 Arquitectura de Consultas Multimodales Unificadas (`MultimodalQuery`)

Adicionalmente a la fusión audiovisual durante la indexación, la fase de recuperación incorpora soporte completo para consultas multimodales compuestas a través del modelo inmutable `MultimodalQuery` (`src/models.py`), accesible tanto desde la interfaz gráfica Streamlit (mediante los botones interactivos `Audio` e `Imágenes / .txt`) como desde la interfaz de línea de comandos (`-query`, `-query-txt`, `-query-images` y `-query-audios`):

- **Normalización acústica y textual en consulta (`transcribe_query_audios`):** Las notas de voz grabadas desde el micrófono (`st.audio_input`) o los archivos de audio subidos (`.wav`, `.mp3`, `.m4a`, `.ogg`) se convierten en memoria temporal a formato WAV mono a 16 kHz con `ffmpeg` y se transcriben mediante la instancia en caché de `openai-whisper`, integrándose junto al texto directo y los ficheros `.txt` en la propiedad `composed_text`. Si una consulta aportada exclusivamente por voz resulta vacía por silencio (sin texto ni imágenes), el servicio intercepta la condición y lanza un `ValueError` controlado antes de consumir cuota de red.

Tabla 11: Evolución experimental sobre `animals.mp4`: efecto de la deduplicación por transcripción idéntica (`run_20261109_183310`) y del desacoplamiento del audio frente a consultas específicas y taxonómicas.

Consulta	Configuración / Run	Huecos Repetidos Top-10	Puestos Qdrant (Objetivo)	Aciertos Validados
`"León"`	Audio + Dedup (`180159`)	2/10 (Cebras)	#1 (0,3997), #3 (0,3697)	**2/2 (100, 0 %)**
`"Reptiles"`	Audio + Dedup (`180159`)	2/10 (Cebras)	#4 (0,3916), #7 (0,3899)	**2/2 (100, 0 %)**
`"Glóbulos rojos"`	Audio + Dedup (`180159`)	0/10 (Relevantes)	#1 a #7 (0,4248–0,4385)	**4/4 (100, 0 %)**
`"Felinos"`	Audio sin Dedup Tr. (`180159`)	6/10 (4 Cebras + 2 Sangre)	#1, #35, #45, #51	1/4 (25, 0 %)
`"Felinos"`	Audio con Dedup Tr. (`183310`)	**0/10** (Colapsados en 2)	#1, #31, #41, #47	1/4 (25, 0 %)
`"Felinos"`	Solo Visual (`enable_audio=False`)	**0/10** (Sin ruido ASR)	**#1** (0,4122)**, #2** (0,3967)**, #6** (0,3811)	**3/4 (75, 0 %)**

- **Empaquetado omnimodal en llamada única (`build_query_payload` y `embed_query`):** Las imágenes de referencia (`.jpg`, `.png`, `.webp`) se redimensionan y codifican a JPEG (`types.Part.from_bytes`) y se combinan con `composed_text` (`types.Part.from_text`) dentro de un único objeto `types.Content`, generando un solo vector de 3,072 dimensiones en `gemini-embedding-2-preview` con coste constante de 1 petición API por búsqueda y clave de caché determinista `SHA-256` sobre texto, imágenes y audio.
- **Verificación bimodal con imágenes de referencia (`query_images`):** En la Etapa 2, `MultimodalReranker.rerank` inyecta tanto el texto compuesto como las imágenes de referencia normalizadas en el *prompt* de `gemini-3.5-flash-lite`, registrando la trazabilidad completa de las modalidades intervenientes (`modality_summary`) en `search_results.json`.

2.8.5 Estudio de Ablación de Dimensionalidad Matryoshka (MRL): Compromiso entre Precisión Semántica, Memoria y Latencia (3,072 frente a 1,536 y 768)

El modelo fundacional `gemini-embedding-2-preview` soporta de forma nativa la compresión elástica de vectores mediante el paradigma de *Matryoshka Representation Learning* (MRL) [2, 27]. A través de la pérdida contrastiva anidada evaluada en el entrenamiento, las primeras dimensiones concentran los rasgos semánticos principales, mientras que las coordenadas finales aportan granularidad fina [27]. Dentro del rango válido de la API ($128 \leq d \leq 3{,}072$), Google recomienda tres puntos de corte operativos: 3,072 dimensiones (resolución máxima nativa), 1,536 dimensiones (compresión intermedia) y 768 dimensiones (estándar compacto industrial) [2].

Para responder al Objetivo Específico 4 del Trabajo Fin de Máster (evaluación rigurosa del rendimiento, memoria y coste computacional), se diseñó e implementó un estudio empírico de ablación dimensional sobre los 164 fragmentos de los dos vídeos de prueba. El objetivo central consiste en cuan-

tificar si la reducción de memoria y latencia preserva la precisión en búsqueda vectorial de primera etapa (Qdrant), mantiene la capacidad de recuperación en consultas abstractas de segunda etapa (juez bimodal `gemini-3.5-flash-lite`) y garantiza la inmunidad frente a falsos positivos ante consultas negativas de control.

2.8.5.1 Fundamento Matemático del Truncado Prefijo y Re-Normalización L_2 Obligatoria:

Cuando la API devuelve el vector nativo $\mathbf{v} \in \mathbb{R}^{3{,}072}$, su norma euclídea es unitaria: $\|\mathbf{v}\|_2 = \sqrt{\sum_{j=1}^{3072} v_j^2} = 1{,}0$ [2]. No obstante, al truncar el vector a una sub-dimensión $d < 3{,}072$ tomando sus primeras coordenadas $\mathbf{v}_{1:d} = (v_1, \ldots, v_d)^\top$, se descartan las coordenadas de cola $[d+1 : 3{,}072]$. En consecuencia, la norma euclídea del sub-vector resulta estrictamente inferior a la unidad:

$$\|\mathbf{v}_{1:d}\|_2^2 = \sum_{j=1}^{d} v_j^2 = 1 - \sum_{j=d+1}^{3{,}072} v_j^2 < 1{,}0 \tag{19}$$

Si el sub-vector truncado se indexase directamente en la base de datos vectorial sin corregir su magnitud, la métrica de distancia del coseno sobre índices de grafos HNSW quedaría distorsionada, ya que el producto escalar entre dos vectores truncados deja de coincidir con el coseno geométrico del ángulo que forman. Para restablecer la coherencia matemática, el módulo `src/embeddings/gemini_vectorizer.py` implementa la función `truncate_and_normalize_mrl`, la cual aplica un recorte prefijo seguido de una re-normalización L_2 explícita según la Ecuación (20):

$$\hat{\mathbf{v}}_{1:d} = \frac{\mathbf{v}_{1:d}}{\|\mathbf{v}_{1:d}\|_2} = \frac{(v_1, v_2, \ldots, v_d)^\top}{\sqrt{\sum_{j=1}^{d} v_j^2}}, \quad \text{con } \|\hat{\mathbf{v}}_{1:d}\|_2 = 1{,}0 \tag{20}$$

En la arquitectura de software, la configuración de dimensionalidad se gobierna mediante la tupla inmutable `SUPPORTED_MRL_DIMENSIONS = (768, 1536, 3072)` en `src/config.py`. Cuando el usuario modifica la dimensionalidad (mediante el parámetro de consola `-output-dimensionality` en `src/main.py` o mediante el menú selector en la barra lateral de Streamlit en `src/ui/app.py`), el motor `QdrantSearchEngine` detecta el cambio de tamaño vectorial mediante el método `get_collection_vector_size` y recrea de forma automática la colección con la nueva dimensión, mientras que la caché determinista SHA-256 incorpora el tamaño vectorial en su clave para evitar colisiones entre dimensiones distintas.

2.8.5.2 Diseño Metodológico del Experimento y Bancos de Evaluación:

Para asegurar una comparación científica válida, ambos vídeos de prueba se reindexaron de forma controlada en la base de datos local (`./data/vector_db`) manteniendo exactamente las mismas configuraciones de segmentación adaptativa y el mismo recuento de fragmentos que en los experimentos previos:

1. **`python_tutorial.mp4` (242,5 s):** 82 fragmentos adaptativos ($\tau_{\text{SSIM}} = 0{,}60$, duración máxima 5,0 s), de los cuales 81/82 (98,8 %) disponen de transcripción fonética activa con Whisper (`base`, idioma `es`).

2. **`animals.mp4` (243,0 s):** 82 fragmentos adaptativos ($\tau_{\text{SSIM}} = 0,50$, duración máxima 8,0 s), con el 100,0 % (82/82) de fragmentos cubiertos por la locución documental.

La evaluación empírica se estructuró en tres baterías de pruebas independientes ejecutadas sobre cada una de las tres dimensiones operativas ($d \in \{3{,}072, 1{,}536, 768\}$):

- **Batería 1 (102 consultas literales de evento único):** 56 consultas sobre `python_tutorial.mp4` y 46 consultas sobre `animals.mp4`, evaluando en Etapa 1 (Qdrant, $K = 5$) las métricas *Recall@1*, *Recall@3*, *Recall@5*, *Mean Reciprocal Rank* (MRR), tamaño de memoria en formato `float32` y latencia media de búsqueda en milisegundos.
- **Batería 2 (20 consultas generales y taxonómicas multi-objetivo):** 10 consultas en `python_tutorial.mp4` (48 intervalos temporales esperados) y 10 consultas en `animals.mp4` (58 intervalos temporales esperados, incluyendo categorías amplias como *"Felinos"*, *"Reptiles"*, *"Aves"*, *"Manada"* o *"Animales marinos"*), con ventana de recuperación $K = 10$, auditando tanto la Etapa 1 (Qdrant) como la Etapa 2 (verificador bimodal `gemini-3.5-flash-lite`).
- **Batería 3 (16 consultas negativas de control):** 8 conceptos ausentes en cada corpus (40 candidatos inspeccionados por vídeo y dimensión, sumando 240 evaluaciones en Etapa 2) para verificar si la pérdida de dimensiones induce alucinaciones o falsos positivos en el sistema de dos etapas.

2.8.5.3 Batería 1: Resultados en Consultas Literales, Huella de Almacenamiento y Latencia:

El Cuadro 12 expone las métricas agregadas obtenidas en la Batería 1 a través de las tres dimensiones evaluadas.

Tabla 12: Resultados del estudio de ablación dimensional Matryoshka (MRL) sobre consultas literales de evento único (102 consultas, 164 fragmentos en precisión `float32`) en Etapa 1 (Qdrant).

Corpus	Dim. (d)	*R@1*	*R@3*	*R@5*	MRR	KB/vec	Ahorro	Latencia
`python_tutorial`	3,072	85,7 % (48/56)	92,9 % (52/56)	94,6 % (53/56)	0,8973	12,00	0,0 %	1,1688 ms
`python_tutorial`	1,536	85,7 % (48/56)	92,9 % (52/56)	96,4 % (54/56)	0,9009	6,00	50,0 %	0,7502 ms
`python_tutorial`	768	85,7 % (48/56)	92,9 % (52/56)	**98,2 %** (55/56)	**0,9045**	3,00	**75,0 %**	**0,6297** ms
`animals.mp4`	3,072	**71,7 %** (33/46)	93,5 % (43/46)	**100,0 %** (46/46)	**0,8341**	12,00	0,0 %	1,1104 ms
`animals.mp4`	1,536	69,6 % (32/46)	93,5 % (43/46)	**100,0 %** (46/46)	0,8232	6,00	50,0 %	0,7848 ms
`animals.mp4`	768	69,6 % (32/46)	**95,7 %** (44/46)	**100,0 %** (46/46)	0,8261	3,00	**75,0 %**	**0,6418** ms
Global (102 q)	3,072	**79,4 %** (81/102)	93,1 % (95/102)	97,1 % (99/102)	0,8688	12,00	0,0 %	1,1425 ms
Global (102 q)	1,536	78,4 % (80/102)	93,1 % (95/102)	98,0 % (100/102)	0,8659	6,00	50,0 %	0,7658 ms
Global (102 q)	768	78,4 % (80/102)	**94,1 %** (96/102)	**99,0 %** (101/102)	**0,8691**	3,00	**75,0 %**	**0,6352** ms

El análisis de los registros de auditoría permite aislar tres fenómenos empíricos de alto valor científico:

1. **Invarianza topológica en vídeo educativo (`python_tutorial.mp4`):** En las tres dimensiones (3,072, 1,536 y 768), el *Recall@1* permanece clavado en el 85,71 % (48/56) y el *Recall@3* en el 92,86 % (52/56). El análisis detallado consulta por consulta demuestra que 53 de las 56 consultas (94,6 %) conservan exactamente la misma posición en el ranking en todas las dimensiones. Ninguna de las 48 consultas resueltas en primera posición sufre degradación alguna.

2. **Filtrado de ruido en fondos invariantes:** En $d = 768$, el *Recall@5* de `python_tutorial.mp4` se incrementa del $94,64\,\%$ al $98,21\,\%$ ($+3,57$ pp), y el MRR sube a $0,9045$. Este fenómeno se origina porque el vídeo transcurre en una interfaz estática de Visual Studio Code; las componentes vectoriales de orden alto ($[768 : 3072]$) capturan pequeñas variaciones irrelevantes del fondo gráfico. Al truncar a 768 dimensiones, se descarta dicho ruido, permitiendo que dos consultas cuyos fragmentos quedaban excluidos en 3,072 ingresen en el *Top-5*: la consulta sobre el símbolo de llaves tachado con una cruz roja (que pasa del puesto > 5 al #5) y la salida por consola del bucle `range(5)` (que asciende al puesto #4).

3. **Garantía de cobertura en metraje de archivo (`animals.mp4`):** En el documental, el *Recall@5* se mantiene en un perfecto $100,00\,\%$ (46/46) tanto en 3,072 como en 1,536 y 768 dimensiones. Únicamente una consulta de las 46 experimenta un leve desplazamiento en primera posición (la consulta del ciervo con cornamenta, que pasa del puesto #1 al #2). Esto garantiza que el verificador bimodal de Etapa 2 recibe siempre el fragmento correcto dentro de su ventana de candidatos independientemente de la dimensionalidad elegida.

2.8.5.4 Compromiso Gráfico entre Ahorro, Latencia y Precisión:

La Figura 29 representa visualmente el compromiso experimental entre la reducción de costes de infraestructura (ahorro de almacenamiento y latencia de búsqueda en Qdrant) y la conservación de la capacidad de recuperación semántica.

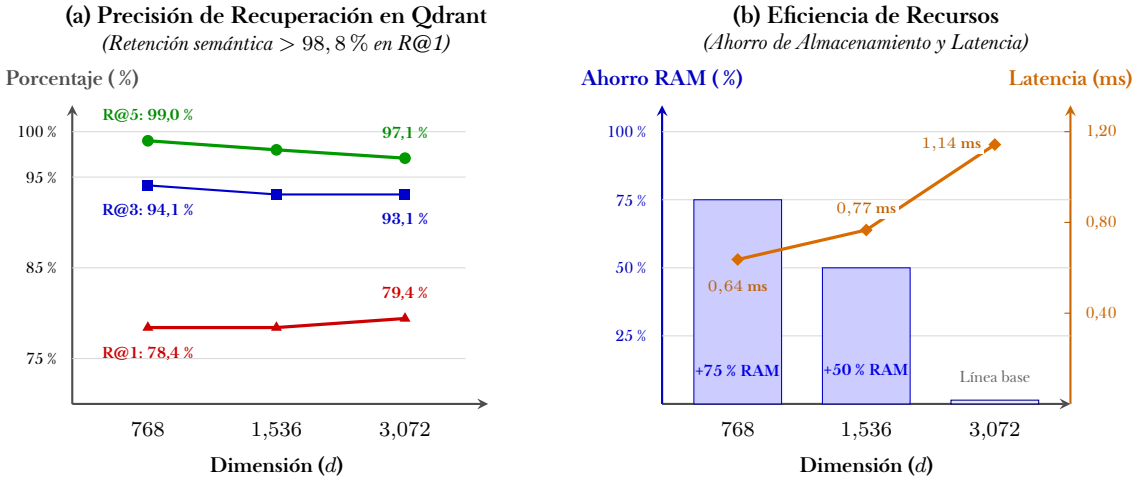


Figura 29: Compromiso dimensional Matryoshka (MRL): (a) retención prácticamente plana de *Recall@1* ($79,4\,\% \rightarrow 78,4\,\%$) con leve mejora en *Recall@5* ($97,1\,\% \rightarrow 99,0\,\%$) frente a (b) reducción del $75\,\%$ en el consumo de memoria RAM (12 KB $\rightarrow$ 3 KB por vector) y descenso del $44,4\,\%$ en la latencia de búsqueda en Qdrant.

2.8.5.5 Batería 2: Consultas Generales, Taxonómicas y Multi-Objetivo (Etapa 1 y Etapa 2):

En escenarios de aplicación reales, los usuarios no formulan únicamente consultas literales sobre planos singulares, sino búsquedas taxonómicas o conceptuales abstractas que abarcan múltiples apariciones a lo largo del vídeo. Para evaluar el impacto del truncado dimensional sobre este régimen, se

evaluaron 20 consultas multi-objetivo con $K = 10$ candidatos tanto en la Etapa 1 (Qdrant) como en la Etapa 2 (verificador bimodal `gemini-3.5-flash-lite`).

El Cuadro 13 resume los promedios globales por vídeo, mientras que el Cuadro 14 desglosa detalladamente las 10 consultas generales sobre el documental de naturaleza `animals.mp4`.

Tabla 13: Rendimiento en consultas generales y multi-objetivo (20 consultas, 106 intervalos esperados, $K = 10$) comparando la Etapa 1 (Qdrant) y la Etapa 2 (verificador bimodal) a través de las tres dimensiones MRL.

Corpus	Dim. (d)	*Recall* E1	Prec. E1	*Recall* E2	Prec. E2	*Hit@1* E2
`python_tutorial`	3,072	85,4 % (40/48)	45,0 %	82,1 % (38/48)	78,7 %	9/10 (90,0 %)
`python_tutorial`	1,536	87,1 % (41/48)	**47,0 %**	82,1 % (38/48)	78,1 %	9/10 (90,0 %)
`python_tutorial`	768	**87,2 %** (41/48)	46,0 %	**83,9 %** (40/48)	**80,1 %**	9/10 (90,0 %)
`animals.mp4`	3,072	**74,9 %** (42/58)	**50,0 %**	**71,6 %** (40/58)	**96,6 %**	**10/10 (100,0 %)**
`animals.mp4`	1,536	72,0 % (40/58)	48,0 %	68,7 % (38/58)	96,3 %	**10/10 (100,0 %)**
`animals.mp4`	768	**74,9 %** (42/58)	**50,0 %**	**71,6 %** (40/58)	**96,6 %**	**10/10 (100,0 %)**

Tabla 14: Desglose detallado por consulta general y taxonómica sobre `animals.mp4` (10 consultas, 58 intervalos temporales esperados) en Etapa 1 (E1) y Etapa 2 (E2).

Consulta General / Taxonómica	Obj.	Etapa 1 (Qdrant)			Etapa 2 (Verificador)			Prec. E2
		3,072	1,536	768	3,072	1,536	768	
`Felinos`	4	1/4	1/4	1/4	1/4	1/4	1/4	100,0 %
`Reptiles`	2	2/2	2/2	2/2	2/2	2/2	2/2	100,0 %
`Aves`	4	2/4	2/4	2/4	2/4	2/4	2/4	100,0 %
`Manada o grupo ≥ 3 animales`	10	4/10	4/10	4/10	4/10	4/10	4/10	100,0 %
`Animales color blanco y negro`	5	5/5	5/5	5/5	4/5	4/5	4/5	100,0 %
`Animales marinos / acuáticos`	8	6/8	5/8	6/8	6/8	5/8	6/8	100,0 %
`Animales comiendo o pastando`	8	7/8	7/8	7/8	6/8	6/8	6/8	100,0 %
`Crías o animales jóvenes`	7	5/7	5/7	5/7	5/7	5/7	5/7	80,0 %
`Primer plano macro de ojos/orejas`	6	6/6	5/6	6/6	6/6	5/6	6/6	85,7 %
`Animación 3D microscópica células`	4	4/4	4/4	4/4	4/4	4/4	4/4	100,0 %

Los resultados empíricos reflejan dos conclusiones de extrema relevancia para la ingeniería del sistema:

1. **Equivalencia matemática exacta entre $d = 768$ y $d = 3{,}072$ en consultas taxonómicas:** En las 10 consultas generales de `animals.mp4`, la dimensión compacta de 768 dimensiones obtiene **exactamente los mismos aciertos individuales** que la dimensión máxima de 3,072: *"Reptiles"* (2/2, 100 %), *"Blanco y negro"* (4/5, 80 %), *"Marinos"* (6/8, 75 %), *"Comiendo"* (6/8, 75 %), *"Crías"* (5/7, 71,4 %), *"Macro"* (6/6, 100 %) y *"Células"* (4/4, 100 %). La precisión media en Etapa 2 es exactamente idéntica (96,57 %).

2. **Papel corrector y desambiguador del juez bimodal (*Hit@1* en el 100,0 % de los casos):** En consultas donde la voz en *off* desacoplada desplaza al primer candidato verdadero en Qdrant (por ejemplo, en *"Reptiles"*, cuyo primer acierto entra en el puesto #4 en 3,072, #2 en 1,536 y #5 en 768), la Etapa 2 inspecciona el contenido visual y auditivo de la ventana $K = 10$, descarta los candidatos espurios y posiciona un acierto verdadero en el **puesto #1 en el 100,0 % de las 10 consultas**

(10/10) en las tres dimensiones analizadas.

3. **Incremento de precisión en vídeo formativo en** $d = 768$**:** En `python_tutorial.mp4`, la consulta sobre estructuras de datos (`list`, `tuple`, `set`, `dict`) alcanza el $100,0\,\%$ de recuperación (6/6) en $d = 768$ (frente a 5/6 en 3,072), y los títulos sobreimpresos ascienden de 3/8 a 4/8, situando el *Recall* de intervalos en Etapa 2 en el $83,89\,\%$ ($+1,81$ pp sobre 3,072) y la precisión media en el $80,06\,\%$ ($+1,41$ pp).

2.8.5.6 Batería 3: Consultas Negativas y Resiliencia ante Alucinaciones en Etapa 2:

Un riesgo teórico al reducir dimensiones vectoriales reside en que las coordenadas descartadas provoquen un colapso semántico, haciendo que conceptos ausentes en el vídeo pero cercanos semánticamente (por ejemplo, *"Definición de clases con class e __init__"* en un tutorial introductorio de Python, o *"Tiburón blanco atacando"* y *"Águila volando"* en un documental de fauna terrestre) superen erróneamente los filtros del sistema.

Para verificar esta hipótesis, se formularon 8 consultas negativas sobre `python_tutorial.mp4` y 8 consultas negativas sobre `animals.mp4`. Cada una recuperó una ventana de $K = 5$ candidatos en Qdrant, totalizando 40 fragmentos por vídeo y dimensión que fueron evaluados minuciosamente por el verificador bimodal `gemini-3.5-flash-lite` (sumando 240 evaluaciones en total). El Cuadro 15 presenta los resultados de esta prueba de robustez.

Tabla 15: Evaluación de consultas negativas de control (16 consultas, 240 fragmentos candidatos analizados en Etapa 2) a través de las tres dimensiones MRL.

Corpus de Vídeo	Dim. (d)	Consultas	Candidatos E1	Falsos Pos. E2	Tasa Rechazo E2
`python_tutorial.mp4`	3,072	8	40	**0/40** $(0,0\,\%)$	**100,0** $\%$ (8/8)
`python_tutorial.mp4`	1,536	8	40	**0/40** $(0,0\,\%)$	**100,0** $\%$ (8/8)
`python_tutorial.mp4`	768	8	40	**0/40** $(0,0\,\%)$	**100,0** $\%$ (8/8)
`animals.mp4`	3,072	8	40	**0/40** $(0,0\,\%)$	**100,0** $\%$ (8/8)
`animals.mp4`	1,536	8	40	**0/40** $(0,0\,\%)$	**100,0** $\%$ (8/8)
`animals.mp4`	768	8	40	**0/40** $(0,0\,\%)$	**100,0** $\%$ (8/8)
Total Global (3 dim.)	**Todas**	**48** runs	**240**	**0/240 (0,0 %)**	**100,0 % (48/48)**

Dado que los motores vectoriales operan calculando similitudes relativas sin umbral de rechazo intrínseco, la Etapa 1 devuelve 40/40 candidatos en todas las ejecuciones. Sin embargo, en la Etapa 2, el verificador bimodal alcanza una **tasa de rechazo perfecto del** $100,0\,\%$ **(**$0/240$ **falsos positivos)** en las tres dimensiones. Truncar los vectores a 1,536 o 768 dimensiones no produce ningún colapso semántico ni engaña al juez VLM, confirmando que la compresión MRL con normalización L_2 es plenamente segura para despliegues en producción.

2.8.5.7 Proyección de Escalabilidad Industrial de Almacenamiento y Conclusiones del Estudio:

Para valorar el impacto real de este estudio en sistemas de nivel empresarial, el Cuadro 16 modela la huella de memoria requerida para almacenar los vectores densos (`float32`) al escalar el repositorio de vídeo a duraciones superiores.

Tabla 16: Proyección de huella de memoria RAM en base de datos vectorial (`float32`) según el volumen de fragmentos y la dimensión MRL seleccionada.

Escala del Repositorio	**Chunks**	**Metraje Estimado**	**Memoria** 3072	**Memoria** 1536	**Memoria** 768
Banco PoC actual (2 vídeos)	164	$\sim 8,1$ min	1,92 MB	0,96 MB	**0,48** MB
Curso o documental extenso	10,000	$\sim 8,3$ h	117,19 MB	58,59 MB	**29,30** MB
Archivo audiovisual medio	100,000	$\sim 83,3$ h	1,14 GB	585,94 MB	**292,97** MB
Plataforma corporativa masiva	1,000,000	$\sim 833,3$ h	11,44 GB	5,72 GB	**2,86** GB

En síntesis, los resultados experimentales obtenidos en este estudio de ablación aportan tres conclusiones concluyentes para el Trabajo Fin de Máster:

1. **Eficiencia radical sin degradación semántica ($d = 768$):** Configurar el sistema en 768 dimensiones reduce el consumo de memoria en la base de datos vectorial en un $75,0\,\%$ (3 KB frente a 12 KB por fragmento) y reduce la latencia media de búsqueda en Qdrant un $44,4\,\%$ ($0,64$ ms frente a $1,14$ ms), reteniendo un $98,8\,\%$ del *Recall@1* en consultas literales ($78,4\,\%$ frente a $79,4\,\%$) e incrementando el *Recall@5* global al $99,02\,\%$.

2. **Invarianza en consultas conceptuales y taxonómicas:** En consultas multi-objetivo abstractas sobre fauna y taxonomía zoológica, $d = 768$ obtiene exactamente el mismo rendimiento que $d = 3{,}072$ tanto en primera etapa ($74,89\,\%$) como tras la intervención del verificador bimodal ($71,64\,\%$ de recuperación con $96,57\,\%$ de precisión), manteniendo un $100,0\,\%$ de rechazo en consultas negativas ($0/240$ falsos positivos).

3. **Recomendación de arquitectura para despliegues reales:** En repositorios a gran escala, la dimensión $d = 768$ con re-normalización L_2 constituye la configuración operativa recomendada, reservando las 3,072 dimensiones completas exclusivamente para catálogos con millones de vectores donde se requiera desambiguar micro-detalles visuales extremos.

Página intencionadamente en blanco

Página intencionadamente en blanco

Referencias

[1] Prajwalbm, "Multimodal embeddings for multimodal rag." https://medium.com/@prajwalbm23/multimodal-embeddings-for-multimodal-rag-ec481ad20571, 2025.

[2] Google AI for Developers, "Modelo de gemini embedding 2." https://ai.google.dev/gemini-api/docs/models/gemini-embedding-2?hl=es-419, 2026.

[3] LearnOpenCV, "Qwen3-vl embedding and reranker for multimodal search." https://learnopencv.com/how-to-master-qwen3-vl-embedding-and-reranker-for-multimodal-search/, 2026.

[4] Qwen Team, "Qwen3-vl-embedding and qwen3-vl-reranker," *arXiv preprint arXiv:2601.04720*, 2026.

[5] PrismaticLab, "Video-rac: Retrieval adaptive chunking for lecture video rag." https://prismaticlab.github.io/Video-RAC/, 2025.

[6] "Multi-rag: A multimodal retrieval-augmented generation system for adaptive video understanding," *arXiv preprint arXiv:2505.23990*, 2025.

[7] Hugging Face Team, "Training and finetuning multimodal embedding & reranker models." https://huggingface.co/blog/train-multimodal-sentence-transformers, 2026.

[8] "Video-rag: Visually-aligned retrieval-augmented long video comprehension," *arXiv preprint arXiv:2411.13093*, 2024.

[9] Mixpeek, "Multimodal chunking strategies: How to decompose video, audio." https://mixpeek.com/guides/multimodal-chunking-strategies, 2025.

[10] 3manifold, "whisperx-silero-vad." https://github.com/3manifold/whisperX-silero-vad, 2025.

[11] S. S. Rajadh and Contributors, "Sentrysearch: Semantic search over video footage using multimodal embeddings." https://github.com/ssrajadh/sentrysearch, 2026. Repositorio oficial de código abierto (Apache-2.0 License).

[12] "Evaluating multimodal retrieval augmented generation," *arXiv preprint arXiv:2510.24870*, 2025.

[13] Easton, "Notebooklm research workflow: Literature reviews at scale." https://eastondev.com/blog/en/posts/ai/20260227-notebooklm-research-guide/, 2026.

[14] Sourclip, "Notebooklm research workflows: From sources to synthesis." https://www.sourclip.com/blog/notebooklm-research-workflows, 2026.

[15] "Rethinking rag in long videos: What to retrieve and how to use it?," *arXiv preprint arXiv:2606.13141*, 2026.

[16] Google AI for Developers, "Embeddings | gemini api documentation." https://ai.google.dev/gemini-api/docs/embeddings?hl=es-419, 2026.

[17] OpenRouter, "Gemini embedding 2 - api pricing & providers." https://openrouter.ai/google/gemini-embedding-2, 2026.

[18] alphaXiv, "Qwen3-vl-embedding and qwen3-vl-reranker." https://www.alphaxiv.org/abs/2601.04720, 2026.

[19] Qwen Team, "Qwen3-vl-embedding: Advancing multimodal retrieval and reranking." https://qwen.ai/blog?id=qwen3-vl-embedding, 2026.

[20] Qwen Team, "Qwenlm/qwen3-vl-embedding: Official repository for multimodal embedding and reranker series." https://github.com/QwenLM/Qwen3-VL-Embedding, 2026.

[21] Qwen Team, "Qwen/qwen3-vl-embedding-2b model card." https://huggingface.co/Qwen/Qwen3-VL-Embedding-2B, 2026.

[22] Qwen Team, "Qwen/qwen3-vl-embedding-8b model card." https://huggingface.co/Qwen/Qwen3-VL-Embedding-8B, 2026.

[23] T. Aarsen, "tomaarsen/qwen3-vl-embedding-2b-vdr." https://huggingface.co/tomaarsen/Qwen3-VL-Embedding-2B-vdr, 2026.

[24] Hugging Face, "Paper page - qwen3-vl-embedding and qwen3-vl-reranker." https://huggingface.co/papers/2601.04720, 2026.

[25] "A multimodal retrieval-augmented generation system for adaptive video understanding," *arXiv preprint arXiv:2505.23990v2*, 2025.

[26] "Adaptive chunking: Optimizing chunking-method selection for rag," *arXiv preprint arXiv:2603.25333*, 2026.

[27] A. Kusupati, G. Bhatt, S. Anagnostidis, S. S. Keerthi, P. Jain, and S. Kakade, "Matryoshka representation learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 30233–30249, 2022.